Software Requirements Specification

for

Herd Resource Patient Portal

Version 0.4

Prepared by

Group Name: GROUP_03

Colton Gebhard	903028111	gebhard@marshall.edu
Jeffrey Jezewski	903016178	jezewski1@marshall.edu
Kendra Adkins	903059767	adkins1783@marshall.edu
Kirubel Melaku Assefa	903040540	assefa1@marshall.edu
Michael Shawn William Davis	903035163	davis1468@marshall.edu
Keshon Bryant	901786223	bryant181@marshall.edu

Instructor:	**Dr. Tanvir Irfan Chowdhury**
Course:	**CS 310 Software Engineering II**
Date:	**April 16th**

Table of Contents

Revisions

Version	Primary Authors	Description of Versions	Date Completed
PP-0.1	Colton Gebhard Jeffrey Jezewski Kendra Adkinsi Kirubel Melaku Assefa Michael Davis	TASK01 - Initial Drafting of SRS and layout.	01/29/2026
PP-0.2	Colton Gebhard Jeffrey Jezewski Kendra Adkinsi Kirubel Melaku Assefa Michael Davis Keshon Bryant	TASK02 - Writing down diagrams for use cases, classes, and sequences. Adding Project Backlog and Sprint	02/23/2026
PP-0.3	Colton Gebhard Jeffrey Jezewski Kendra Adkinsi Kirubel Melaku Assefa Michael Davis Keshon Bryant	TASK03 - PROTOTYPE PRESENTATION	03/09/2026
PP-0.4	Colton Gebhard Jeffrey Jezewski Kendra Adkinsi Kirubel Melaku Assefa Michael Davis Keshon Bryant	REMOVING UNUSED IMPLEMENTATIONS CODEBASE ADDITION	04/16/2026

1 - Introduction

1.1 Document Purposes

This Software Requirement Specifications document will lay out the functional and non-functional requirements for the Herd Resource Patient Portal. The purpose of this SRS document is to define what the system is expected to do and how it should behave under various conditions. This document will serve as a reference for our group of developers through the system design phases of the project, as well as provide instructions for future testing teams who will overview the functionality of the system and future developers who want to modify the codebase. It will also ensure that future stakeholders will have a shared understanding of the system with the document.

1.2 Product Scope

The Herd Resource Patient Portal is a web-based application that is designed to supply patients with a secure way of accessing their personal healthcare-related information through an online interface. The patient portal is intended to enhance the structure and accessibility of patient data while maintaining ease of use for users, although it will not provide any kind of medical diagnosis or treatment functions. The portal supports basic information management and features within the system. As well, the system allows users to view their non-clinical information, such as their schedules and information they entered. The application is developed using HTML/CSS, Python, MySQL, and the Django web framework.

1.3 Definitions and Acronyms/Abbreviations

BigINT	Big Integer – Data type used for numbers that are larger than 64-bits
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DB	Database
Django	A Python framework for server-side data-handling. Includes request handling and authentication processes.
HTML	Hypertext Markup Language
HRPP	Herd Resource Patient Portal
IDE	Integrated Development Environment
Int	Integer – 32-bit (2^(32-1)) values used to represent a range of values.

MFA	Multi-Factor Authentication
MySQL	My Structured Query Language
Patient Portal	User interface for accessing a dashboard of personal patient information
PPS	Patient Portal System
Python	An interpreted, object-oriented, high-level programming language with dynamic semantics
SQL	Structured Query Language
SRS	Software Requirements Specification
TinyINT	TinyINT – 8-bit (0-255) values used in SQL
UI	User Interface
UUID	Universally Unique Identifier – 128-bit label in computer systems to identify without a central registration authority
VS Code	Virtual Studio Code

1.4 Document Overview and Intended Audience

This document is intended for various groups and audiences that are involved in the development and assessment of the Herd Resource Patient Portal. This SRS is designed to provide an overview of the system foremost, followed by more detailed descriptions of interfaces and functionality. It also serves as the baseline reference for supporting future maintenance and improvements through the system's lifecycle. Individuals who are assigned to test this system will be referencing this document to assess the implementation of code from the document and its accuracy and completeness. Contributors can cross-reference this document to obtain insight into the system's capabilities and limitations.

1.5 Document Conventions

Fonts: Arial
Sizes: 11, 12, 13, 14, 24, 36
Default Text Size: 11
Sub-Sub-Sub Headers Size: 12
Sub-Sub Headers Size: 13
Sub Headers Size: 14
Headers Size: 24
Title Size: 36

2 - Overall Description

2.1 Product Perspective

The Herd Resource Patient Portal is intended as a web-based system that allows patients to access and manage their information relating to their personal healthcare. The system follows a client-server architecture in which users interact with the application through a web browser interface while the database and backend are handled by the server. The portal does not integrate with the external clinical systems in the current project scope; instead, it operates using its own database and application logic. The design choice simplifies the development process while still showcasing the core patient portal functionality. The system is intended to be maintainable for future development processes.

2.2 Product Functionality

The Herd Resource Patient Portal delivers a set of core features that allow users to manage and view their information through an online interface. These functionalities focus on ease of use and organization features.

A. User Registration and Account Creation

   Patients can register for an account with the Patient Portal by providing the requested personal information. The system will then store the provided information from the user securely in the database. This function allows new and existing patients to access the portal. Users may complete registration from any supported device with internet access and a web browser.

B. User Authentication

   Users must log in to the portal with their email and password each time to access the portal. Authentication is currently limited to single-factor verification using a username/email and password. Additional security, such as multifactor authentication are not included in the current project scope.

C. Viewing & Updating Personal Information

   Patients can view information associated with their accounts on their profile pages. This includes account information and insurance data that are linked to the patient's profile, such as their email, name, phone number, and address. The page also displays upcoming appointments and appointment history for reference and data collection purposes. Patients can update their information on their own in the portal. If they notice that any given field is out of date or incorrect when viewing personal information, there will be an edit button that allows them to correct it. This will ensure that important information is up to date and accurate within the Herd Resource Patient Portal System.

D. Viewing Appointments and Schedules

The portal provides patients with access to their past and upcoming appointments. Appointment entries are displayed in an ordered format based on the times and dates. This feature helps patients stay informed and organized.

E. Messaging and Notifications

The portal provides basic internal messaging or notification features. Messages are stored securely and linked to the user account. This feature improves communication efficiency.

F. Session Management

The system runs user sessions to establish secure access to the patient portal. Users can log out of their accounts at any point they want to through the portal's interface. While a user is logged into their accounts, the system starts to track their periods of activity, and if no activity is spotted then the session will become terminated.

G. Error Handling

When the system encounters an error, a message identifying the error as well as relevant next steps (restarting/refreshing) will be displayed for the user on top of the current page. Reliability will be important to the system to ensure patients are able to access their medical information through the portal.

H. Role-Based Access Control

The Patient Portal forces restrictions based on the user roles that are assigned, any unauthorized access will be blocked, Patients, doctors, receptionists, and administrators/developers are granted access to functions that are relevant to their responsibilities.

I. Data Validation/Integrity

The system validates all of the data submitted from the users before storing it in the MySQL database. The validation checks include value constraints and formatting checks, as well as any invalid/insufficient submissions on forms being rejected with feedback. This will ensure that the data will be accurate, as well as reduce the risk of the data being corrupted.

2.3 User Characteristics

The Herd Resource Patient Portal is planned to be used by individuals with various levels of technological experience. The system is designed to be accessible and intuitive to users without advanced technical knowledge.

A. Technical Experience

Users are expected to be familiar with using web browsers and standard web applications, essentially basic experience. They are not required to have any sort of advanced technical skills to use the system.

B. Expected Behavior of Users

Users are expected to input accurate and truthful information. They will use the system to view and manage their data. Users should follow standard security practices such as keeping login credentials private.

C. Security Awareness

Individuals should have their login information kept secure and private, always logging out after they are finished with their accounts. It is the user's responsibility for actions that take place while logged into their accounts. The portal is not designed or responsible for any abuse that takes place if login information is shared.

D. Accessing Requirements

Users must have access to a functioning internet connection. No additional software installations are required. Accessibility considerations are taken into account where possible

E. Cultural Language

Users are expected to at least understand basic written instructions in the portal, written in English.

F. Frequency of Usage

The system is designed to support sporadic usage without any data loss.

2.4 Design and Implementation Constraints

The implementation of the Herd Resource Patient Portal is subject to the listed constraints, which guide the decisions across the project.

A. Visibility of Data

Patients will not be granted direct access to the internal administrative data or schedules. The portal will restrict visibility to only patient-specific info.

B. Payment and Billing

The current implementation of the system does not support any form of online payment processing. Any demonstration of billing-related information that is displayed in the portal is only informational.

C. Access Restrictions

System access is restricted to authorized users only. Sensitive data such as system-level functions is limited to the view of authorized personnel only. As of now, external system access is not supported within the project's current scope. Developer and administrator roles are only for system maintenance and functions, not intended for general use (no UI will be supported for devtools).

D. Scope of Appointment Managing

The appointment-related functionality is bound to viewing and basic requests with the patient portal. Scheduling and cancellation are handled within the system's defined scope and do not supply direct access to any sort of external calendars or external systems.

E. Authentication Framework Constraints

User authentication will use Django session handling together with a custom SQL-backed authentication backend that validates portal users against the MySQL database, verifies bcrypt-hashed passwords and then enforces role-based access through application-defined permission checks.

F. Database Constraints

Patient-related data must be stored inside a MySQL database. Direct access to the database by regular users is not permitted.

G. Logging Limitation

The system will only provide the basic logging tools and functionality, as audit trails will not be included in the current project scope.

H. Security Limitation

Advanced security features such as multifactor authentication are not included within the project’s current scope. Basic authentication and access control are implemented instead.

I. Technology Constraints

Frontend: HTML and CSS
Programming Language: Python (3.x)
Database System: MySQL
Backend Framework: Django
Application is accessed through a web browser.

J. Deviations from Final Project SRS:

The following will be a list of modifications from the Final Project_SR document that was built on and modified:

- There is no online payment or billing processing; any billing-related data is only for testing purposes.
- Advanced security functionalities such as MFA and fingerprinting are not implemented.

- Managing appointments is restricted to basic request functionality and viewing; there is no synchronization to external calendars.
- Detailed logging and audit trails are excluded from the current scope.
- The notification functionality is restricted to system-generated messages without any real-time provider messaging.

2.5 Assumptions and Dependencies

The system assumes users have reliable internet access and a compatible web browser. It assumes a properly configured server environment with MySQL installed. It also assumes users will provide accurate personal information during registration and profile updates. The application depends on database connectivity and server availability.

3 - Specific Requirements

3.1 Functional Requirements

3.1.1 Use Case and Diagram(s) with Detailed Description

Use Case # - Name	1 - Patient Signup / Create Account
Author	Jeffrey Jezewski
Purpose	Allows patient to create an account in the Herd Resource Patient Portal
Requirements/Traceability	FR-Auth-02, FR-Profile-01
Priority	High
Preconditions	● Patient is not logged in ● Patient is on the signup page
Postconditions	● Patient account is created and stored in the database ● Patient is redirected to the login page or logged in automatically
Actors	Patient
Extends	N/A
Flow of Events	Basic Flow: 1. Patient selects “Create Account” 2. Patient enters required information (name, email, password) 3. System validates the input fields 4. System verifies email uniqueness 5. System creates the account in the database 6. System confirms account creation Alternative Flow: 1. Patient enters an email already in use

	2. System displays an error message and requests a different email
Exceptions	If account creation fails, no partial account is stored
Includes	(Other use-case IDS)
Notes/Issues	Password rules must be finalized

Use Case # - Name	2 - Patient Login
Author	Jeffrey Jezewski
Purpose	Allows a patient to securely log into the Herd Resource Patient Portal and access their dashboard
Requirements/Traceability	FR-Auth-01, FR-Session-01
Priority	High
Preconditions	● Patient accounts exists in the system ● Patient is on the login page
Postconditions	● Patient is authenticated
Actors	Patient
Extends	[20 - View Login History]
Flow of Events	Basic Flow: 1. Patient opens the login page 2. Patient enters username/email and password 3. System validates the credentials 4. System creates an authenticated session 5. System redirects the patient to the dashboard

	Alternative Flow: 1. The patient enters incorrect login information 2. The system displays an error message 3. The patient remains on the login page and may retry
Exceptions	● If the database is unavailable, the system displays a service error and logs the issue
Includes	[1 - Patient Signup / Create Account], [13 - Validate User]
Notes/Issues	Decision needed on whether login uses email or username

Use Case # - Name	3 - View Patient Profile
Author	Jeffrey Jezewski
Purpose	Allow a logged-in patient to view their profile information
Requirements/Traceability	FR-Profile-02
Priority	High
Preconditions	● Patient is logged in
Postconditions	● Patient profile information is displayed
Actors	Patient
Extends	N/A

Flow of Events	Basic Flow: 1. Patient selects “Profile” from the dashboard 2. System retrieves patient profile data from the database 3. System displays the patient’s profile information Alternative Flow: 1. If optional fields are missing, the system displays “Not provided”
Exceptions	• If data retrieval fails, the system displays an error message
Includes	[USE CASE #2]
Notes/Issues	Determine which profile fields are required

Use Case # - Name	4- Update Patient Profile
Author	Jeffrey Jezewski
Purpose	Allows patient to update their personal profile information
Requirements/Traceability	FR-Profile-03
Priority	High
Preconditions	• Patient is logged in • Patient is viewing their profile
Postconditions	• Updated profile information is saved to the database • Confirmation message is displayed
Actors	Patient
Extends	N/A

Flow of Events	Basic Flow: 1. Patient selects “Edit Profile” 2. System displays editable profile fields 3. Patient updates information and submits changes 4. System validates the input data 5. System updates the database 6. System displays a success message Alternative Flow: 1. Patient enters invalid data 2. System highlights errors requests correction
Exceptions	• If the update fails, original profile data remains unchanged
Includes	Add/Edit Allergies, Enter Personal Health Info, Update Contact Information
Notes/Issues	Decision needed on whether patient can change their email address

Use Case # - Name	5 - Request Appointment
Author	Jeffrey Jezewski
Purpose	Allows a patient to submit a request for an appointment
Requirements/Traceability	Allows a patient to submit a request for an appointment
Priority	High

Preconditions	• Patient is logged in
Postconditions	• Appointment request is stored with status (Pending) • Patient receives confirmation of the request
Actors	Patient
Extends	N/A
Flow of Events	Basic Flow 1. Patient selects “Appointments” 2. Patient selects “Request Appointment” 3. Patient enters preferred date, time, and reason 4. Patient submits the request 5. System validates required fields 6. System stores the appointment request 7. System confirms submission Alternative Flow 1. Patient submits incomplete information 2. System displays validation errors
Exceptions	• If the request cannot be saved, the system displays an error message
Includes	[19 - Receive Appointment Reminder]
Notes/Issues	Determine whether appointment request require staff approval

Use Case # - Name	6 - View Appointments

Author	Jeffrey Jezewski
Purpose	Allows a patient to view a list of their appointments and appointment request statuses
Requirements/Traceability	FR-Appt-03
Priority	High
Preconditions	● Patient is logged in
Postconditions	● Patient's appointment list and statuses are displayed
Actors	Patient
Extends	N/A
Flow of Events	Basic Flow: 1. Patient selects "Appointments" from the dashboard/menu 2. System retrieves the patients appointments and appointment requests from the database 3. System displays the appointments (date, time, provider if applicable, status). Alternative Flow: 1. Systems finds no appointments or request for the patient 2. System displays "No appointment found" and provides an option to request one.
Exceptions	● If appointment data cannot be retrieved, the system displays an error message

Includes	N/A
Notes/Issues	Decide what statuses will be shown (Pending, Approved, Cancelled)

Use Case # - Name	7 - Cancel Appointment Request
Author	Jeffrey Jezewski
Purpose	Allows a patient to cancel an appointment request that is still pending (or cancel an upcoming appointment if allowed)
Requirements/Traceability	FR-Appt-04
Priority	Medium
Preconditions	● Patient is logged in ● Patient has a pending appointment request OR an upcoming appointment eligible for cancellation
Postconditions	● Appointment request/appointment is marked as “Cancelled”
Actors	Patient
Extends	UC-06-View Appointments
Flow of Events	Basic Flow: 1. Patient views their appointments/requests 2. Patient selects a pending request and clicks “Cancel” 3. System asks the patient to confirm cancellation 4. Patient confirms 5. System updates the status to “Cancelled” in the database

	6. System displays a confirmation message Alternative Flow 1. Patient selects an appointment that cannot be cancelled 2. System displays a message explaining why cancellation is not available
Exceptions	● If the system cannot update the status, it displays an error message and does not change the record
Includes	N/A
Notes/Issues	Decide cancellation rules

Use Case # - Name	8 - View Messages/ Notifications
Author	Jeffrey Jezewski
Purpose	Allows a patient to view messages or notifications sent by the portal
Requirements/Traceability	FR-Msg-01
Priority	Medium
Preconditions	● Patient is logged in
Postconditions	● Patient sees a list of messages/notifications and can open a message to read it
Actors	Patient
Extends	N/A

Flow of Events	Basic Flow: 1. Patient selects “Messages” 2. System retrieves the patient’s messages from the database 3. System displays message in a list 4. Patient selects a message to open it 5. System displays the full message content Alternative Flow: 1. System find no messages for the patient 2. System displays “No messages found”
Exceptions	● If message cannot be retrieved, the system displays an error message
Includes	N/A
Notes/Issues	Decide whether this “messages,””notifications,” or both

Use Case # - Name	9 - Send Message to Staff
Author	Jeffrey Jezewski
Purpose	Allows a patient to send a message to staff through the portal
Requirements/Traceability	FR-Msg-02
Priority	Medium
Preconditions	● Patient is logged in

Postconditions	• Message is stored and marked as sent • Patient receives confirmation that the message was submitted
Actors	Patient
Extends	UC-08-View messages/Notifications
Flow of Events	Basic Flow: 1. Patient selects “Messages And chooses “New Message” 2. Patient enters a subject and message content 3. Patient selects a message category 4. Patient submits the message 5. System validates required fields 6. System stores the message in the database 7. System displays a confirmation message Alternative Flow: 1. Patient submits without subject or message content 2. System displays validation errors and request correction
Exceptions	• If the message cannot be saved, the system displays an error message
Includes	N/A
Notes/Issues	Decide if patients can message a specific staff member or only a general inbox

Use Case # - Name	10 - Patient Logout
Author	Jeffrey Jezewski
Purpose	Allows a patient to log out of the portal and end their authenticated session
Requirements/Traceability	FR-Auth-03, FR-Session-02
Priority	High
Preconditions	• Patient is logged in
Postconditions	• Patient session is terminated • Patient is redirected to the login page
Actors	Patient
Extends	N/A
Flow of Events	Basic Flow: 1. Patient selects “Logout” 2. System terminates the authenticated session 3. System redirects the patient to the login page 4. System displays a logout confirmation Alternative Flow: N/A
Exceptions	• If logout fails, the system still clears the local session and redirects the patient
Includes	N/A
Notes/Issues	Decide whether the portal redirects to a landing page or directly to the login page

Use Case # - Name	11 - Access Patient File
Author	Colton Gebhard
Purpose	Allows an authorized user to access a patient profile stored in the portal's database
Requirements/Traceability	
Priority	High
Preconditions	● Doctor is authenticated in the portal ● Doctor is logged into the portal ● Doctor has permissions to access the patient records ● Patient Records exists in the database
Postconditions	● Patient file is displayed to doctor
Actors	Doctor
Extends	[13 - Validate User]
Flow of Events	Basic Flow: 1. Doctor selects the "Patient File" 2. Doctor enters the patient information 3. System validates the doctor authorization to view records 4. System retrieves the patient data 5. System displays the patient data to the doctor
Exceptions	If the doctor is not authorized, display an error message and log

	If patient record cannot be found, display “No patient found” If the database is unavailable, display an error message.
Includes	[13 - Validate User]
Notes/Issues	Decide what staff roles, beside the doctors, can access the patient files.

Use Case # - Name	12 - Update Patient Profile
Author	Colton Gebhard
Purpose	Allows an authorized user (such as a doctor) to update patient file information within the portal's database
Requirements/Traceability	
Priority	High
Preconditions	• Doctor is authenticated • Patient file is accessible in database • Doctor is authorized to modify the patient file
Postconditions	• Updates saved in the database • Update is logged in a simple text file • System confirms the successful update
Actors	Doctor
Extends	[11 - Access Patient File]
Flow of Events	Basic Flow: 1. Doctor accesses a patient file

	2. Doctor selects "Update Patient File" 3. System displays a list of editable fields that the doctor can edit 4. Doctor enters updated information 5. Doctor selects "Save Fields" 6. System validates input 7. System saves changes to database
Exceptions	If validation fails system highlights fields and prevents saving
Includes	[13 - Validate User]
Notes/Issues	Need to define what will be editable and what will be read-only

Use Case # - Name	13 - Validate User
Author	Colton Gebhard
Purpose	Confirms that the user attempting to access a restricted action is authorized to do
Requirements/Traceability	
Priority	High
Preconditions	• User attempts to access an unauthorized function
Postconditions	• User is denied access unless authorized to do so via roles
Actors	Patient Portal System (Main) | All Users/Staff (Secondary)
Extends	

Flow of Events	Basic Flow: 1. User pleas for access to a restricted function 2. System checks if session is authenticated 3. System checks if the user is authenticated via role permissions to allow or deny 4. System grants permission to the user and continues operation
Exceptions	• If user is not authenticated, send to login page • If user lacks permission, permission denied popup appears
Includes	N/A
Notes/Issues	Confirm the role permissions for patients/doctors/receptionists/administrators

Use Case # - Name	14 - Enter Personal Health Info
Author	Colton Gebhard
Purpose	Allows a patient to update their reported information that is stored in the database portal.
Requirements/Traceability	
Priority	Medium-High
Preconditions	• Patient is logged in • Patient profile exists

Postconditions	● Patient's information is updated in the database ● The patient receives a confirmation of their update.
Actors	Patient (Main) | Patient Portal System (Secondary)
Extends	[13 - Validate User]
Flow of Events	Basic Flow: 1. Patient goes to "Health Information" from the dashboard profile 2. System displays their health info form 3. Patient updates information 4. Patient hits submit 5. System validates input from fields and formatting 6. System stores a new update in the database 7. System confirms update success
Exceptions	If there are fields missing that are required, system will notify patient and refuse to store until they are inputted
Includes	[13 - Validate User]
Notes/Issues	Should fields be verified via patients or by upper management staff?

Use Case # - Name	16 - Session Timeout Handling
Author	Colton Gebhard

Purpose	Protects users by expiring their sessions after a period of inactivity, required to authenticate again.
Requirements/Traceability	
Priority	High
Preconditions	● User is logged in with an active session ● User has been inactive for a duration period to set the inactivity flag
Postconditions	● Session is invalidated ● User redirected to login page shamefully ● User must log in again to reaccess content
Actors	Patient Portal System (Main) | All Users/Staff (Secondary)
Extends	[13 - Validate User]
Flow of Events	Basic Flow: 1. System monitor activity within user session 2. After inactivity limit reaches MAXIMUM threshold, system invalidates session 3. System redirects user to login page 4. User reauthenticates if they want to access content again
Exceptions	If the user tries to submit the current page form after the timeout, the system will reject this and request the user to relogin into their account

Includes	[13 - Validate User]
Notes/Issues	Confirm the timeout duration

Use Case # - Name	17 - View Staff-Message Details
Author	Colton Gebhard
Purpose	Allows a user to open and read content of a message/notification
Requirements/Traceability	
Priority	Medium
Preconditions	● User is logged in ● (Minimum of) One message/notification exists for the user
Postconditions	● Messages and details are displayed
Actors	Doctor (Main) | Patient Portal System (Secondary)
Extends	[13 - Validate User]
Flow of Events	Basic Flow: 1. User goes to "Messages" 2. System displays list of messages 3. User selects a message 4. System retrieves the content of the message 5. System displays the message content to the user

Exceptions	If the message that is selected does not exist, the system will display an error and refresh the list
Includes	N/A
Notes/Issues	Decide if users should be able to delete their messages, and if the database should keep these messages

Use Case # - Name	18 - Update Contact Information
Author	Colton Gebhard
Purpose	Allows a patient to update their contact information
Requirements/Traceability	
Priority	Medium
Preconditions	• Patient is logged in • Patient's profile is in the database
Postconditions	• Contact information is updated within the database • System displays update to the patient
Actors	Patient (Main) | Patient Portal System (Secondary)
Extends	[4 - Update Patient Profile]
Flow of Events	Basic Flow: 1. Patient goes to the profile page 2. Patient selects "Edit Contact Info" 3. System displays contact fields 4. Patient modifies the editable fields

	5. Patient selects "Save" 6. System verifies the new values from patient 7. System stores and updates the new values in the database
Exceptions	If system verification checks fails, system will highlight fields that are invalid and prevent the user from saving
Includes	[13 - Validate User]
Notes/Issues	Need to decide if email changes requires additional verification

Use Case # - Name	19 - Receive Appointment Reminder
Author	Colton Gebhard
Purpose	Notifies a patient of an upcoming appointment they had previously scheduled
Requirements/Traceability	
Priority	Medium
Preconditions	● Patient has a future appointment scheduled ● Notifications are functional and enabled
Postconditions	● Notification about the scheduled appointment is sent to the recipient ● Notification is stored in the messages of the recipient
Actors	Patient Portal System (Main) | Patient (Secondary)

Extends	[13 - Validate User]
Flow of Events	Basic Flow: 1. System views upcoming appointments and their scheduled times 2. System generates a notification reminder for the appointment 3. System sends the notification to the patient's account 4. System stores the notification in the database
Exceptions	If there is no appointment data, then there is no notification If notification is not successfully delivered, the system will retry based on its configuration
Includes	N/A
Notes/Issues	Need to define how long the notifications will appear for

Use Case # - Name	20 - View Login History
Author	Colton Gebhard
Purpose	Allows a patient to review login activity with their account (security reasons)
Requirements/Traceability	
Priority	Medium
Preconditions	● Patient is logged in ● There is an active database entry with the account

Postconditions	• No modification to account data • Recent login history is displayed to patient
Actors	Patient (Main) | Patient Portal System (Secondary)
Extends	[13 - Validate User]
Flow of Events	Basic flow: 1. Patient goes to the account settings page 2. Patient selects "View Login History" 3. System checks if patient's session and permissions are valid 4. System retrieves login records from database 5. System displays login information with time and date
Exceptions	If there are no login records available, system will display "activity not found" If the login records can't be retrieved, give error
Includes	N/A
Notes/Issues	Decide on how many login records should be displayed/logged at once.

Use Case # - Name	23 - Enter Patient Vitals
Author	Kendra Adkins

Purpose	Allows nurse to record patient vital signs during check in or appointment
Requirements/Traceability	
Priority	High
Preconditions	● Nurse is logged into the portal ● Patient has arrived for appointment ● Patient record exists in system
Postconditions	● Vitals are saved to the patients medical record ● Doctor can view vitals during appointment ● System timestamps the entry
Actors	Nurse (Main) | Patient Portal System (Secondary)
Extends	NA
Flow of Events	Basic Flow: 1. Nurse navigates to patient chart 2. Nurse selects “Enter Vitals” 3. System displays vitals entry form 4. Nurse enters blood pressure, temperature, pulse, weight, height 5. Nurse adds any relevant notes 6. Nurse submits vitals 7. System validates entries are within reasonable ranges 8. System saves vitals to patient record with timestamps 9. System displays confirmation
Exceptions	If vitals are outside normal ranges then the system flags a warning (saves but alerts. If required fields are missing, then the system will prompt the nurse to

	complete them. If patient record cant be found, system will display error.
Includes	[13 - Validate User]
Notes/Issues	Should include all common vitals: BP, temp, pulse, 02 sat, weight, and height

Use Case # - Name	24 - Filter Appointments by Date Range
Author	Kendra Adkins
Purpose	Allows a patient to filter their appointment list to only show appointments within a designated date range
Requirements/Traceability	
Priority	Low
Preconditions	• Patient is logged into the portal • Patient is viewing their appointments list • Appointments exist in the system
Postconditions	• Only appointments within a certain date range are displayed • Patient can clear filter to view all appointments again
Actors	Patient (Main) | Patient Portal System (Secondary)
Extends	[6- View Appointments]
Flow of Events	Basic Flow: 1. Patient is viewing appointment list 2. Patient selects the “Filter by date” option

	3. Patient enters start date and end date 4. System queries database for appointments in that range 5. System displays the filtered appointment list
Exceptions	● An invalid date range (end before start) makes the system prompt for valid dates ● If there are no appointments in the range then the system displays “No appointments found for selected dates”
Includes	[13 - Validate User]
Notes/Issues	This should include an option to quickly select “This week”, “This month”, and “Last 6 months”

Use Case # - Name	25 - View Dashboard Summary
Author	Kendra Adkins
Purpose	Provides a patient with an overview of their upcoming appointments, recent messages, and important alerts on their main dashboard
Requirements/Traceability	
Priority	High
Preconditions	● Patient is logged into the portal
Postconditions	● Dashboard displays the summary information

	• Patient can navigate to detailed sections from the summary items
Actors	Patient (Main) | Patient Portal System (Secondary)
Extends	NA
Flow of Events	Basic Flow: 1. Patient logs into the portal 2. System retrieves upcoming appointments for the patient 3. System retrieves recent messages and notifications 4. System retrieves any important alerts or reminders 5. System displays all information in an organized dashboard layout 6. Patient can click on any item to view more details
Exceptions	NA
Includes	[13 - Validate User]
Notes/Issues	The dashboard should show the next 2-3 upcoming appointments, the unread message count, and the recent activity

Use Case # - Name	27 - View Appointment Details
Author	Kendra Adkins
Purpose	Allows for a patient to view their details for a specific appointment including time, location, doctor, and appointment type
Requirements/Traceability	

Priority	Medium
Preconditions	• Patient is logged into the portal • Patient has appointments in the system • Patient is viewing appointments list
Postconditions	• Full appointment details are displayed to the patient
Actors	Patient (Main) | Patient Portal System (Secondary)
Extends	[6- View Appointments]
Flow of Events	Basic Flow: 1. Patient is viewing their appointments list 2. Patient clicks on a specific appointment 3. System retrieves complete appointment information from the database 4. System displays detailed view that includes the date, time, doctor name, appointment type, location, and status 5. Patient views all information
Exceptions	If the appointment isn’t found then the system will display an error message
Includes	[13 - Validate User]
Notes/Issues	Should include the appointment confirmation number and any pre-appointment instructions

Use Case # - Name	28 - View Medical History
Author	Kendra Adkins
Purpose	Allows for a patient to view their entire medical history including past diagnoses, treatments, and procedures
Requirements/Traceability	
Priority	High
Preconditions	• Patient is logged into the portal • Medical history records exist for the patient
Postconditions	• Medical history is displayed in chronological order • Patient can view the details of past medical events
Actors	Patient (Main) | Patient Portal System (Secondary)
Extends	NA
Flow of Events	Basic Flow: 1. Patient navigates into medical history sections 2. System retrieves all historical medical records 3. System displays records in reverse chronological order 4. Patient can view their diagnoses, treatments, and procedures with dates 5. Patients can select specific entries for detailed information

Exceptions	If there is no medical history available then the system will display a message that indicates no history on file
Includes	[13 - Validate User]
Notes/Issues	Should include treatment descriptions, diagnosis codes, and the dates of services

Use Case # - Name	29 - Add/Edit Allergies
Author	Kendra Adkins
Purpose	Allows for a nurse to add or update patient allergy information during intake
Requirements/Traceability	
Priority	Medium
Preconditions	• Nurse is logged into the portal • Nurse has access to patient chart
Postconditions	• Patient allergy information is updated in the system
Actors	Nurse (Main) | Patient Portal System (Secondary)
Extends	Update Patient Profile (Use Case 4)
Flow of Events	Basic Flow: 1. Nurse navigates to patient chart 2. Nurse selects “Edit Allergies” 3. System displays their current allergy list

	4. Nurse can add new allergy(allergen name, reaction, and severity) 5. Nurse can edit existing allergy 6. Nurse can delete allergy 7. Nurse saves changes 8. System validates input 9. System updates allergy record
Exceptions	If there is a duplicate allergy then the system will warn that the allergy already exists. If the input is invalid then the system will prompt for required fields.
Includes	[13 - Validate User]
Notes/Issues	Should include severity levels like mild, moderate, and severe, and also include reaction descriptions

Use Case # - Name	30 - View Privacy Settings
Author	Kendra Adkins
Purpose	Allows for a patient to view HIPAA privacy policies and understand how their medical information is used and protected
Requirements/Traceability	
Priority	Medium
Preconditions	● Patient is logged into the portal
Postconditions	● Patient has viewed privacy policy information
Actors	Patient (Main) | Patient Portal System (Secondary)

Extends	NA
Flow of Events	Basic Flow: 1. Patient navigates to account settings 2. Patient selects “Privacy and Security” 3. System displays HIPAA privacy policy information 4. System displays data usage policies 5. Patient can view information about data protection measures 6. Patients can view their medical information rights
Exceptions	NA
Includes	[13 - Validate User]
Notes/Issues	Should include last updated date for privacy policy and a link to a full HIPAA document

Use Case # - Name	31 - Refill Medication
Author	Michael Davis
Purpose	Allows a doctor to order additional refills of medication within the platform.
Requirements/Traceability	
Priority	Medium
Preconditions	• Doctor is authenticated in the portal • Doctor is logged into the portal

	- Doctor has permissions to access the patient records - Patient Records exists in the database
Postconditions	- Refill request is sent to pharmacy - Patient has an order receipt emailed
Actors	Doctor
Extends	N/A
Flow of Events	Basic Flow: - Doctor navigates to the patient file section - Doctor enters the patient's information - Doctor selects the patient's file - System validates the doctor authorization to view records - System retrieves the patient data - System Displays the patient data to the doctor - Doctor selects the 'refill' button next to the desired prescription - Doctor confirms and the system sends a request to the pharmacy - System sends an email to the patient
Exceptions	If there are no prescriptions for the user, the app will not display an option to refill.
Includes	[13] Validate User
Notes/Issues	N/A

Use Case # - Name	32 - Contact Doctor’s Office
Author	Michael Davis
Purpose	Allows a user to contact their doctors office from the patient
Requirements/Traceability	
Priority	Medium
Preconditions	Patient is logged in
Postconditions	Patient selects the ‘contact doctor’s office’ on the main page
Actors	Patient
Extends	N/A
Flow of Events	1. Patient navigates to the home page 2. Patient selects the ‘contact doctor’s office’ button 3. System displays the phone number and business hours for the physician’s office
Exceptions	If the user doesn’t have a primary care physician, it will just pull up all of the offices and their numbers.
Includes	N/A
Notes/Issues	Might have a button to automatically dial the number on mobile devices

Use Case # - Name	33 - Change Patient’s Pharmacy
Author	Michael Davis

Purpose	Allows a Doctor to change the pharmacy at which the patient's medication is sent to
Requirements/Traceability	
Priority	High
Preconditions	• Doctor is authenticated in the portal • Doctor is logged into the portal • Doctor has permissions to access the patient records • Patient Records exists in the database
Postconditions	• Pharmacy location is updated in the database • System displays updated information to the user
Actors	Doctor
Extends	[4 - Update Patient Profile]
Flow of Events	Basic Flow: 1. Doctor navigates to the patient file section 2. Doctor enters the patient's information 3. Doctor selects the patient's file 4. System validates the doctor authorization to view records 5. System retrieves the patient data 6. System Displays the patient data to the doctor

	7. Doctor selects the 'change pharmacy' button next to the desired prescription 8. Doctor confirms and the system sends a request to the pharmacy 9. System updates the change in database 10. System sends an email to the patient
Exceptions	If pharmacy is not found, display error message to Doctor
Includes	[13 - Validate User]
Notes/Issues	Not sure what service will be used to determine relative locations of nearby pharmacies

Use Case # - Name	36 - Delete Account
Author	Michael Davis
Purpose	Allows the patient to delete their account
Requirements/Traceability	
Priority	High
Preconditions	• Patient must be logged in
Postconditions	• Patient's account is deleted and removed from the data base
Actors	Patient
Extends	N/A

Flow of Events	Basic Flow: 1. Patient navigates to the settings page 2. Patient selects the account section 3. Patient selects the delete account button 4. Patient confirms they are sure they want to delete their account 5. System deletes the account and updates the database 6. Patient is sent back to the log in page
Exceptions	If the user does not have an account signed in, they will not be able to delete their account.
Includes	[13 - Validate User]
Notes/Issues	N/A

Use Case # - Name	38 - View Prescription History
Author	Michael Davis
Purpose	Allows patients to view their previous prescriptions for medical history purposes and convenience
Requirements/Traceability	
Priority	Low
Preconditions	• Patient must be logged in

	• Patient must have previous prescriptions
Postconditions	• System will display prescriptions from the past
Actors	Patient
Extends	N/A
Flow of Events	Basic Flow: 1. Patient navigates to the prescription and medication page 2. Patient selects ‘View Prescription History’ button at the bottom of the page 3. System displays the user’s previous prescriptions information from the database
Exceptions	System will display ‘You have zero non-active prescriptions’ if they do not have any previous prescriptions
Includes	N/A
Notes/Issues	N/A

Use Case # - Name	39 - Prescription History Filters
Author	Michael Davis
Purpose	Allows patients to select how far back they want to view medications
Requirements/Traceability	
Priority	Low
Preconditions	• Patient must be logged in

	• Patient must have previous prescriptions
Postconditions	• System will display only prescriptions from designated time frame
Actors	Patient
Extends	[38] View Prescription History
Flow of Events	Basic Flow: 1. Patient navigates to the prescription and medication page 2. Patient selects 'View Prescription History' button at the bottom of the page 3. System displays the user's previous prescriptions information Patient the database 4. User selects the 'last six months' drop down 5. Patient selects the time frame desired 6. System displays only prescriptions from designated time frame
Exceptions	System will display 'You have zero non-active prescriptions' if they do not have any previous prescriptions
Includes	N/A
Notes/Issues	N/A

Use Case # - Name	40 - Prescription Search

Author	Michael Davis
Purpose	Allows Patients to search for prescriptions
Requirements/Traceability	
Priority	Low
Preconditions	• Patient must be logged in • Patient must have active prescriptions
Postconditions	• System will display active prescriptions
Actors	Patient
Extends	[38] View Prescription History
Flow of Events	Basic Flow: 1. Patient navigates to the prescription and medication page 2. Patient selects the search bar at the top of the page 3. System displays the matching prescriptions according to the search text
Exceptions	N/A
Includes	N/A
Notes/Issues	Not sure what kind of search algorithm will be used. Perhaps search will not be available if there are less than 3 active prescriptions.

Use Case # - Name	41 - View Billing Summary (Informational View)
Author	Kirubel Melaku Assefa
Purpose	Allows a patient to view a read-only summary of billing-related information.
Requirements/Traceability	FR-Bill-01
Priority	Low-Medium
Preconditions	• Patient is logged in • Billing summary exists in the system
Postconditions	• Billing summary is displayed to the patient
Actors	Patient
Extends	[13 - Validate User]
Flow of Events	Basic Flow: 1. Patient selects “Billing Summary” from the dashboard 2. System retrieves billing summary data 3. System displays charges, dates, and status (informational only)
Exceptions	If billing data is unavailable, system displays an error message
Includes	[13 - Validate User]
Notes/Issues	No payments or billing actions are supported

Use Case # - Name	44 - System Notification Read

Author	Kirubel Melaku Assefa
Purpose	Allows a patient to mark a system notification as read or acknowledged.
Requirements/Traceability	FR-Msg-04
Priority	Low
Preconditions	● Patient is logged in ● Unread notification exists
Postconditions	● Notification is marked as acknowledged
Actors	Patient
Extends	[8 - View Messages/Notifications]
Flow of Events	Basic Flow: 1. Patient opens a notification 2. Patient selects “Mark as Read” 3. System updates notification status
Exceptions	If update fails, system displays an error message
Includes	[13 - Validate User]
Notes/Issues	Define difference between “read” and “acknowledged”

Use Case # - Name	45 - Manage Security Questions
Author	Kirubel Melaku Assefa
Purpose	Allows a patient to set or update security questions for account recovery.

Requirements/Traceability	FR-Sec-01
Priority	Low-Medium
Preconditions	• Patient is logged in • [needs one more]
Postconditions	• Security questions are saved in the database
Actors	Patient
Extends	[13 - Validate User]
Flow of Events	Basic Flow: 1. Patient selects “Security Settings” 2. System displays security question options 3. Patient selects and answers questions 4. System validates and stores responses
Exceptions	If validation fails, system requests correction
Includes	[13 - Validate User]
Notes/Issues	Security questions supplement password recovery

Use Case # - Name	46 - Review Data Sharing Preferences
Author	Kirubel Melaku Assefa
Purpose	Allows a patient to review how their data may be shared within the system.
Requirements/Traceability	FR-Privacy-01

Priority	Low
Preconditions	• Patient is logged in
Postconditions	• Data-sharing preferences are displayed
Actors	Patient
Extends	[13 - Validate User]
Flow of Events	Basic Flow: 1. Patient selects “Privacy Settings” 2. System displays current data-sharing preferences
Exceptions	If preferences cannot be retrieved, system displays an error message
Includes	[13 - Validate User]
Notes/Issues	Preferences are informational only in current scope

Use Case # - Name	47 - Request Account Deactivation
Author	Kirubel Melaku Assefa
Purpose	Allows a patient to submit a request to deactivate their account.
Requirements/Traceability	FR-Account-05
Priority	Low
Preconditions	• Patient is logged in
Postconditions	• Deactivation request is recorded

Actors	Patient
Extends	[13 - Validate User]
Flow of Events	Basic Flow: 1. Patient selects “Request Account Deactivation” 2. System displays warning message 3. Patient confirms request 4. System records deactivation request
Exceptions	If request cannot be recorded, system displays an error
Includes	[13 - Validate User]
Notes/Issues	Actual deactivation may require admin approval

Use Case # - Name	48 - View System Maintenance Notices
Author	Kirubel Melaku Assefa
Purpose	Allows a patient to view scheduled system maintenance notices.
Requirements/Traceability	FR-System-01
Priority	Low
Preconditions	• Patient is logged in
Postconditions	• Maintenance notices are displayed
Actors	System Administrator
Extends	[13 - Validate User]

Flow of Events	Basic Flow: 1. Patient selects “System Notices” 2. System retrieves maintenance notices 3. System displays notice details
Exceptions	If no notices exist, system displays “No scheduled maintenance”
Includes	[13 - Validate User]
Notes/Issues	Notices are informational only

Use Case # - Name	50 - View Accessibility Guidelines
Author	Kirubel Melaku Assefa
Purpose	Allows users to view accessibility guidelines supported by the portal.
Requirements/Traceability	FR-UI-04
Priority	Low
Preconditions	• The user is accessing the patient portal from a supported web browser
Postconditions	• Accessibility guidelines are displayed
Actors	Patient
Extends	N/A
Flow of Events	Basic Flow:

	1. User selects “Accessibility Information” 2. System displays supported accessibility features
Exceptions	If content cannot be loaded, system displays an error message
Includes	N/A
Notes/Issues	Feature is informational only

Use Case # - Name	52 - Schedule Follow Up Appointment
Author	Keshon Bryant
Purpose	Enable patient to schedule follow up visit based on the previous appointment
Requirements/Traceability	FR-Appointment-04
Priority	High
Preconditions	● Patient has prior appointment ● Doctor has recommended follow-up
Postconditions	● Appointment is scheduled ● Confirmation sent to patient and doctor
Actors	Patient Portal System
Extends	[5 - Request Appointment]
Flow of Events	Basic Flow: 1. Patient selects “Schedule Follow-Up”

	2. System Suggests recommended time slots 3. Patient selects preferred slot 4. System checks doctor availability 5. System books the appointment and sends confirmation
Exceptions	If there are no available slots, System suggests alternative date If there is a system error, appointment is not saved
Includes	[13 -Validate User]
Notes/Issues	The follow up scheduling is subject to doctors availability and system constraints.

Use Case # - Name	53 - View Doctor Availability
Author	Keshon Bryant
Purpose	Allow patient to check available appointment slots for a doctor
Requirements/Traceability	FR- Scheduling-02
Priority	Medium
Preconditions	● Patient is logged in ● Doctors schedule is published
Postconditions	● Patient views available time slots
Actors	Patient
Extends	N/A
Flow of Events	Basic Flow:

	1. Patient selects find availability 2. System fetches doctors schedule for selected range 3. System displays available slots in calendar view 4. Patient can filter date and time
Exceptions	If Doctor has no schedule loaded, System shows “Unavailable” If there is no internet, data not refreshed
Includes	N/A
Notes/Issues	Feature will use existing data from the system only

Use Case # - Name	55 - Submit insurance information
Author	Keshon Bryant
Purpose	Allow patient to enter/update insurance details for billing
Requirements/Traceability	FR-Billing-03
Priority	Medium
Preconditions	● Patient is logged in ● Insurance fields are available
Postconditions	● Insurance info is saved ● Flagged for verification if needed
Actors	Patient
Extends	[4 - Update Patient Profile]
Flow of Events	Basic Flow:

	1. Patient navigates to “Insurance” section 2. System displays form 3. Patient enters/edits info and submits 4. System validates format and stores data 5. System may trigger verification request to admin
Exceptions	Invalid policy number format, System prompts correction If there's a duplicate entry, System warns patient
Includes	N/A
Notes/Issues	Changes will take effect immediately after its submission

Use Case # - Name	57 - Set Notification Preferences (Informational view)
Author	Keshon Bryant
Purpose	Let patient customize how they receive system notifications
Requirements/Traceability	FR-Notification-02
Priority	Low
Preconditions	● Patient is logged in ● Notification system is configured
Postconditions	● Preferences saved and future notifications follow new settings
Actors	Patient

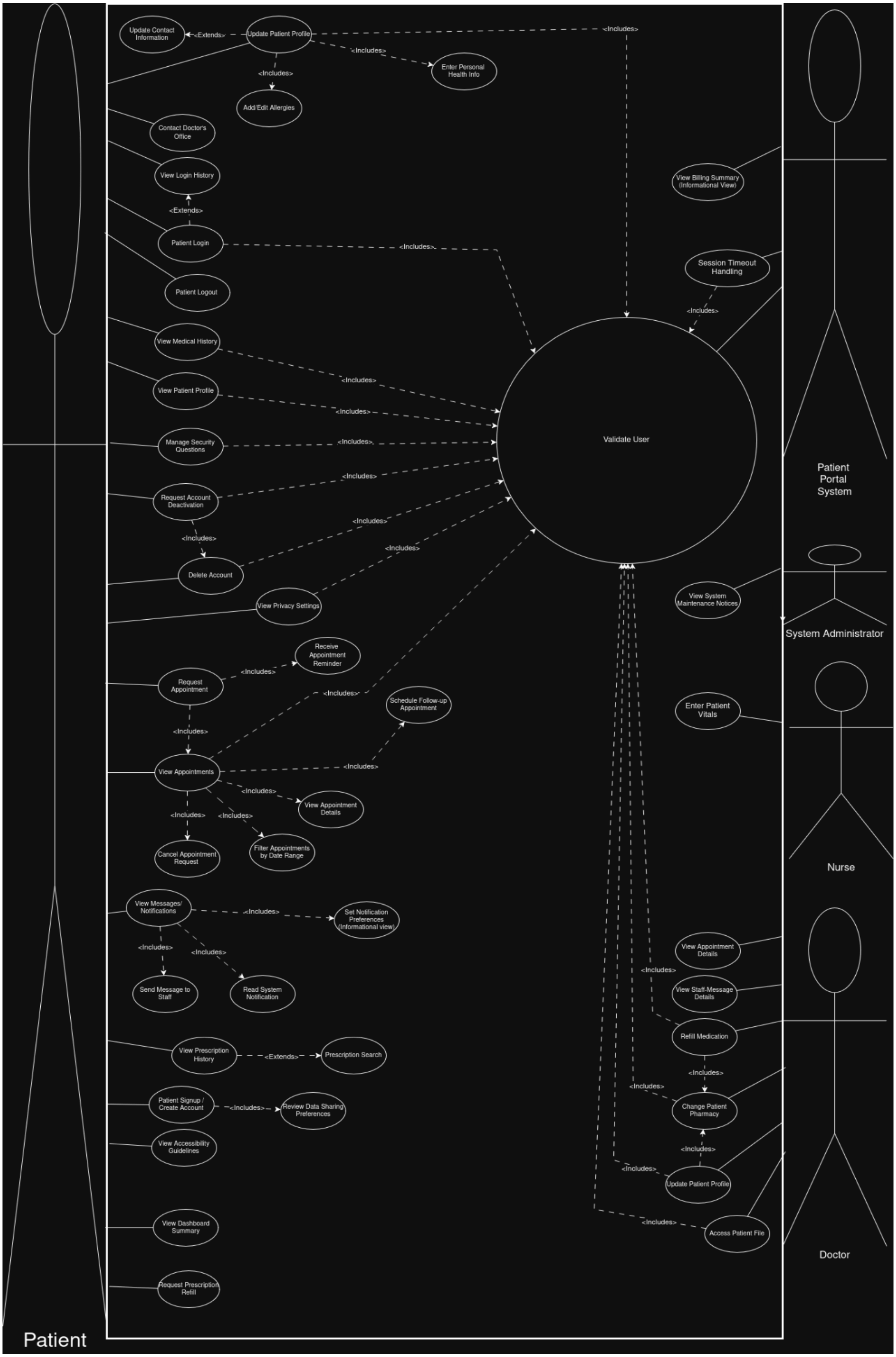
Extends	[13 - Validate User]
Flow of Events	1. Patient goes to Settings>Notifications 2. System shows a toggle for email, SMS, In App 3. Patient selects channels and saves 4. System updates the profile
Exceptions	If save fails, Changes not applied If there is an invalid Preference combination, System warns
Includes	N/A
Notes/Issues	This is for Informational use only. No actual notifications will be sent.

Use Case # - Name	58 - Request prescription Refill
Author	Keshon Bryant
Purpose	Enable patient to request refills for existing prescriptions
Requirements/Traceability	FR-Medication-04
Priority	High
Preconditions	● Patient has active prescription ● Pharmacy info is on file
Postconditions	● Refill request sent to pharmacy/doctor for approval
Actors	Patient
Extends	[31 - Refill Medication]

Flow of Events	Basic Flow: 1. Patient selects “Medications>Request Refill" 2. System list eligible prescriptions 3. Patient selects medication and confirms 4. System sends request to pharmacy/doctor 5. System notifies patient of request status
Exceptions	If prescription information is missing, System prompts user to select a valid prescription
Includes	[13 - Validate User]
Notes/Issues	Refill Request may require doctor approval

3.1.1.2: Use Case Diagram

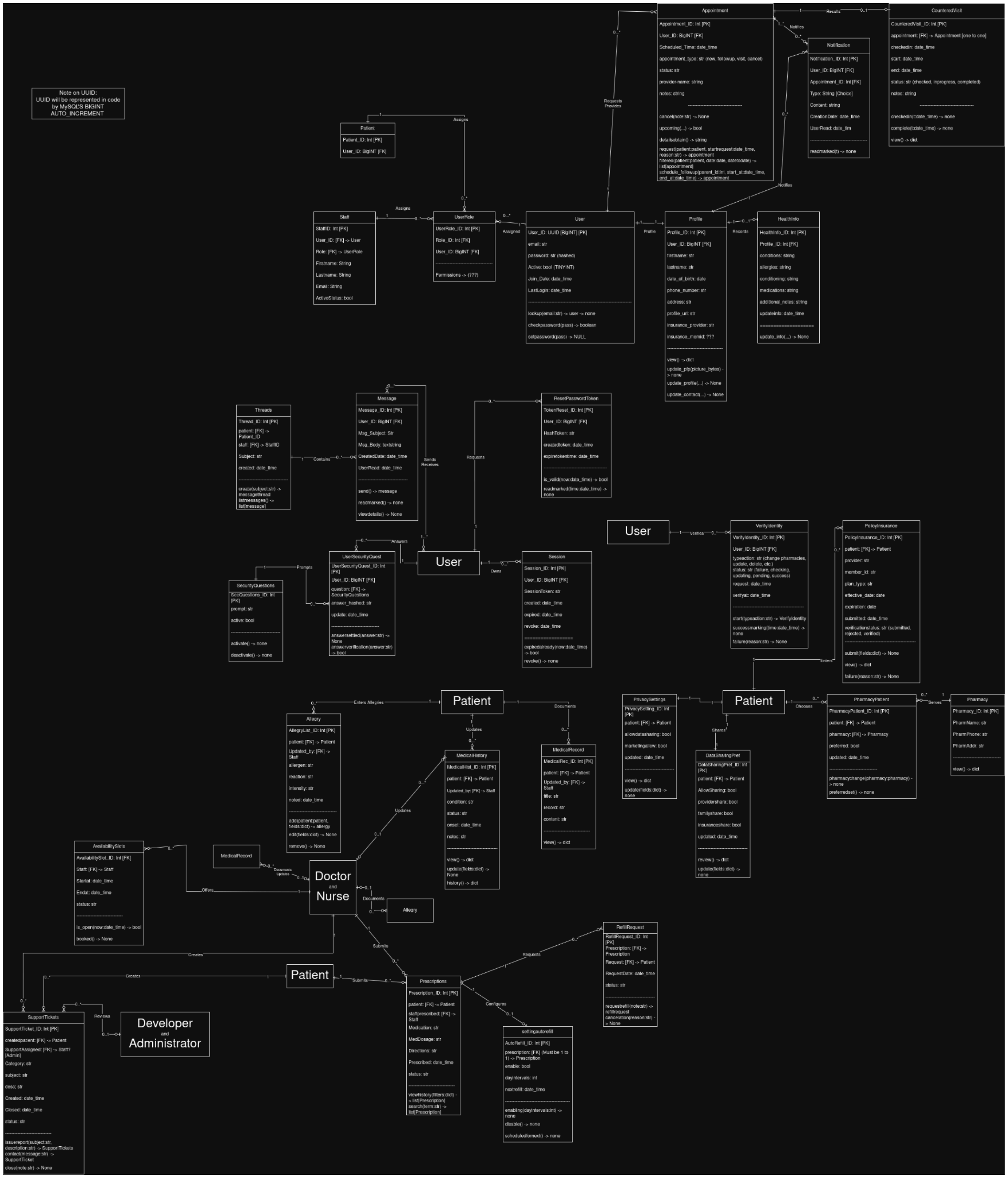
Use Case Diagram Image:


Update Contact Information
<Extends>
Update Patient Profile
<Includes>
Enter Personal Health Info
<Includes>
Add/Edit Allergies
<Includes>
Contact Doctor's Office
View Login History
<Extends>
Patient Login
<Includes>
Patient Logout
View Medical History
<Includes>
View Patient Profile
<Includes>
Manage Security Questions
<Includes>
Request Account Deactivation
<Includes>
<Includes>
Delete Account
<Includes>
View Privacy Settings
<Includes>
Validate User
View Billing Summary (Informational View)
Session Timeout Handling
<Includes>
Patient Portal System
View System Maintenance Notices
System Administrator
Receive Appointment Reminder
<Includes>
Request Appointment
<Includes>
Schedule Follow-up Appointment
<Includes>
View Appointments
<Includes>
<Includes>
View Appointment Details
<Includes>
<Includes>
Cancel Appointment Request
Filter Appointments by Date Range
Enter Patient Vitals
Nurse
View Messages/ Notifications
<Includes>
Set Notification Preferences (Informational view)
<Includes>
<Includes>
Send Message to Staff
Read System Notification
View Appointment Details
View Staff-Message Details
<Includes>
Refill Medication
<Includes>
View Prescription History
<Extends>
Prescription Search
<Includes>
Change Patient Pharmacy
Patient Signup / Create Account
<Includes>
Review Data Sharing Preferences
<Includes>
View Accessibility Guidelines
<Includes>
Update Patient Profile
<Includes>
Access Patient File
View Dashboard Summary
Doctor
Request Prescription Refill
Patient

Description of Use Case Diagram Layout:
In the use case diagram, the actors are the patient, doctor, nurse, patient portal system, and system administrator. If a patient has not previously registered in the patient portal, they will have to first create an account. When the patient inputs their information to create their account, the system will give them a new blank state to enter their information in. The majority of features in the Use Case Diagram will be geared to the user that the platform is most visited by: the patient. The platform is designed around ease of use for this particular actor first and foremost, as patients will have the greatest fluctuation in technical aptitude. Some patients are older and will have trouble seeing certain UI elements, while some patients have an easy time adapting to various UI interfaces. It's very important that priority is put on this group of users since they require the most accommodations and have the largest feature requirements.
From a UI perspective, the system is designed to help guide users through authentication, navigation, and task completion in a smooth and clear way. Each use case corresponds with actions accessible through the portal interface, such as viewing appointments, updating profiles, or reviewing notifications. The system interface is put into a simple design to be user-friendly so that patients can easily navigate through their dashboard, appointments, and profile information. The portal also supports responsive web access; this allows users to interact through desktop or mobile browsers. The patient portal gives easy ways to manage and access medical information through multiple important features. In order to manage appointments, users can filter by date to find certain visits, view details about their upcoming appointments, and see everything at a glance on their dashboard. The patient dashboard shows recent messages and upcoming appointments for a simplified experience. Also, patients have control over their profile settings, like editing or adding allergies and reviewing their medical history.

3.1.2 Class Diagram

Class Diagram Image (Draw IO):

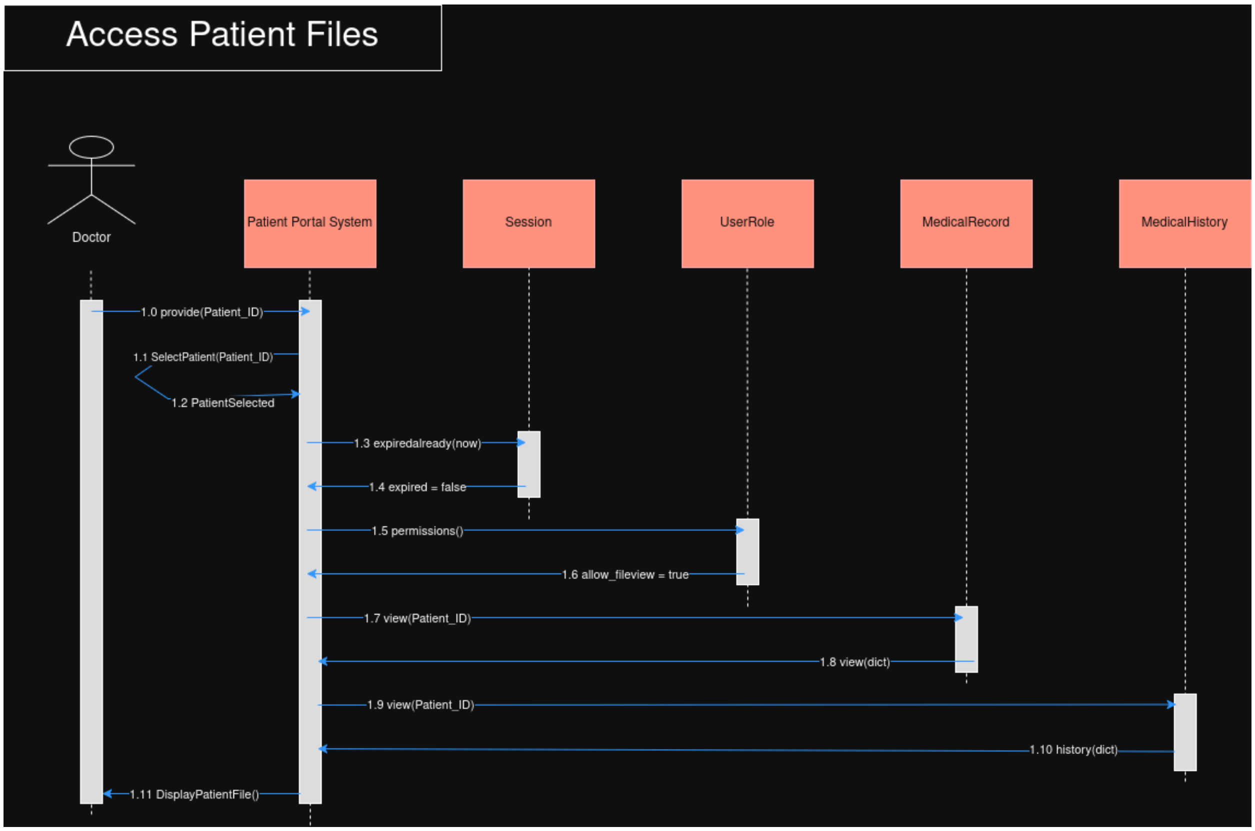


Description of Class Diagram:
The User class represents the core entity for accounts for all of the individuals in the patient portal system, which includes the patients, doctors, staff members, and developers/administrators, and the user role defines the permissions for role-based control access to system features. The Session class handles the login state and session expiration during user timeouts. Password recovery validates the resetting password attempts and marks the tokens as used once a password is changed. The User Profile information is stored in the profile class, which enables users to view and update their contact details and their data. The healthinfo class stores the health information submitted by the patients, which can be updated through the patient portal system. The clinical data is designed with the MedicalRecord and MedicalHistory classes. The MedicalRecord assists with viewing medical data in a structured manner, while the MedicalHistory class allows both viewing and updating previous medical documentation. The allergy class manages the allergy records (create, read/view, and update) entries for patient safety. In addition to allergy tracking, the system organizes broader clinical and visit-related information. Managing appointments is handled by the appointment class, which assists with requesting, scheduling, filtering, and cancelling follow-up appointments that are linked to prior visits from the patient. AvailabilitySlots represents a doctor's available time slots before it books an appointment. Afterwards, visits that are completed are recorded in the CounteredVisit class, which tracks check-ins and visits. Communications between users are backed by the message and messagethread classes. Messagethread will group any related messages into structured discussions. Individual messages can be viewed, sent to another user, and marked as being read. The system alerts are handled by the Notification class, which reminds the user of their upcoming appointments and messages. Medication management is represented by the Prescriptions class, which provides users to view their prescription history and search for their medications. Pharmacy information is stored in the Pharmacy class, and the patient-specific pharmacy preferences are stored in PharmacyPatient. Privacy and data features are controlled through the PrivacySetting and DataSharingPref classes, which enable users to review and update their privacy settings. The security features include securityquestion and usersecurityqa for account recovery verification.

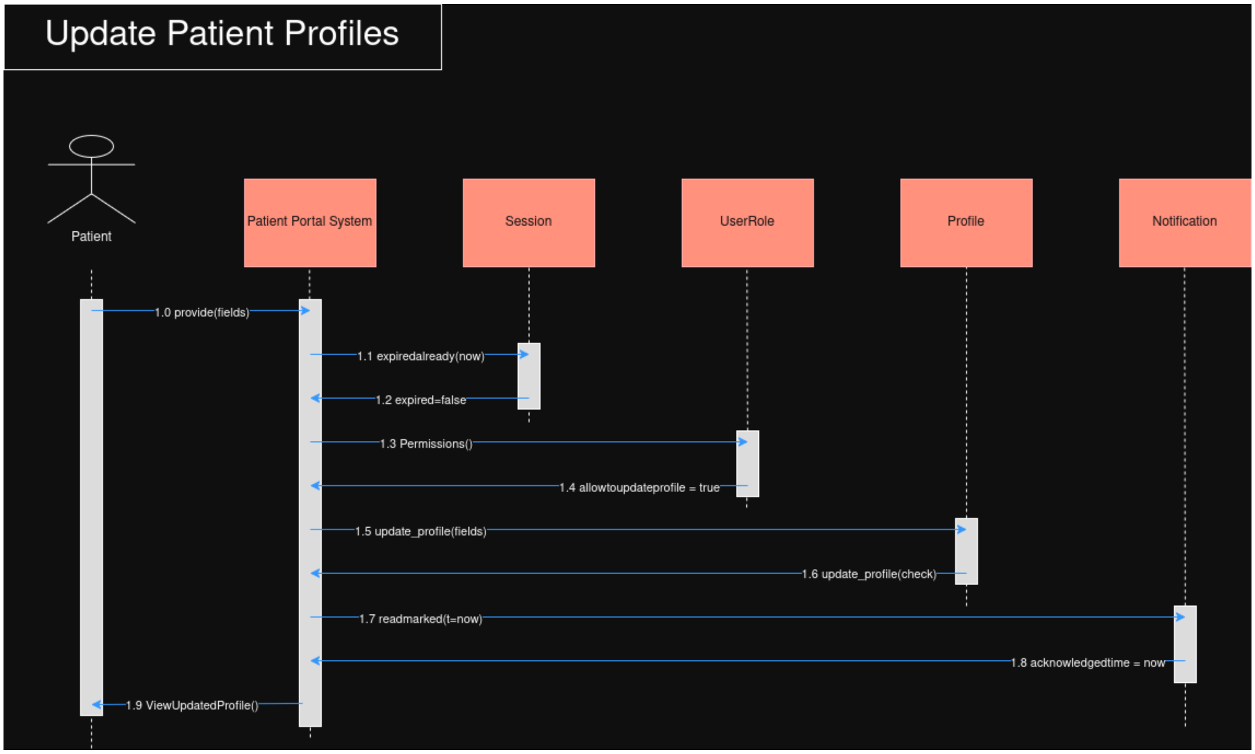
3.1.3 Sequence Diagrams

Colton:

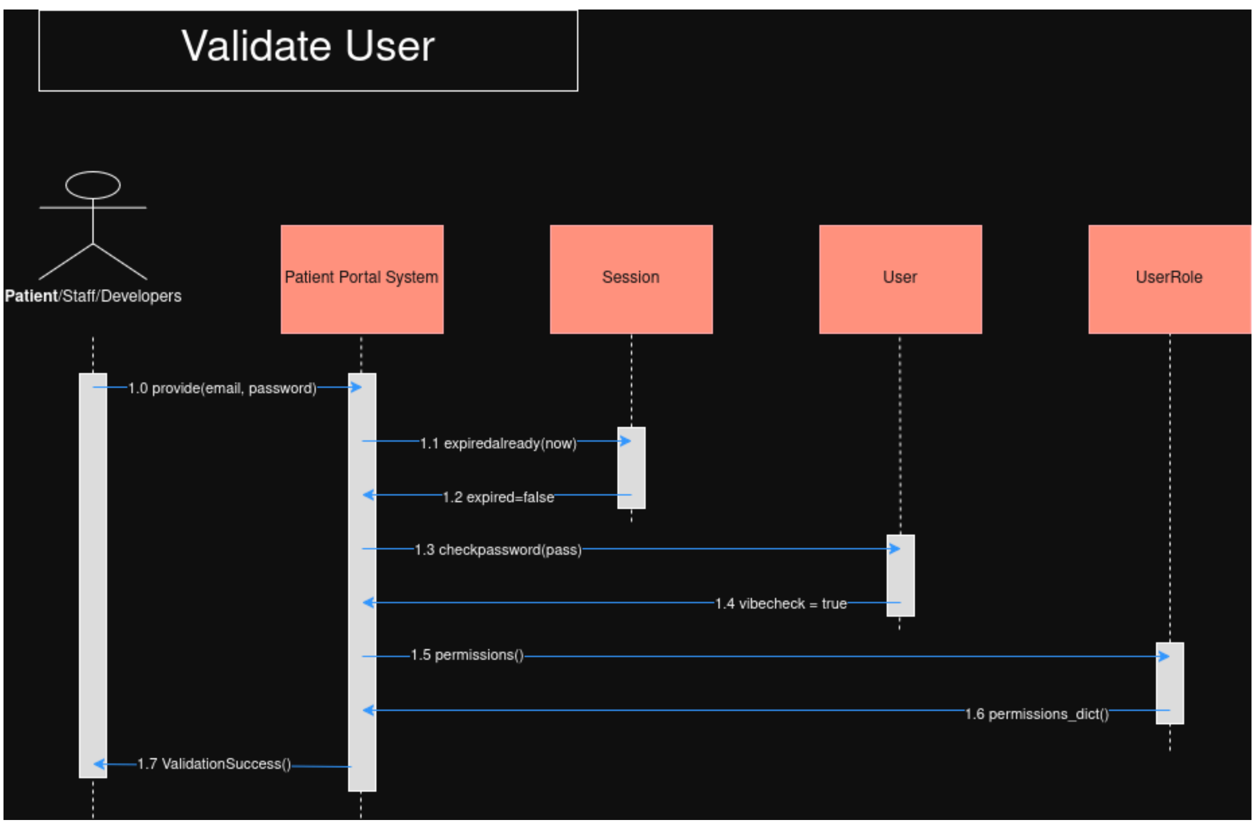
Access Patient Files:



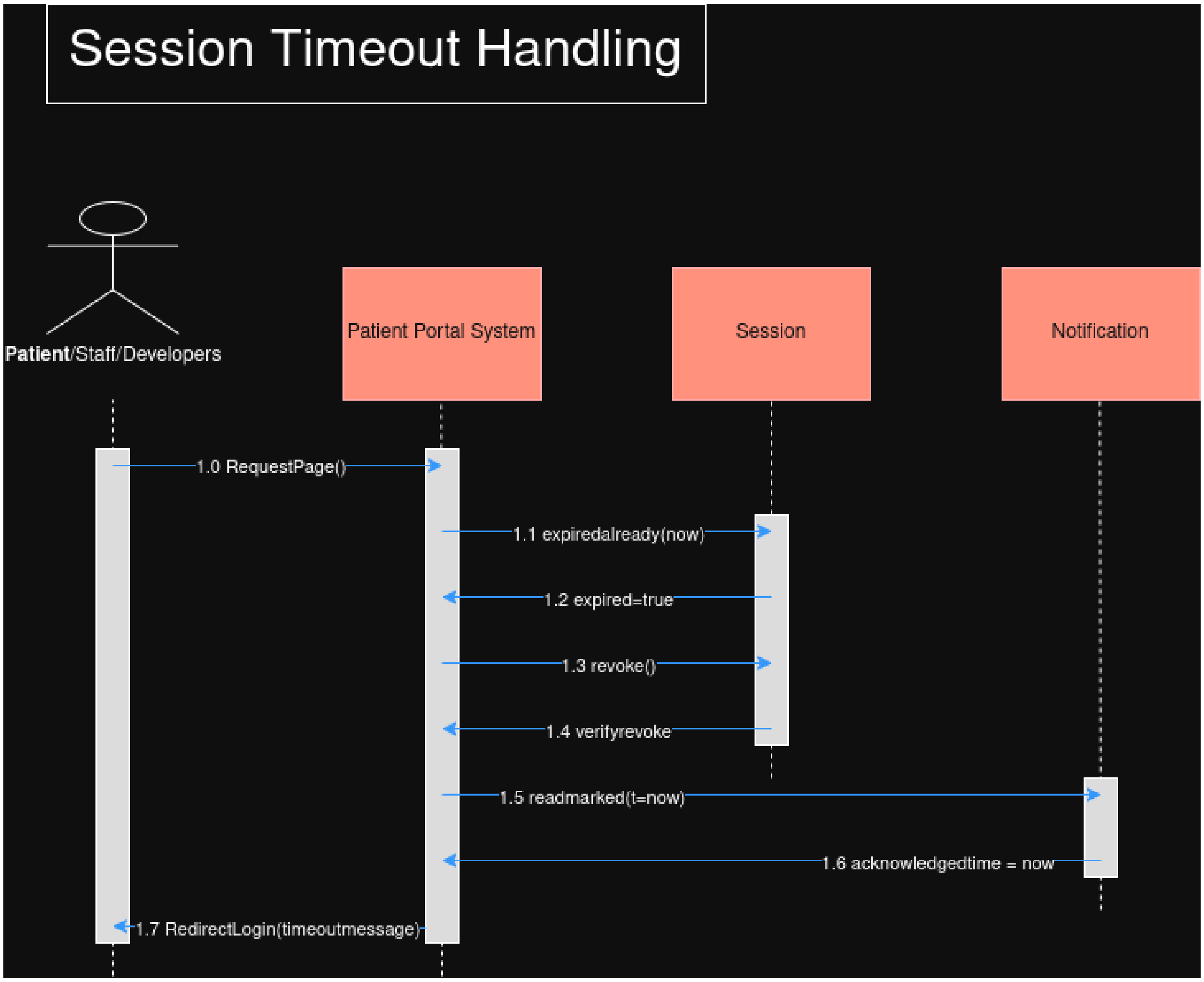
Update Patient Profiles:



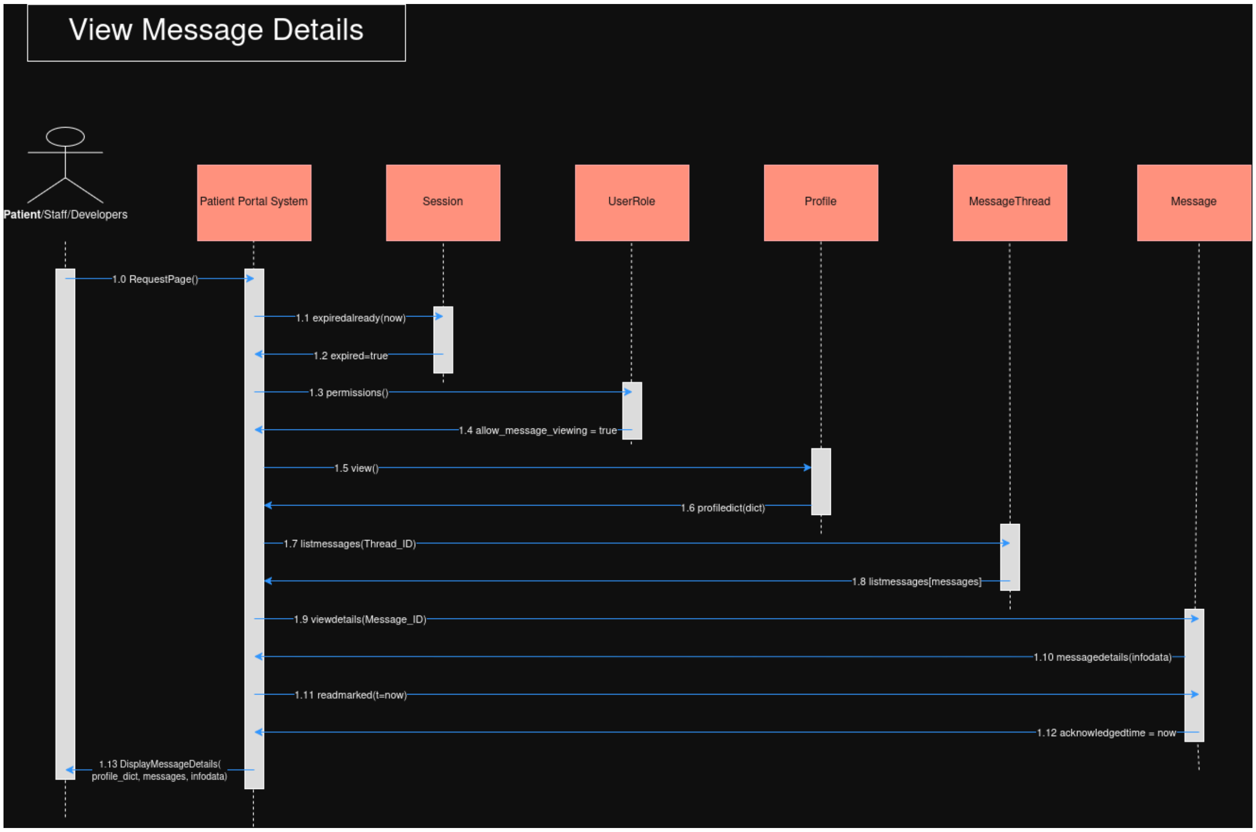
Validate User:



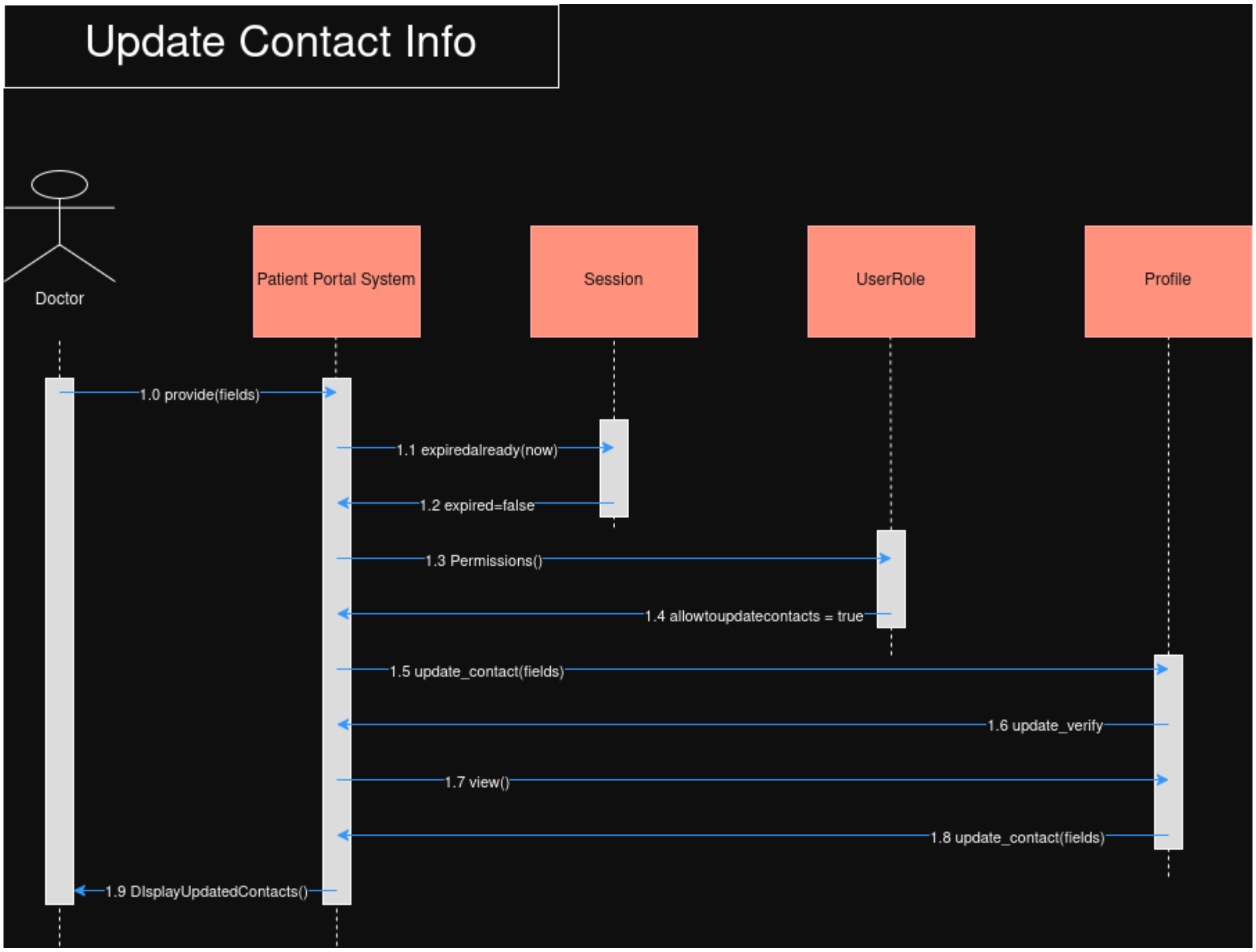
Session Timeout Handling:



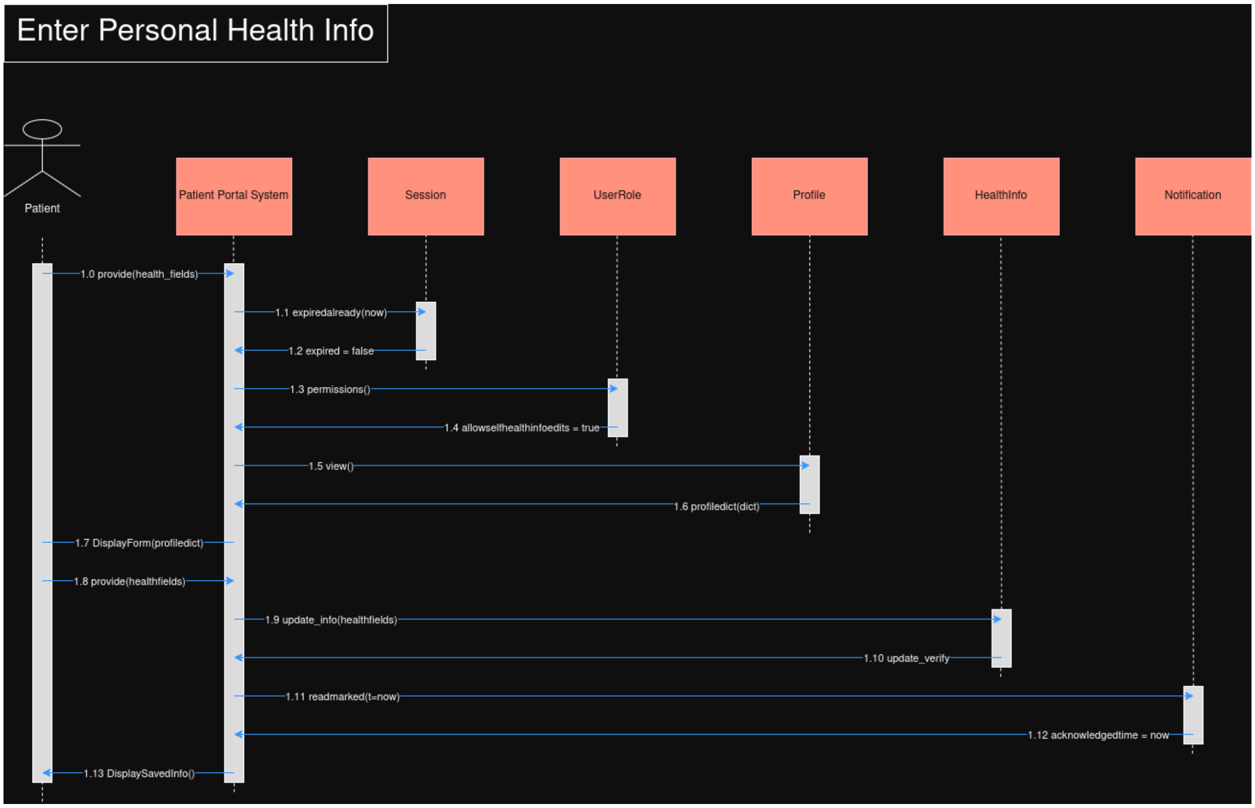
View Message Details:



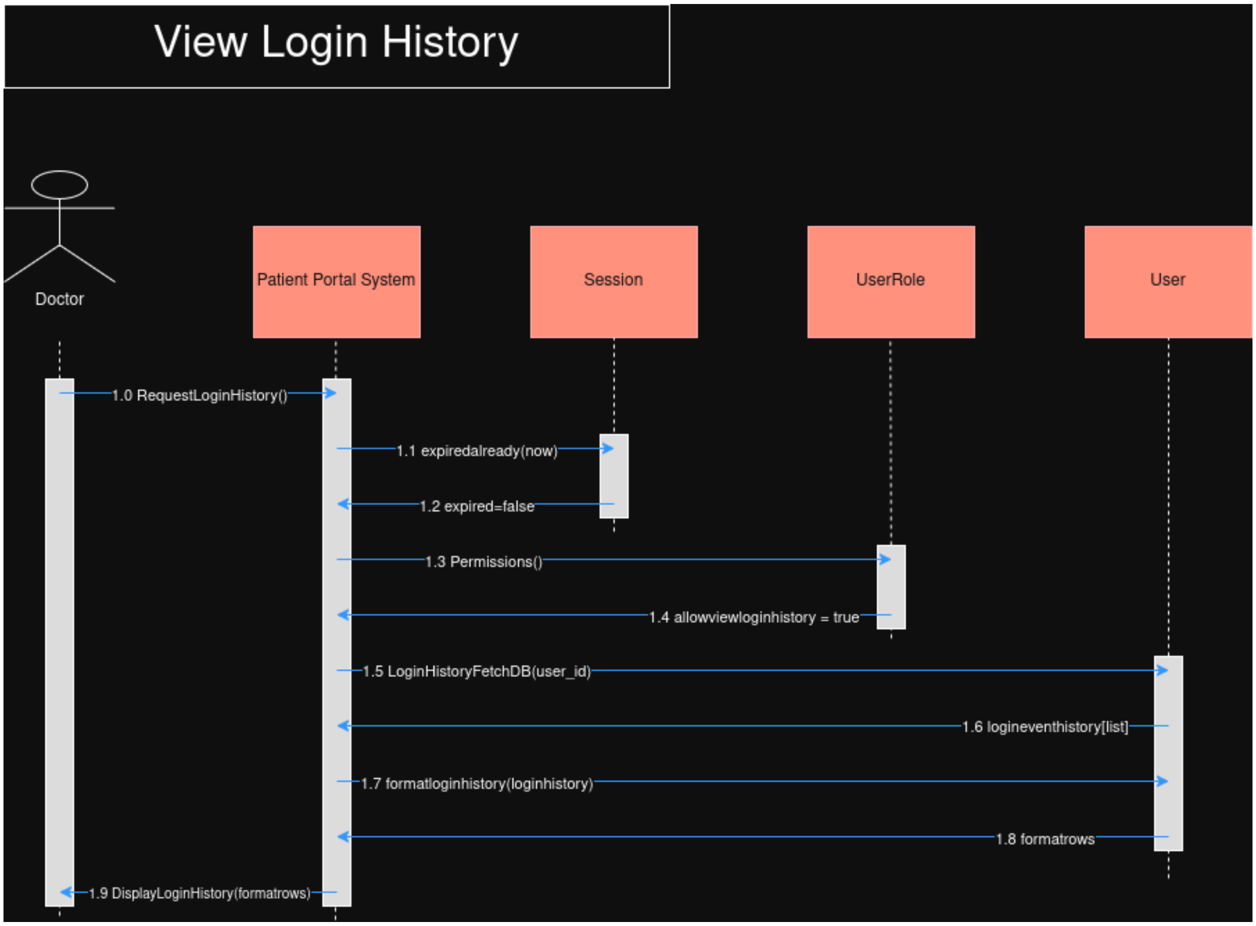
Update Contact Information:



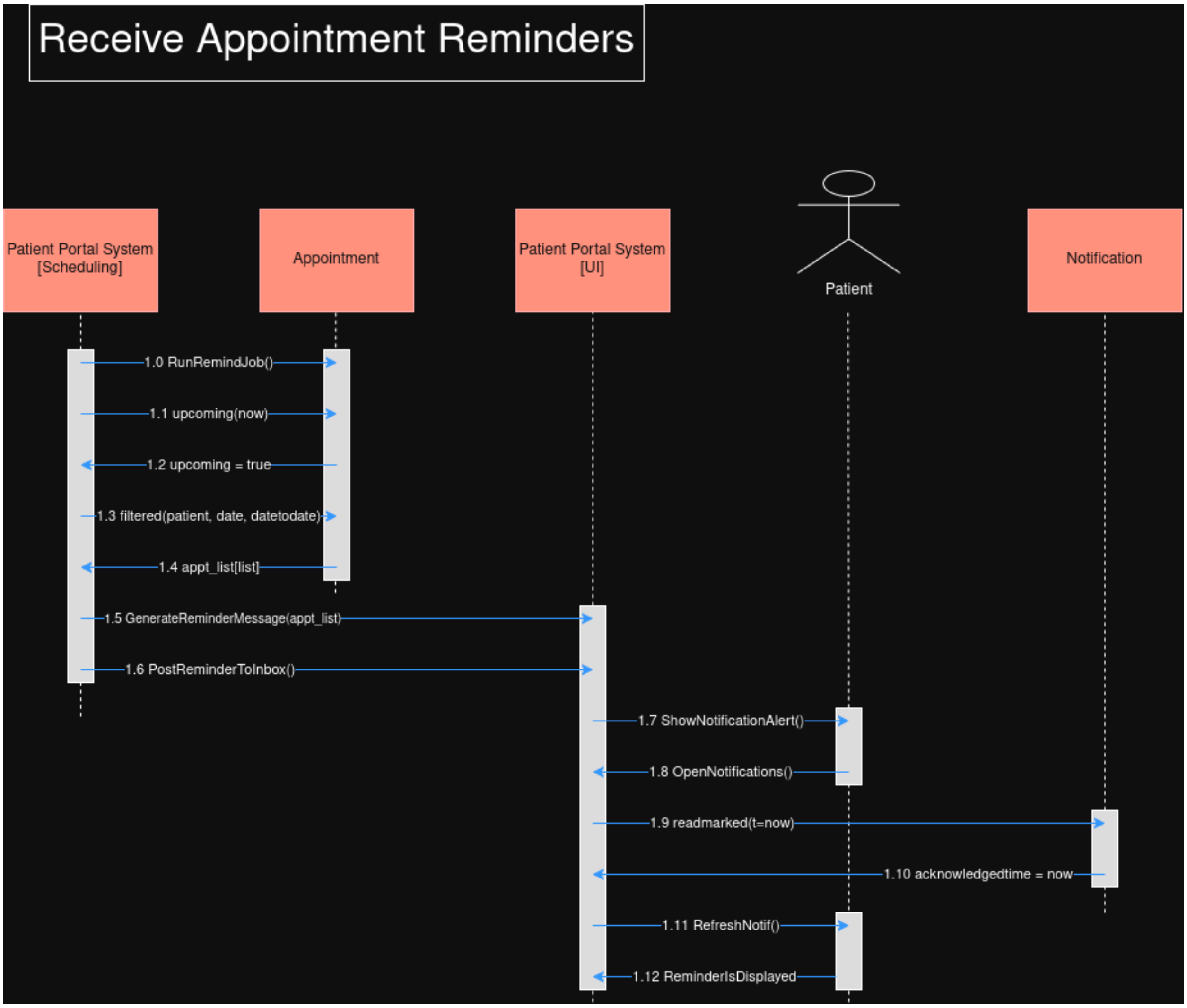
Enter Personal Health Information:



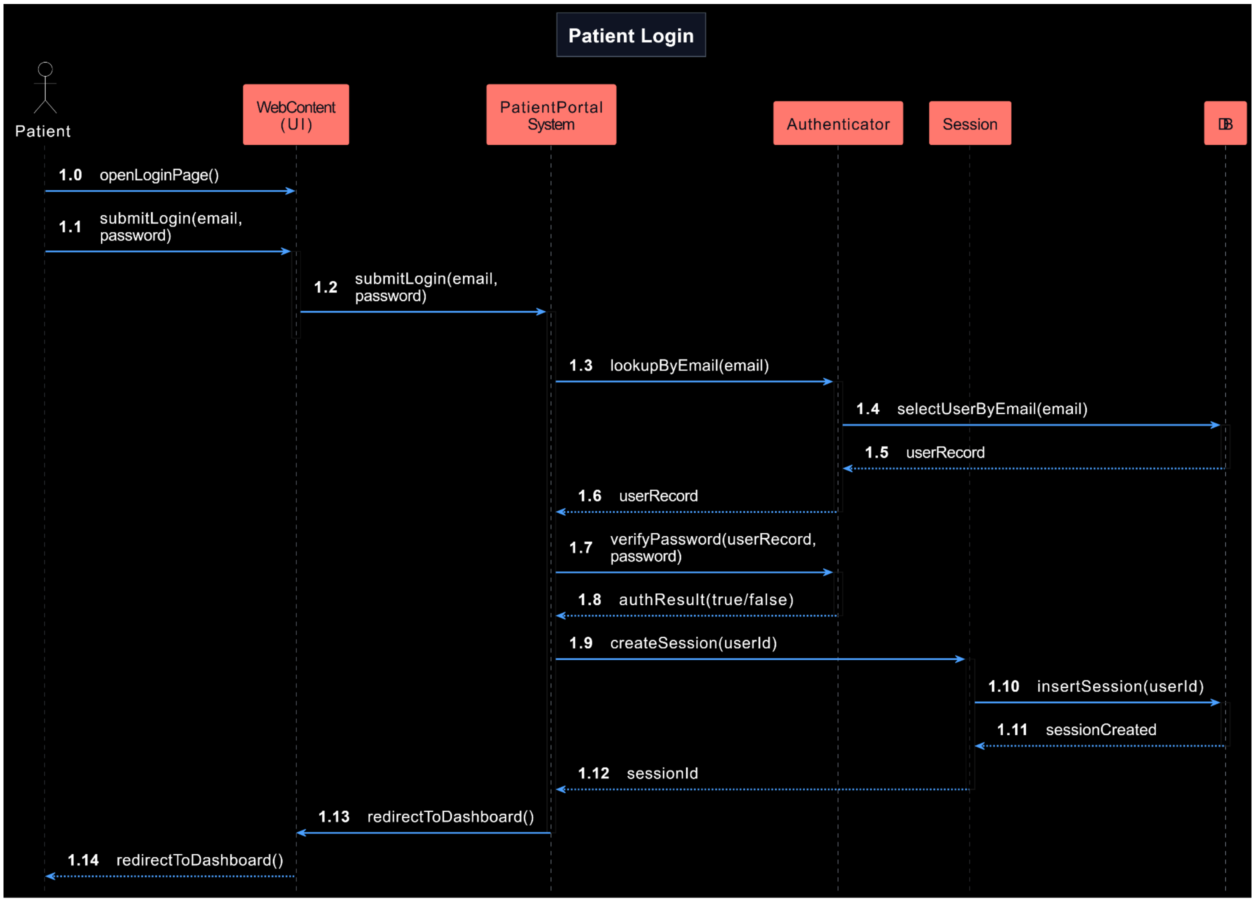
View Login History:



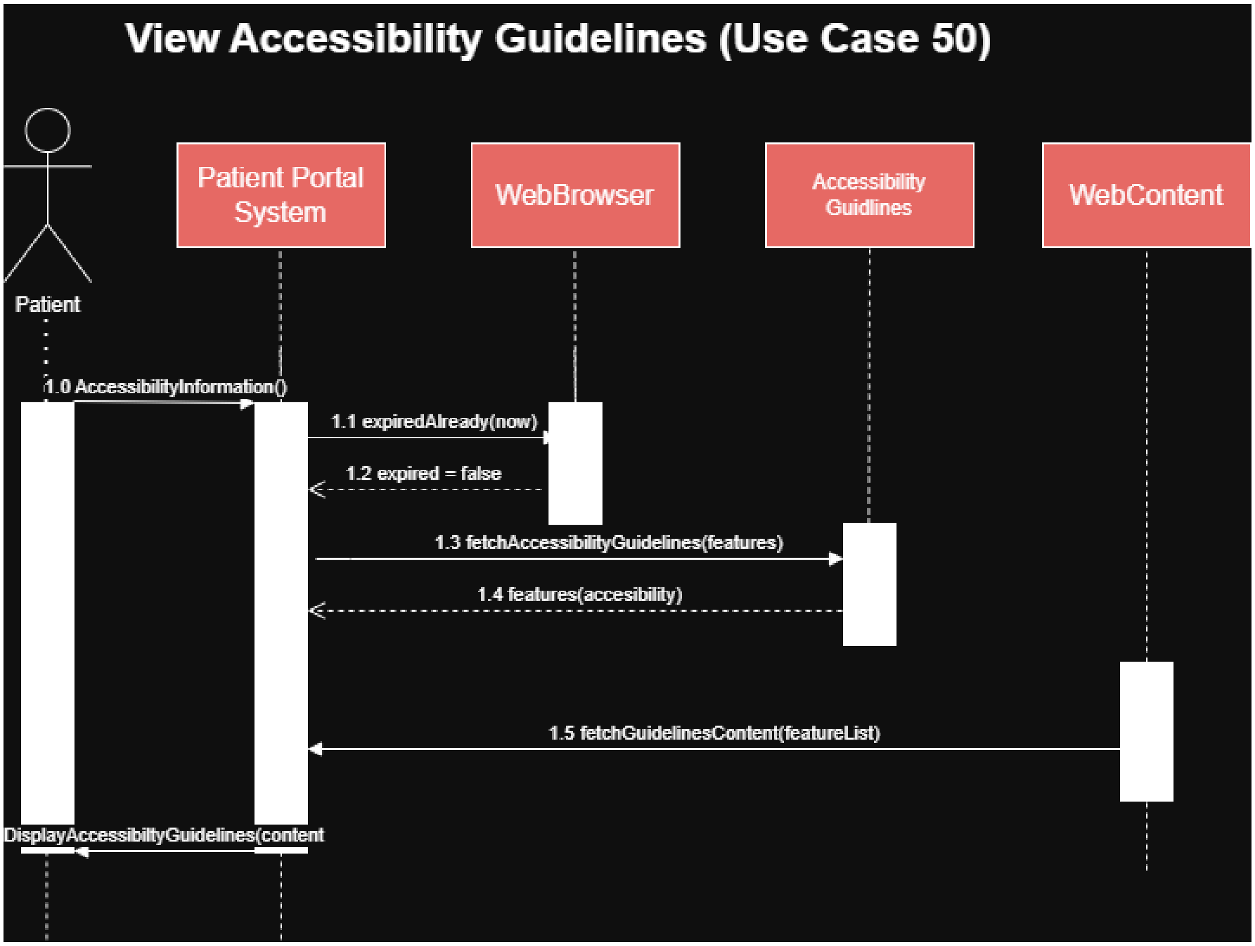
Receive Appointment Reminders:

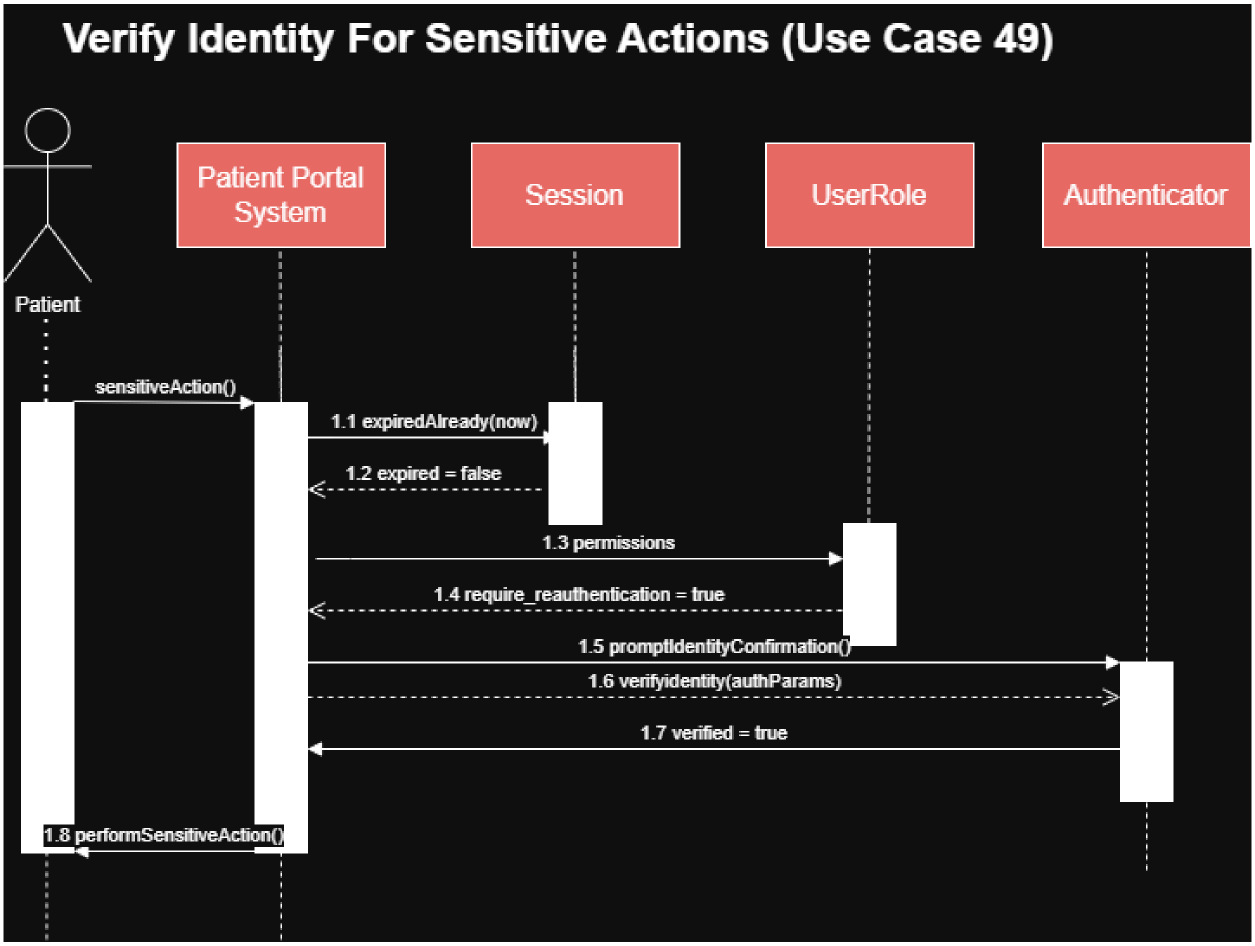

Receive Appointment Reminders
Patient Portal System [Scheduling]
Appointment
Patient Portal System [UI]
Patient
Notification
1.0 RunRemindJob()
1.1 upcoming(now)
1.2 upcoming = true
1.3 filtered(patient, date, datetodate)
1.4 appt_list[list]
1.5 GenerateReminderMessage(appt_list)
1.6 PostReminderToInbox()
1.7 ShowNotificationAlert()
1.8 OpenNotifications()
1.9 readmarked(t=now)
1.10 acknowledgedtime = now
1.11 RefreshNotif()
1.12 ReminderIsDisplayed

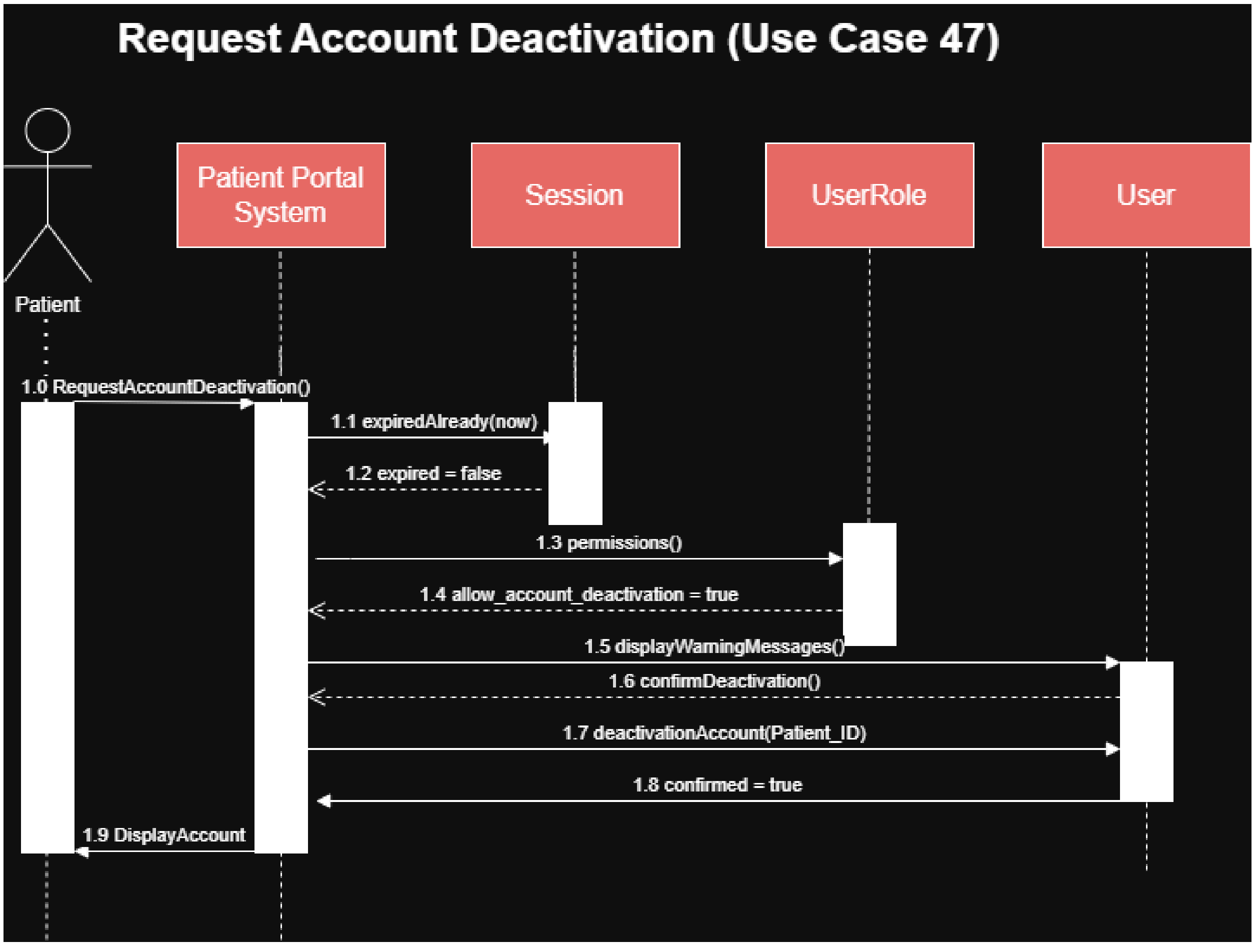
Jeff:

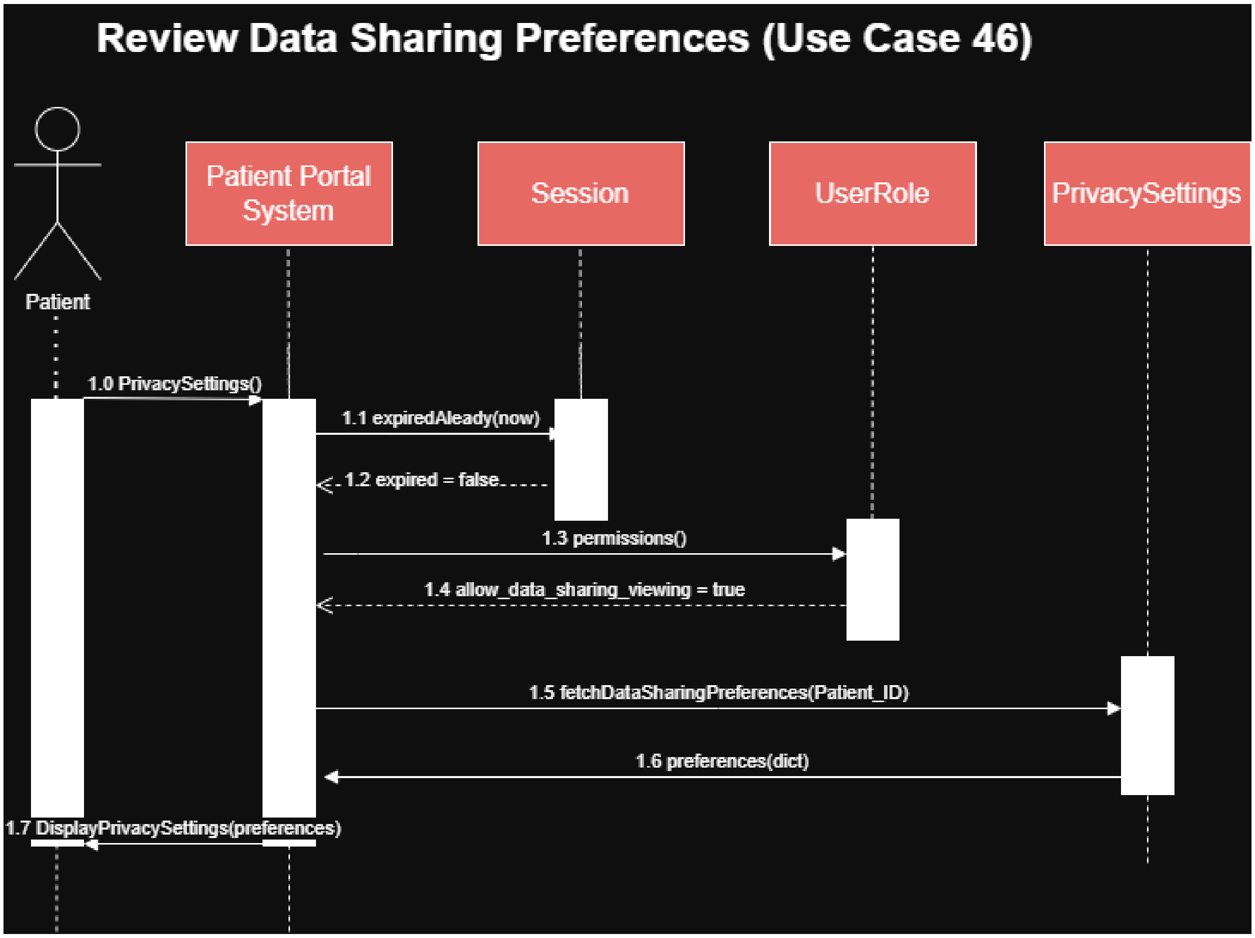

Patient Login
Patient
WebContent (UI)
PatientPortal System
Authenticator
Session
DB
1.0 openLoginPage()
1.1 submitLogin(email, password)
1.2 submitLogin(email, password)
1.3 lookupByEmail(email)
1.4 selectUserByEmail(email)
1.5 userRecord
1.6 userRecord
1.7 verifyPassword(userRecord, password)
1.8 authResult(true/false)
1.9 createSession(userId)
1.10 insertSession(userId)
1.11 sessionCreated
1.12 sessionId
1.13 redirectToDashboard()
1.14 redirectToDashboard()

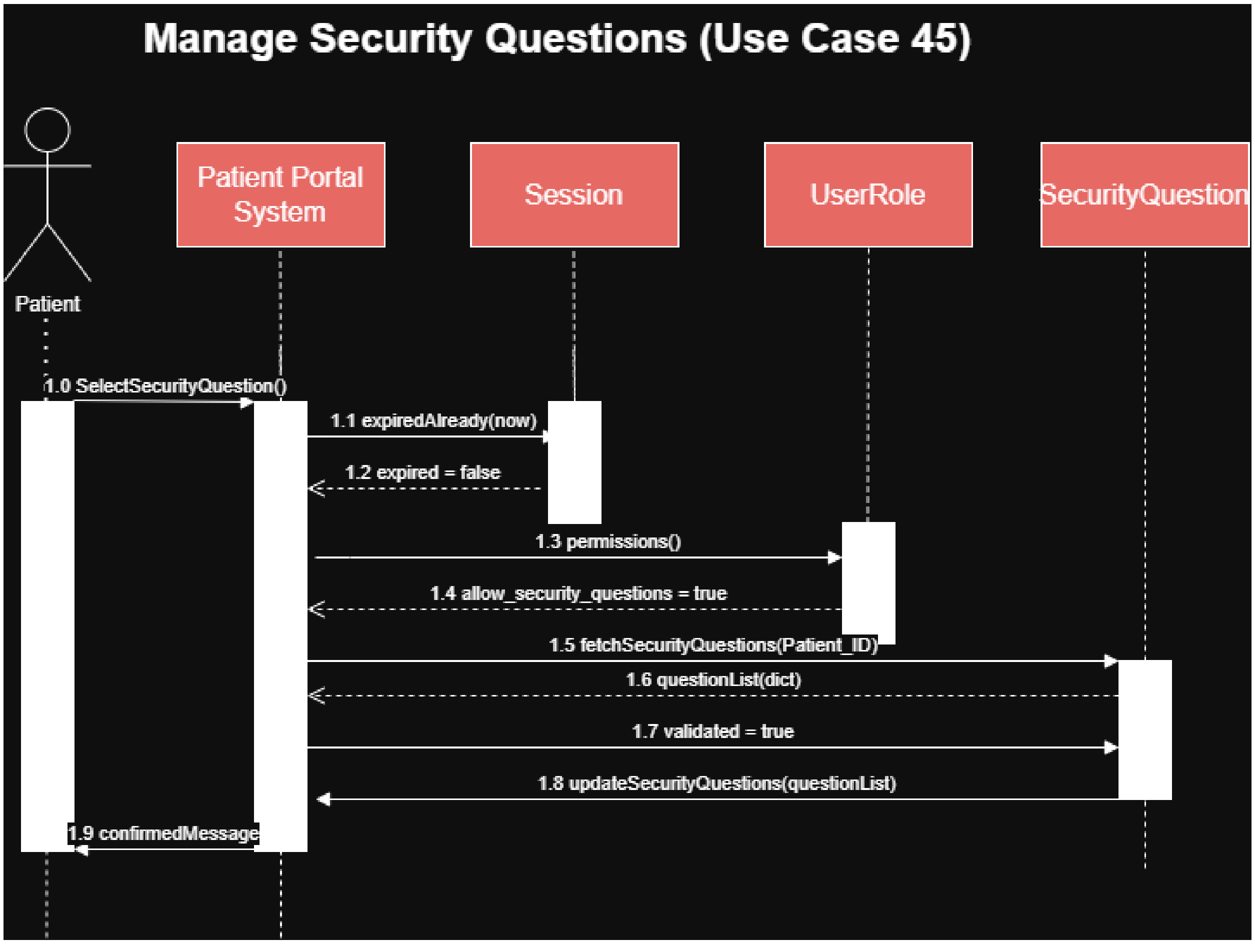
Kiru:

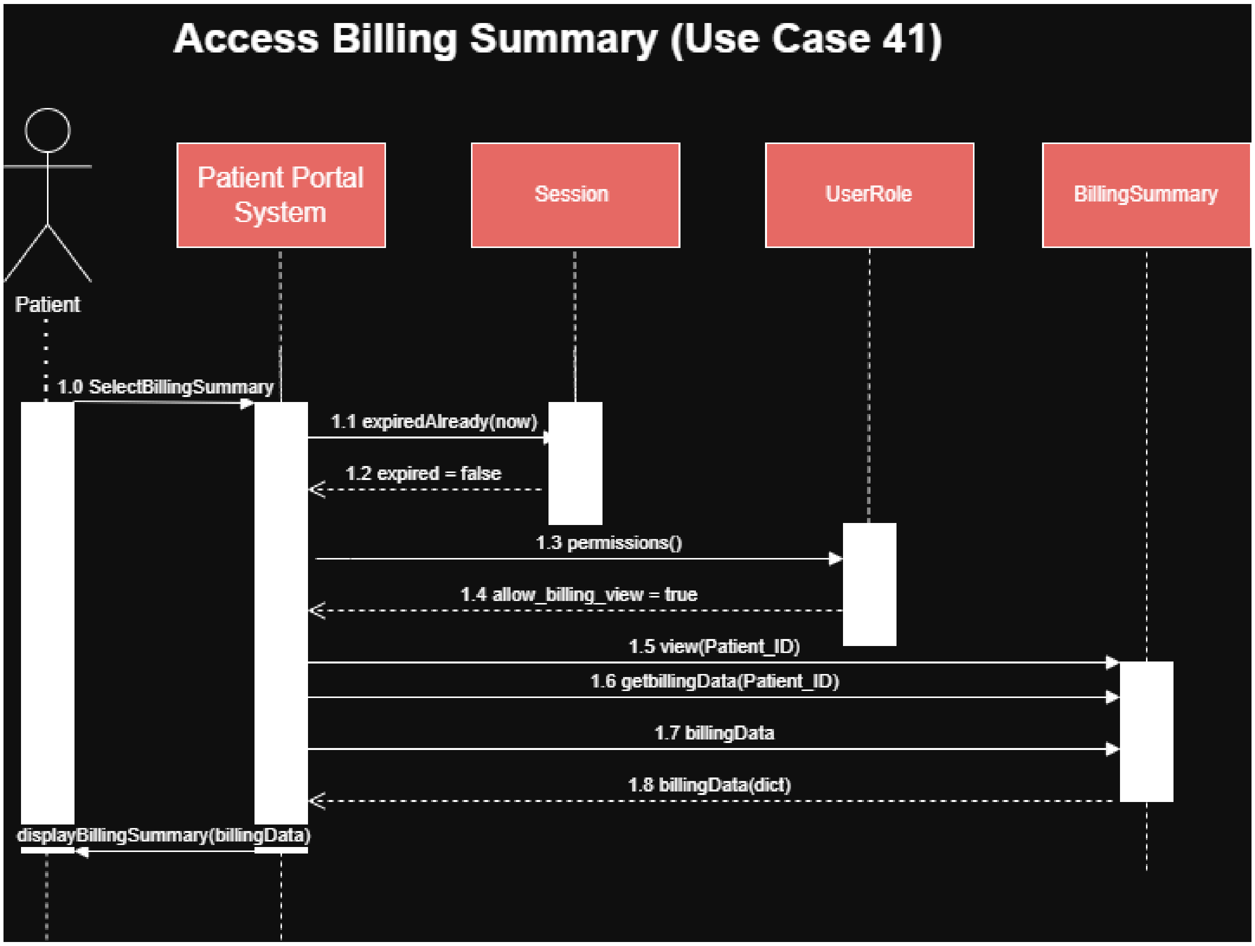

View Accessibility Guidelines (Use Case 50)
Patient
Patient Portal System
WebBrowser
Accessibility Guidlines
WebContent
1.0 AccessibilityInformation()
1.1 expiredAlready(now)
1.2 expired = false
1.3 fetchAccessibilityGuidelines(features)
1.4 features(accesibility)
1.5 fetchGuidelinesContent(featureList)
DisplayAccessibiltyGuidelines(content

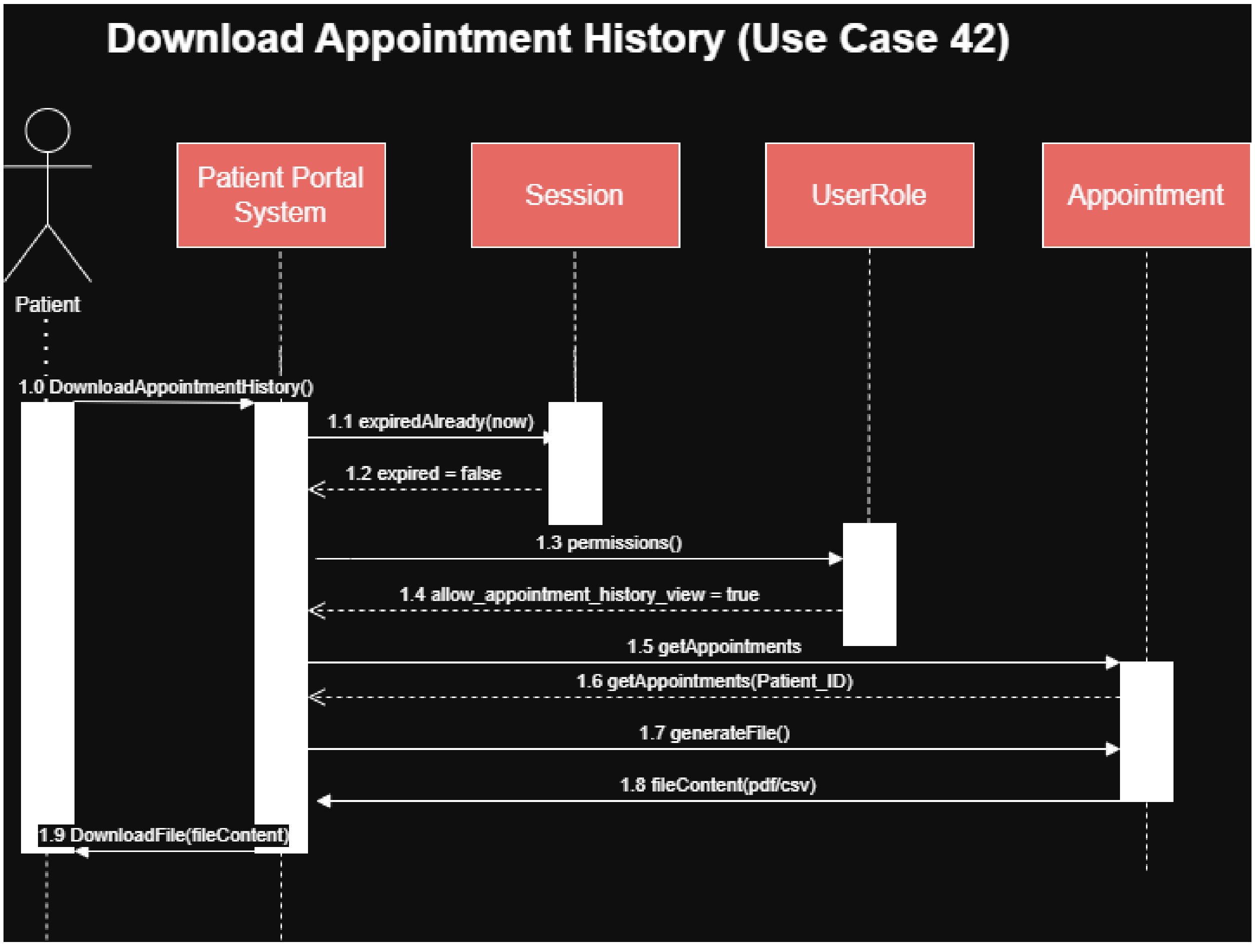

Verify Identity For Sensitive Actions (Use Case 49)
Patient
Patient Portal System
Session
UserRole
Authenticator
sensitiveAction()
1.1 expiredAlready(now)
1.2 expired = false
1.3 permissions
1.4 require_reauthentication = true
1.5 promptIdentityConfirmation()
1.6 verifyidentity(authParams)
1.7 verified = true
1.8 performSensitiveAction()

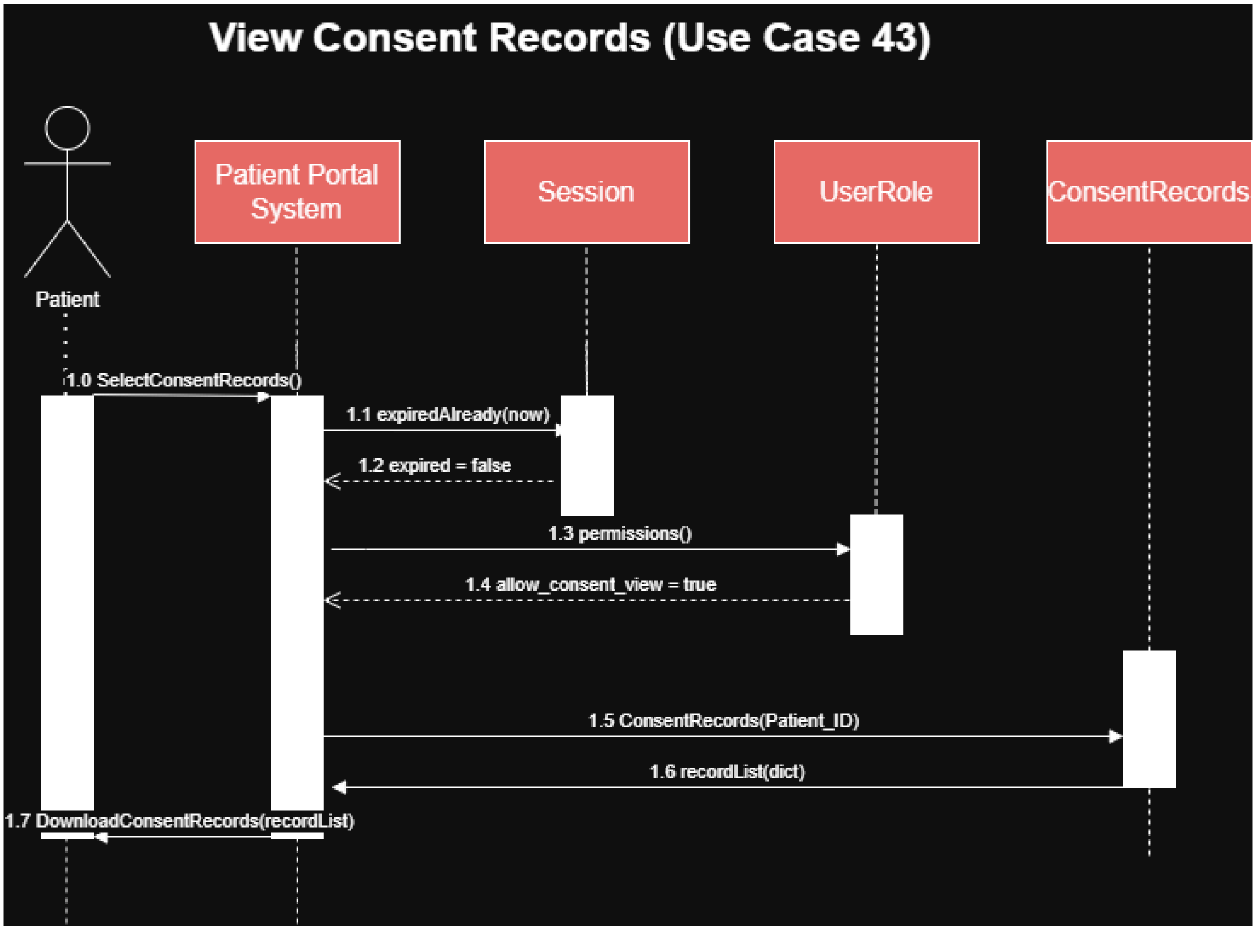

Request Account Deactivation (Use Case 47)
Patient
Patient Portal System
Session
UserRole
User
1.0 RequestAccountDeactivation()
1.1 expiredAlready(now)
1.2 expired = false
1.3 permissions()
1.4 allow_account_deactivation = true
1.5 displayWarningMessages()
1.6 confirmDeactivation()
1.7 deactivationAccount(Patient_ID)
1.8 confirmed = true
1.9 DisplayAccount

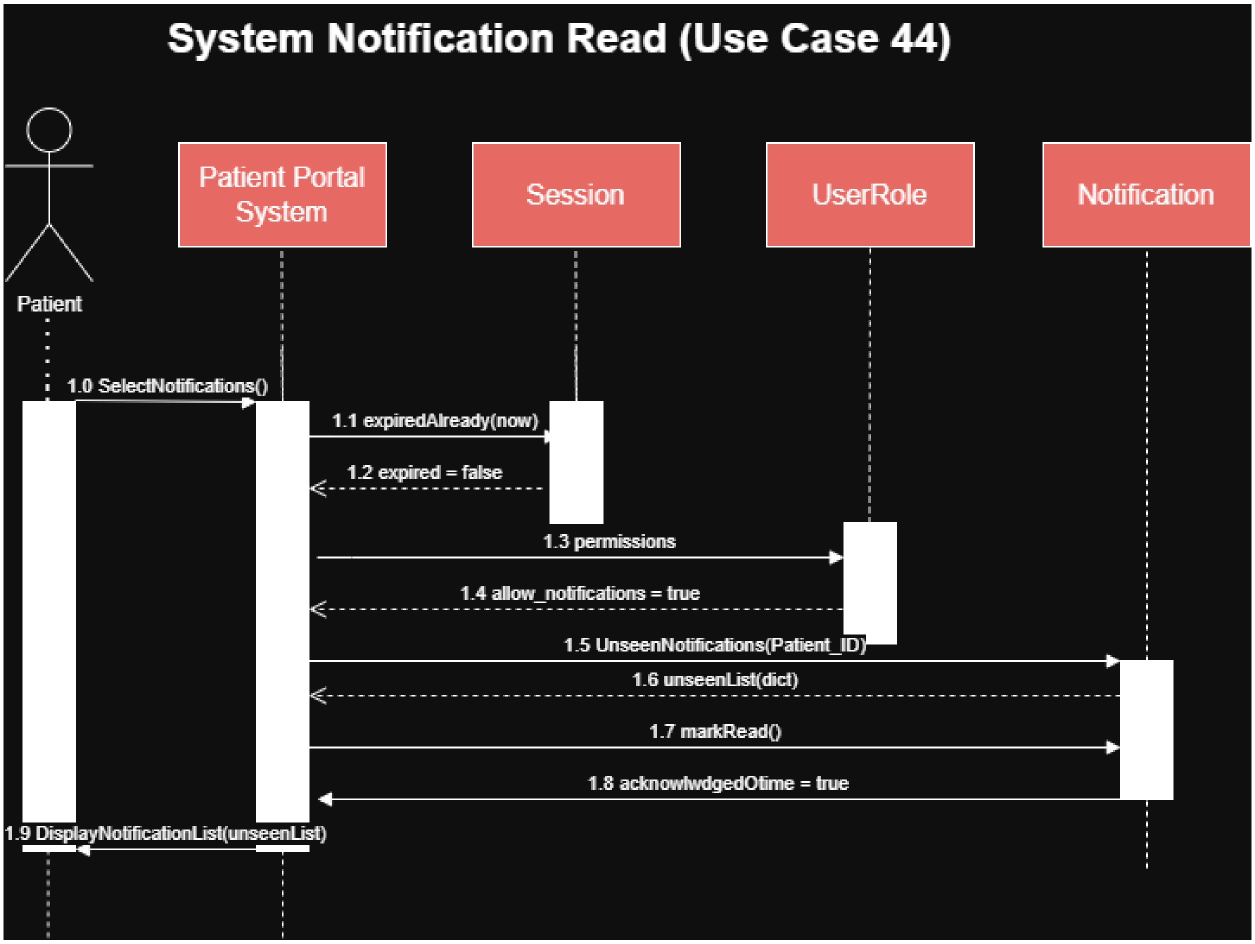

Review Data Sharing Preferences (Use Case 46)
Patient
Patient Portal System
Session
UserRole
PrivacySettings
1.0 PrivacySettings()
1.1 expiredAleady(now)
1.2 expired = false
1.3 permissions()
1.4 allow_data_sharing_viewing = true
1.5 fetchDataSharingPreferences(Patient_ID)
1.6 preferences(dict)
1.7 DisplayPrivacySettings(preferences)


Manage Security Questions (Use Case 45)
Patient
Patient Portal System
Session
UserRole
SecurityQuestion
1.0 SelectSecurityQuestion()
1.1 expiredAlready(now)
1.2 expired = false
1.3 permissions()
1.4 allow_security_questions = true
1.5 fetchSecurityQuestions(Patient_ID)
1.6 questionList(dict)
1.7 validated = true
1.8 updateSecurityQuestions(questionList)
1.9 confirmedMessage


Access Billing Summary (Use Case 41)
Patient
Patient Portal System
Session
UserRole
BillingSummary
1.0 SelectBillingSummary
1.1 expiredAlready(now)
1.2 expired = false
1.3 permissions()
1.4 allow_billing_view = true
1.5 view(Patient_ID)
1.6 getbillingData(Patient_ID)
1.7 billingData
1.8 billingData(dict)
displayBillingSummary(billingData)

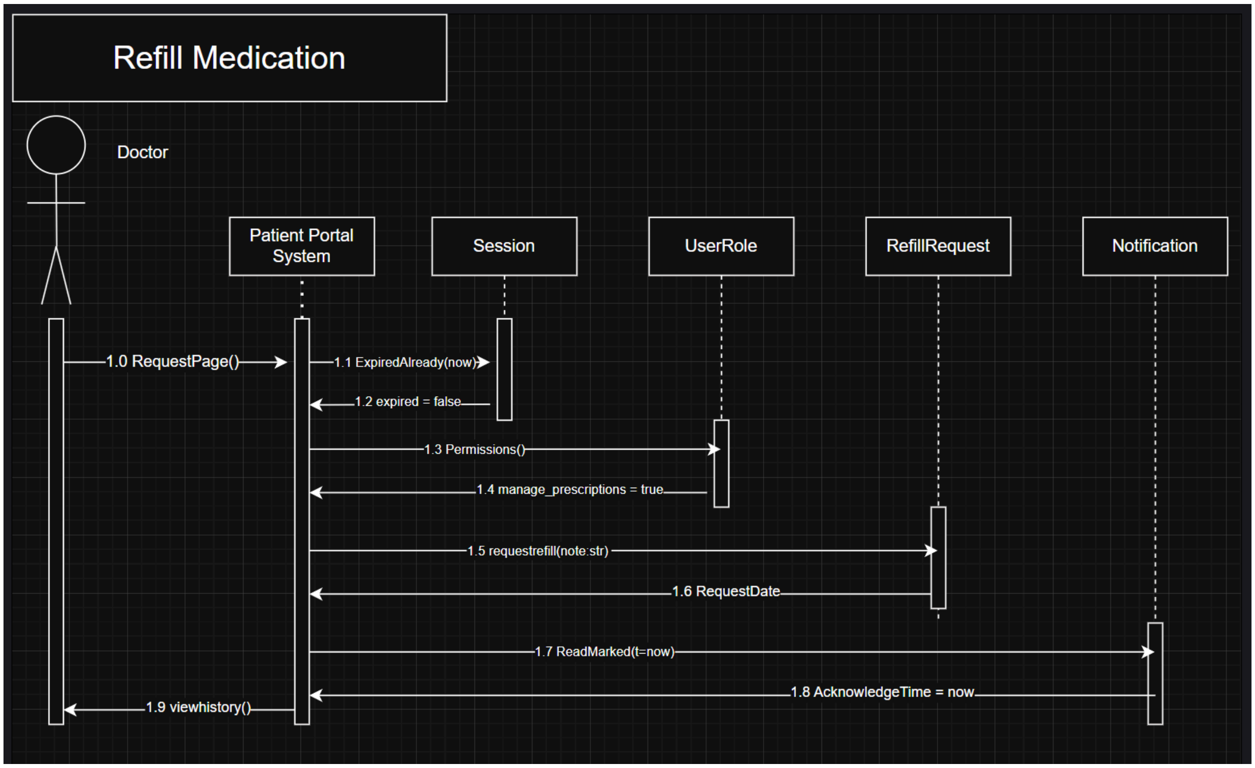

Download Appointment History (Use Case 42)
Patient
Patient Portal System
Session
UserRole
Appointment
1.0 DownloadAppointmentHistory()
1.1 expiredAlready(now)
1.2 expired = false
1.3 permissions()
1.4 allow_appointment_history_view = true
1.5 getAppointments
1.6 getAppointments(Patient_ID)
1.7 generateFile()
1.8 fileContent(pdf/csv)
1.9 DownloadFile(fileContent)


View Consent Records (Use Case 43)
Patient
Patient Portal System
Session
UserRole
ConsentRecords
1.0 SelectConsentRecords()
1.1 expiredAlready(now)
1.2 expired = false
1.3 permissions()
1.4 allow_consent_view = true
1.5 ConsentRecords(Patient_ID)
1.6 recordList(dict)
1.7 DownloadConsentRecords(recordList)

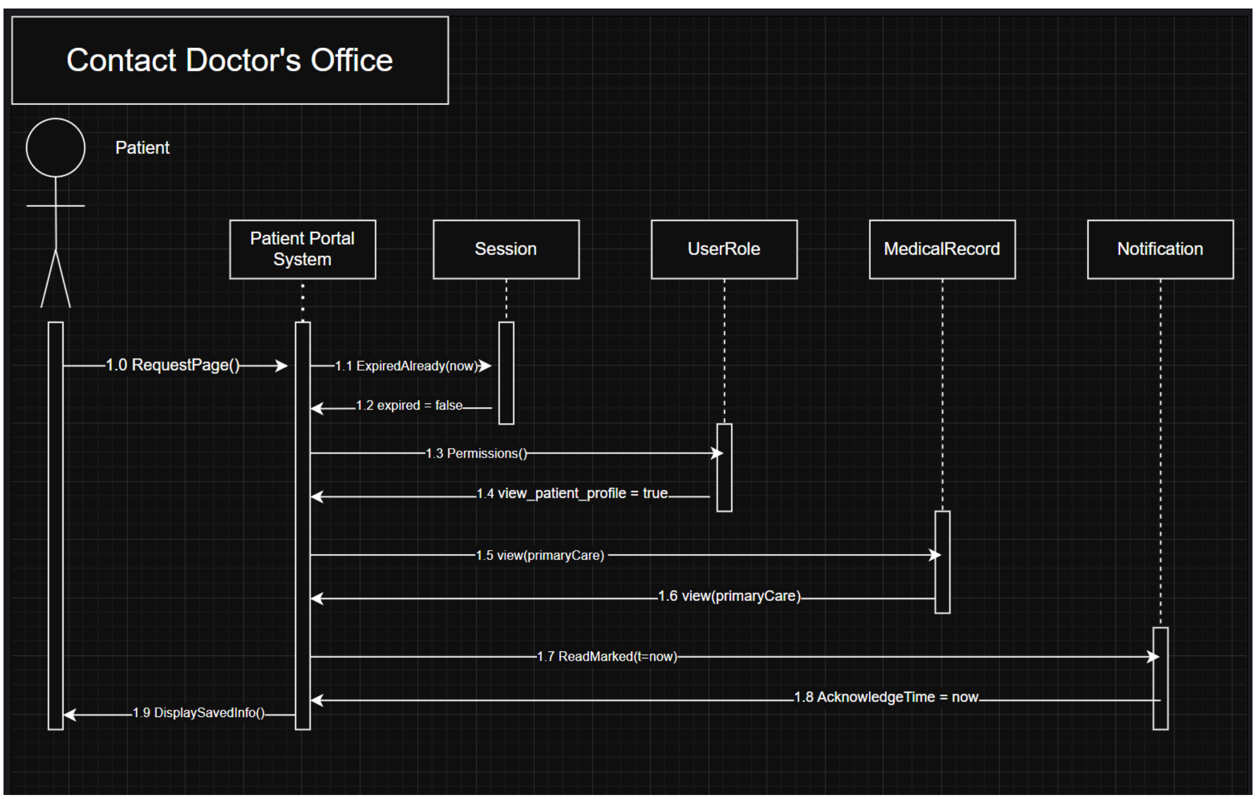

System Notification Read (Use Case 44)
Patient
Patient Portal System
Session
UserRole
Notification
1.0 SelectNotifications()
1.1 expiredAlready(now)
1.2 expired = false
1.3 permissions
1.4 allow_notifications = true
1.5 UnseenNotifications(Patient_ID)
1.6 unseenList(dict)
1.7 markRead()
1.8 acknowlwdgedOtime = true
1.9 DisplayNotificationList(unseenList)

Michael:

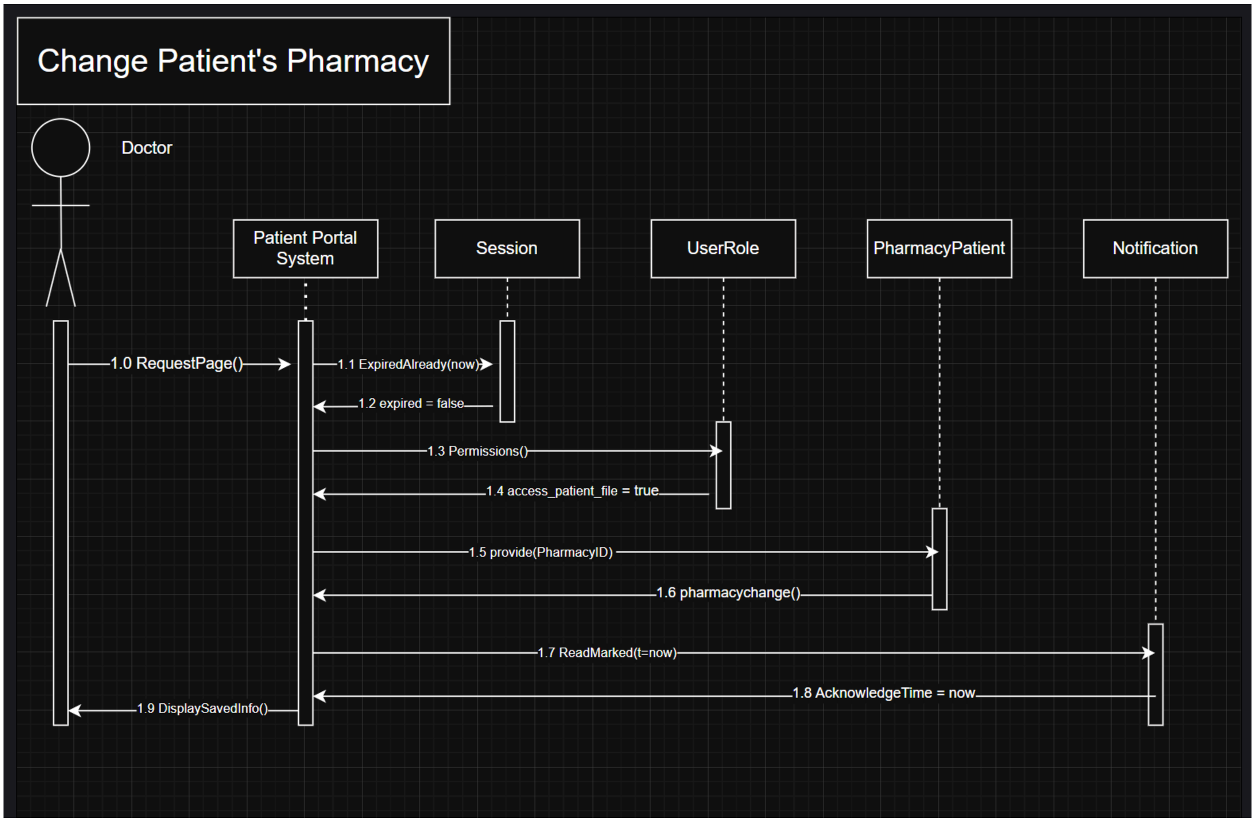
Refill Medication:



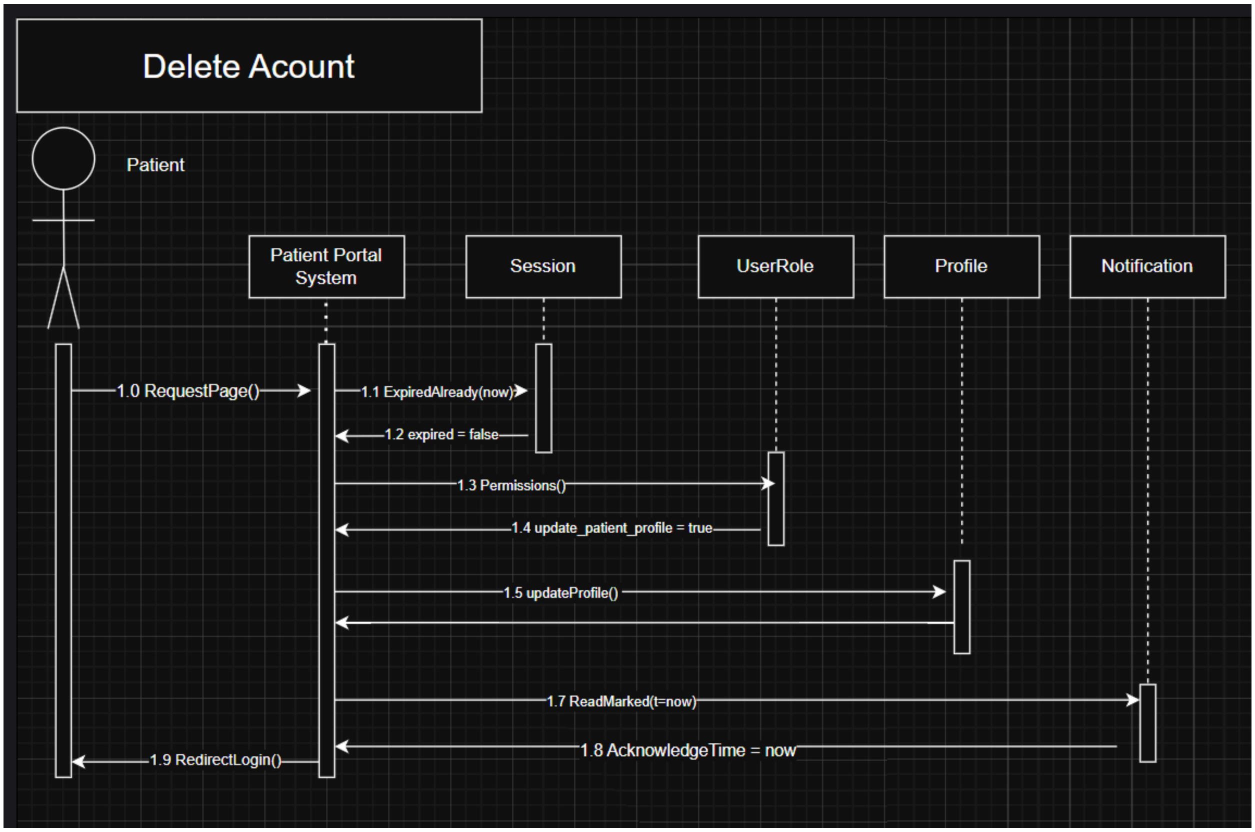
Contact Doctor's Office:



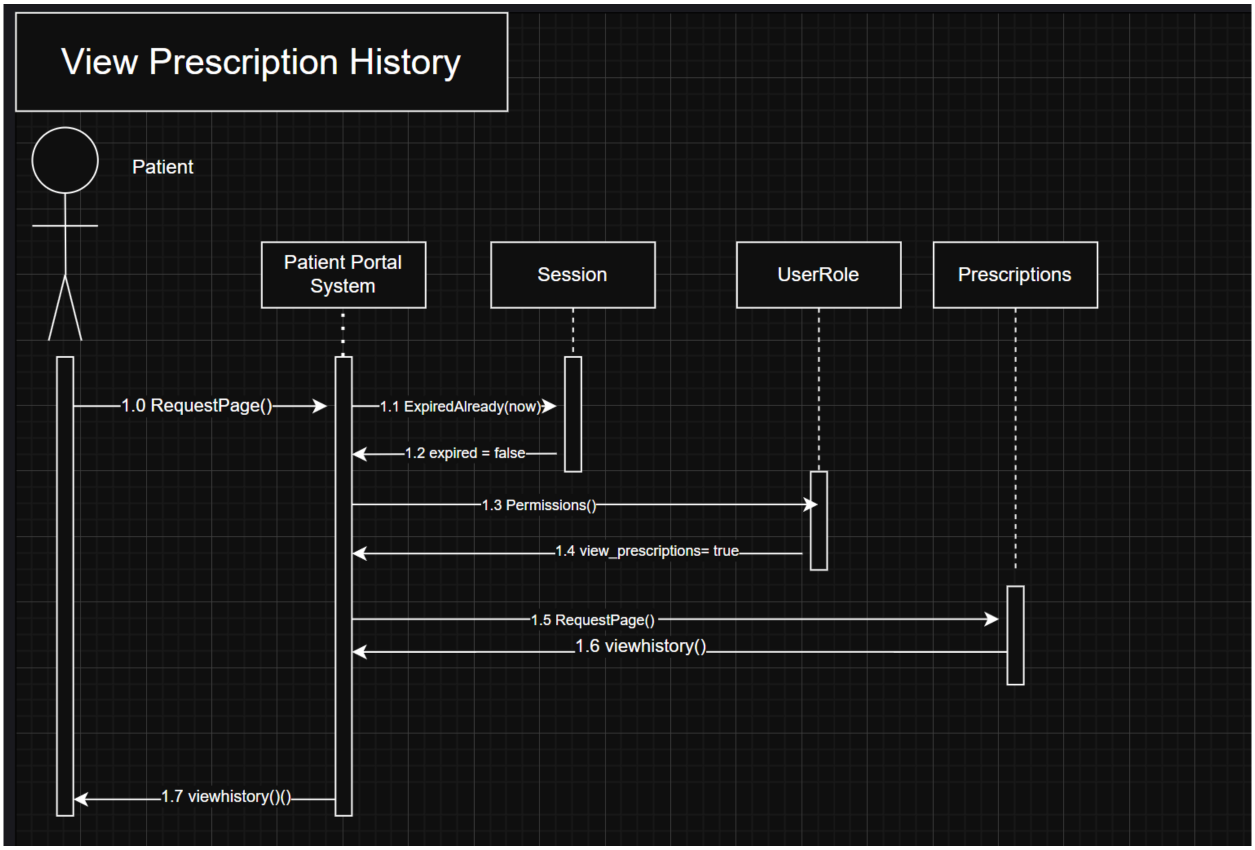
Change Patient's Pharmacy:



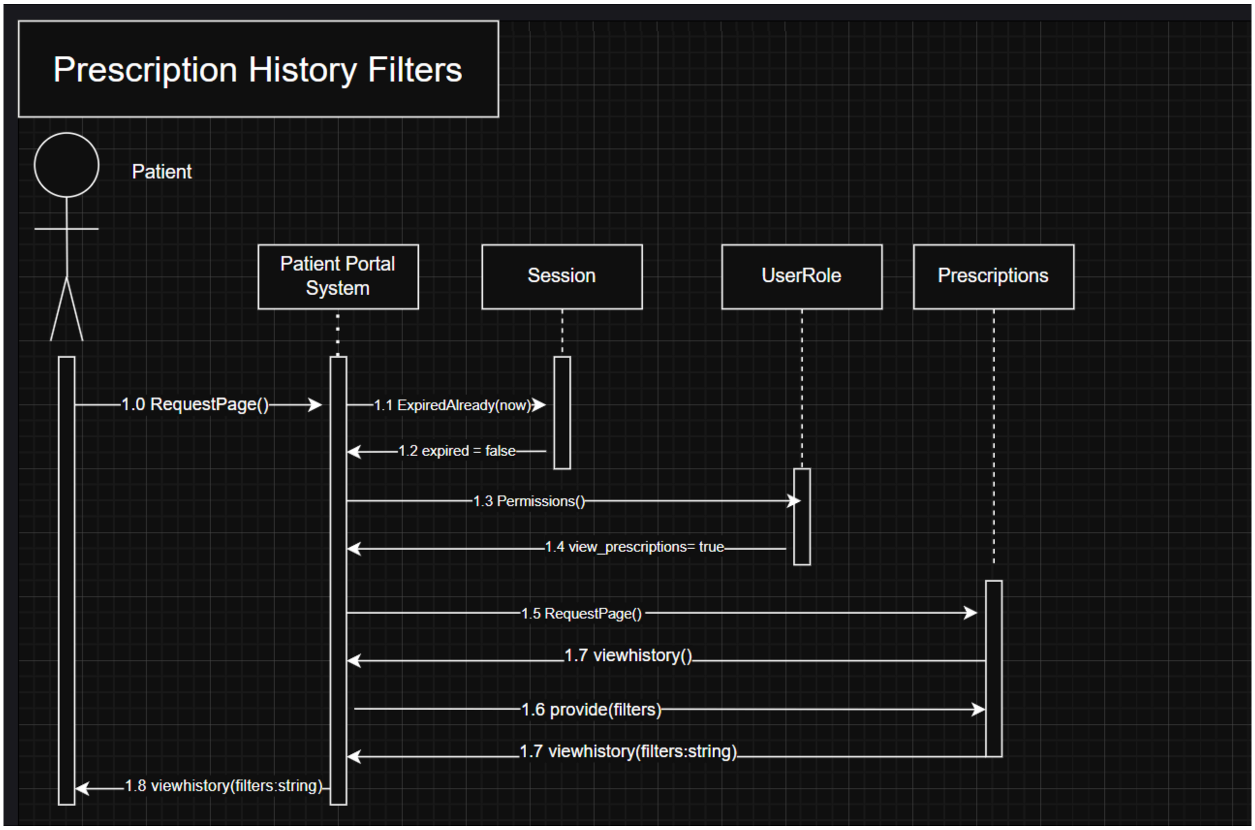
Delete Account:



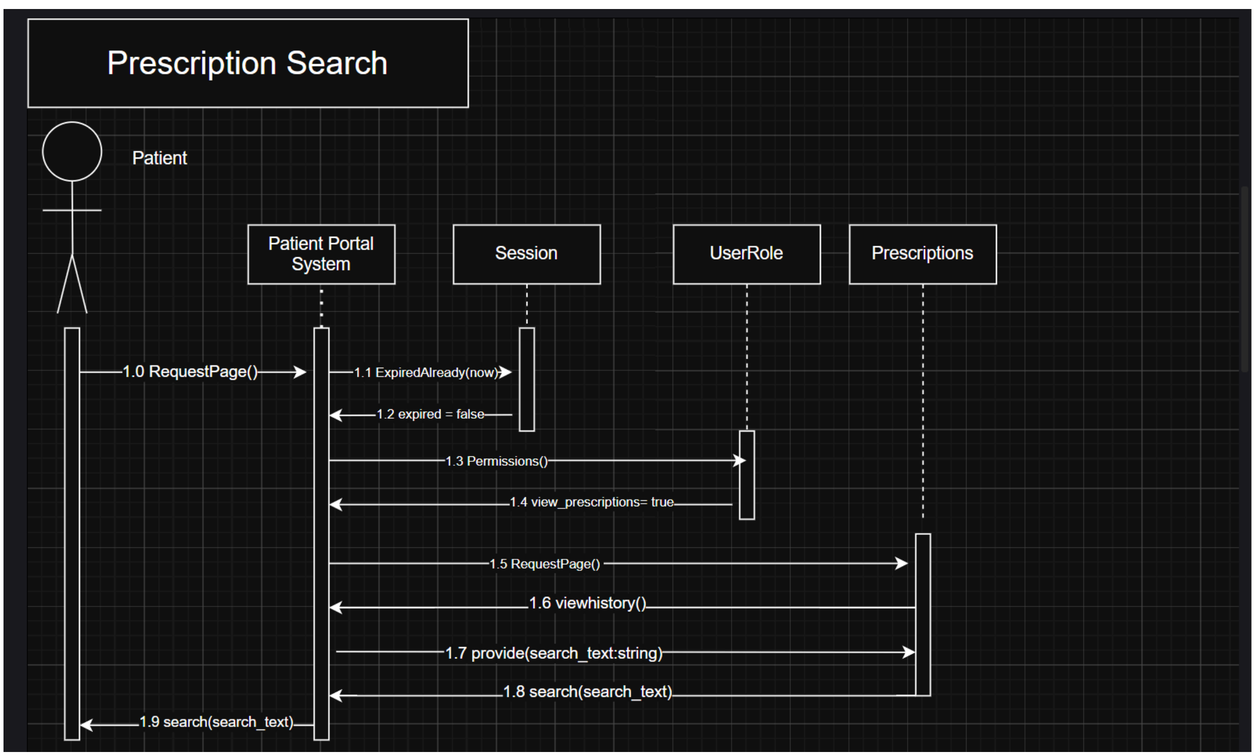
View Prescription History:



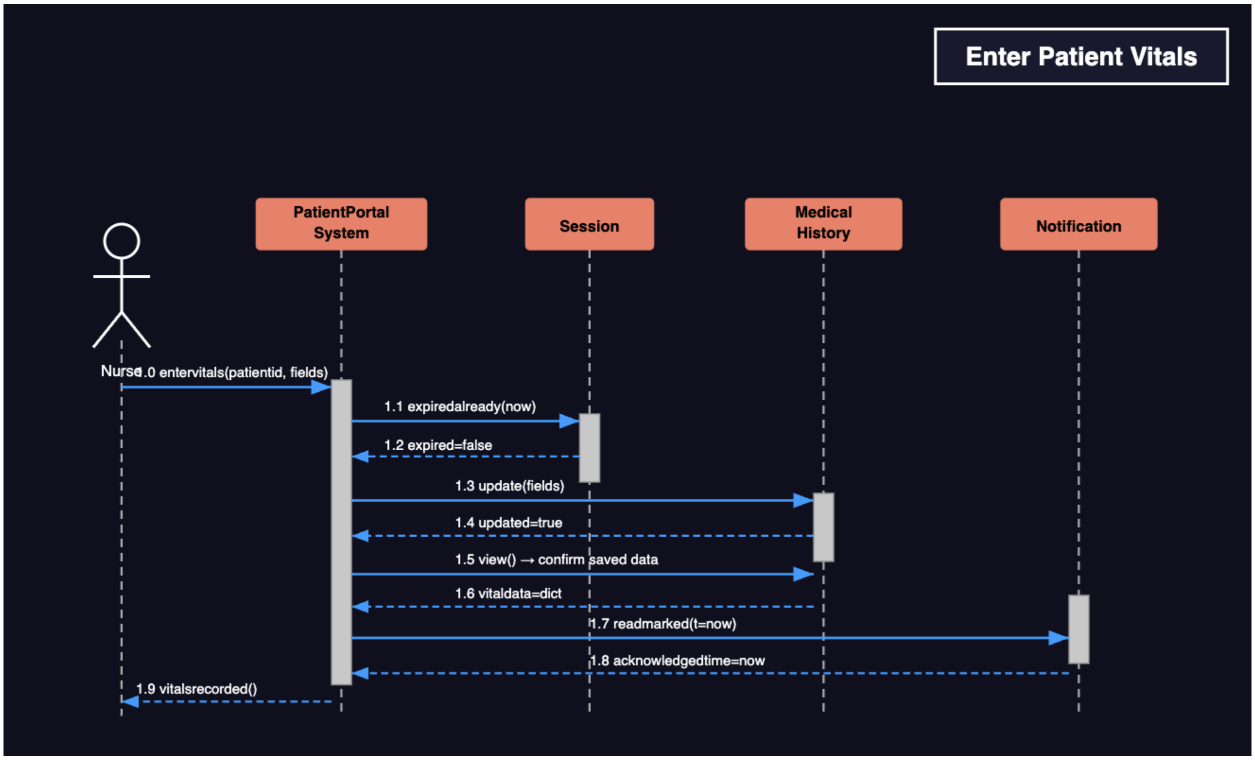
Prescription History Filters:



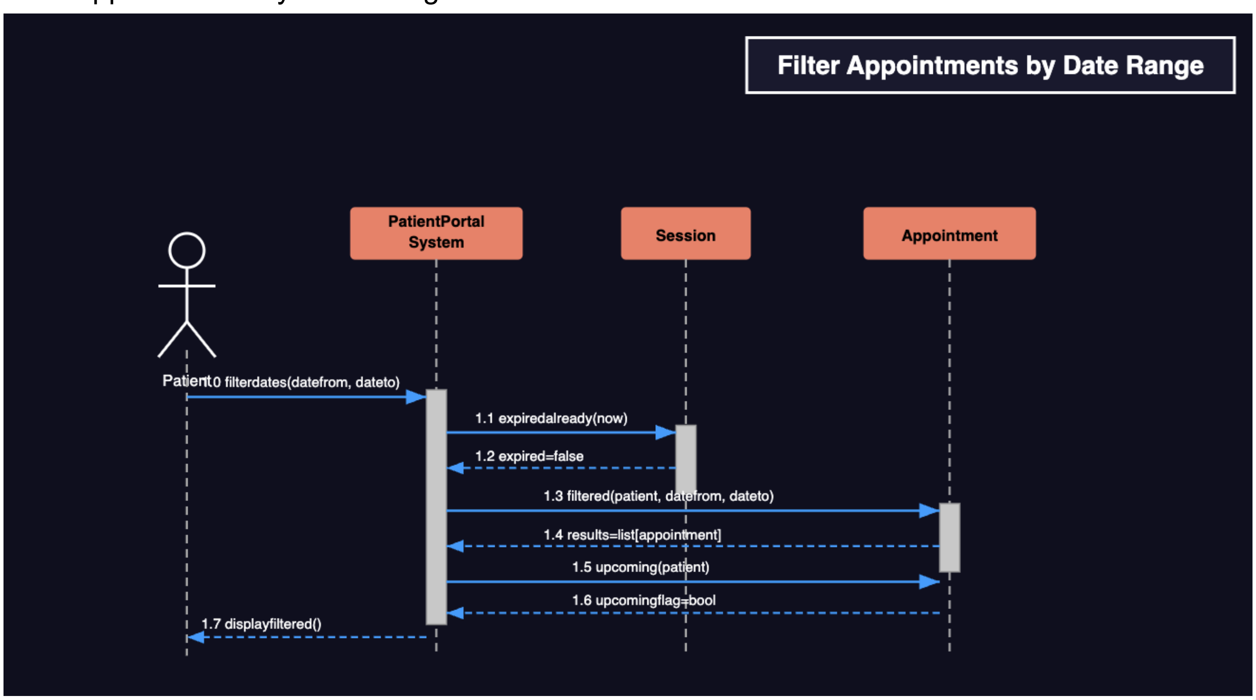
Prescription Search:



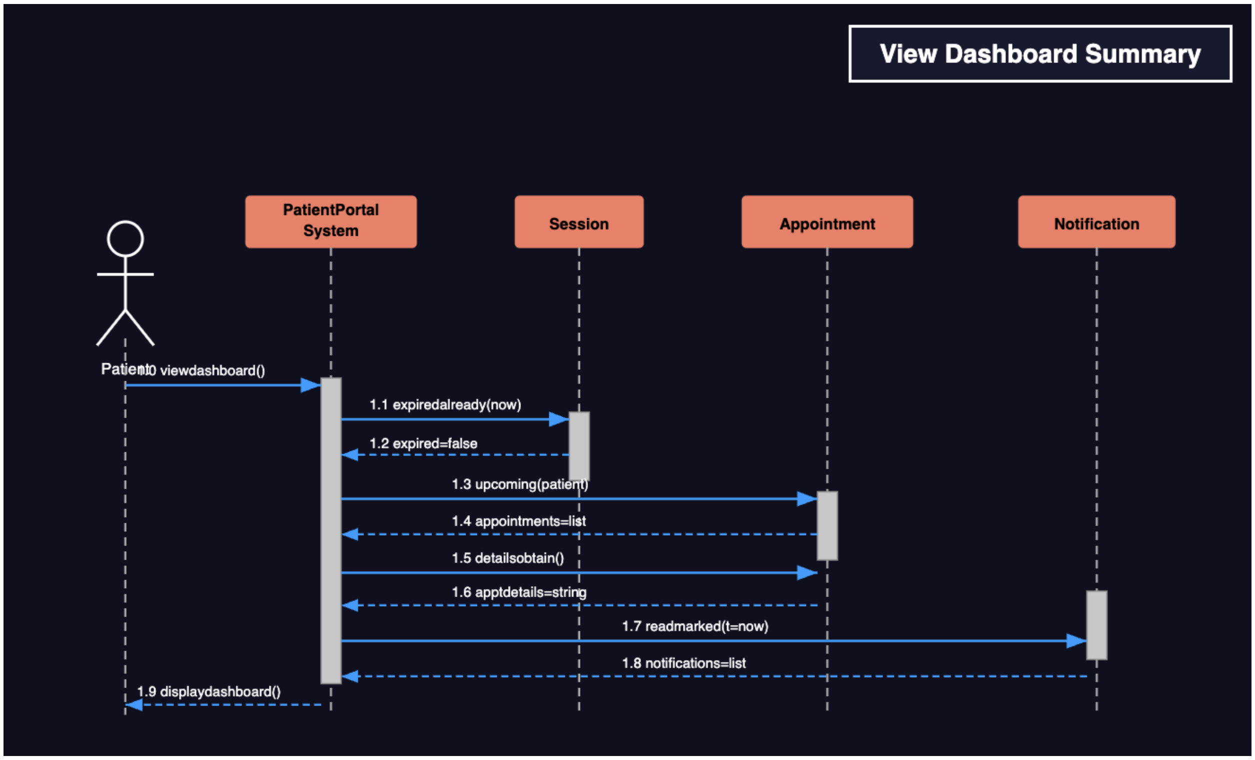
Enter Patient Vitals:



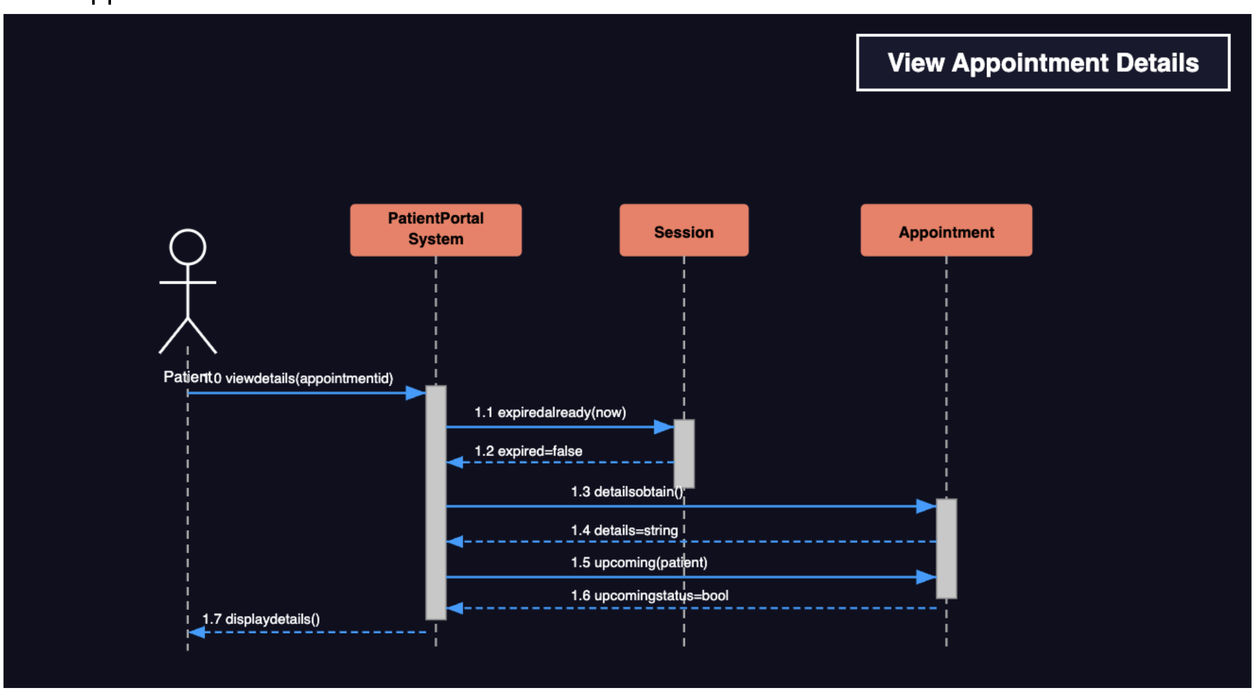
Filter Appointments by Date Range:



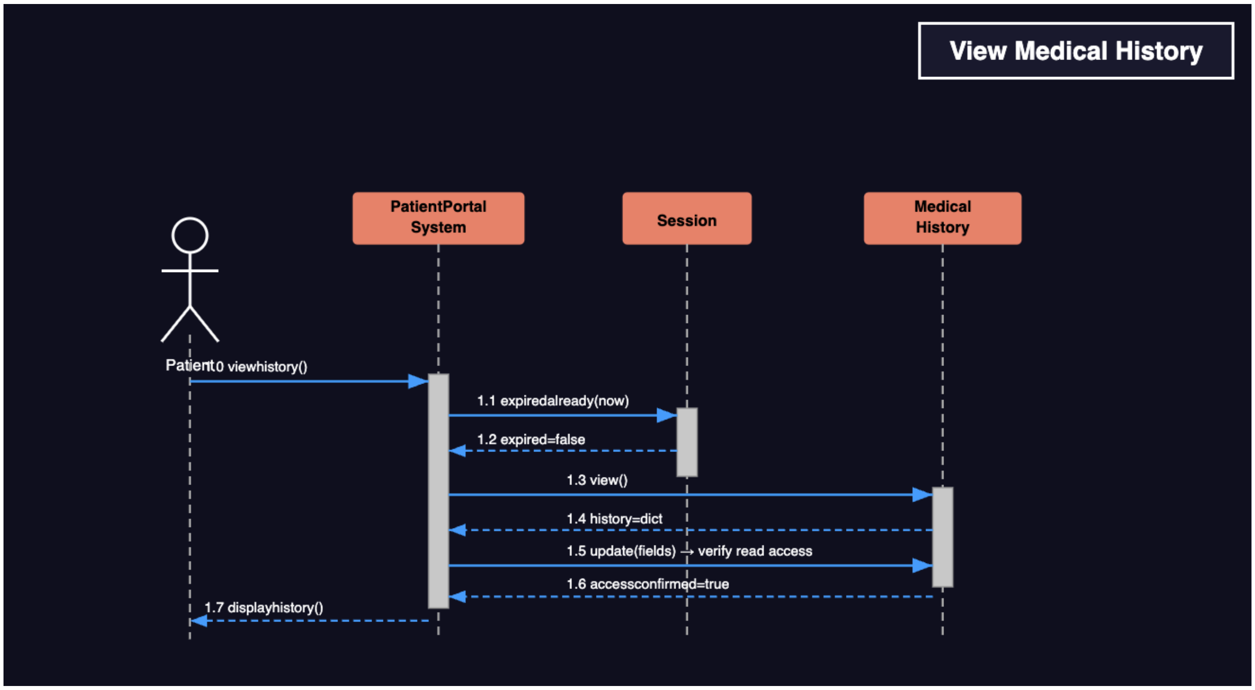
View Dashboard Summary:



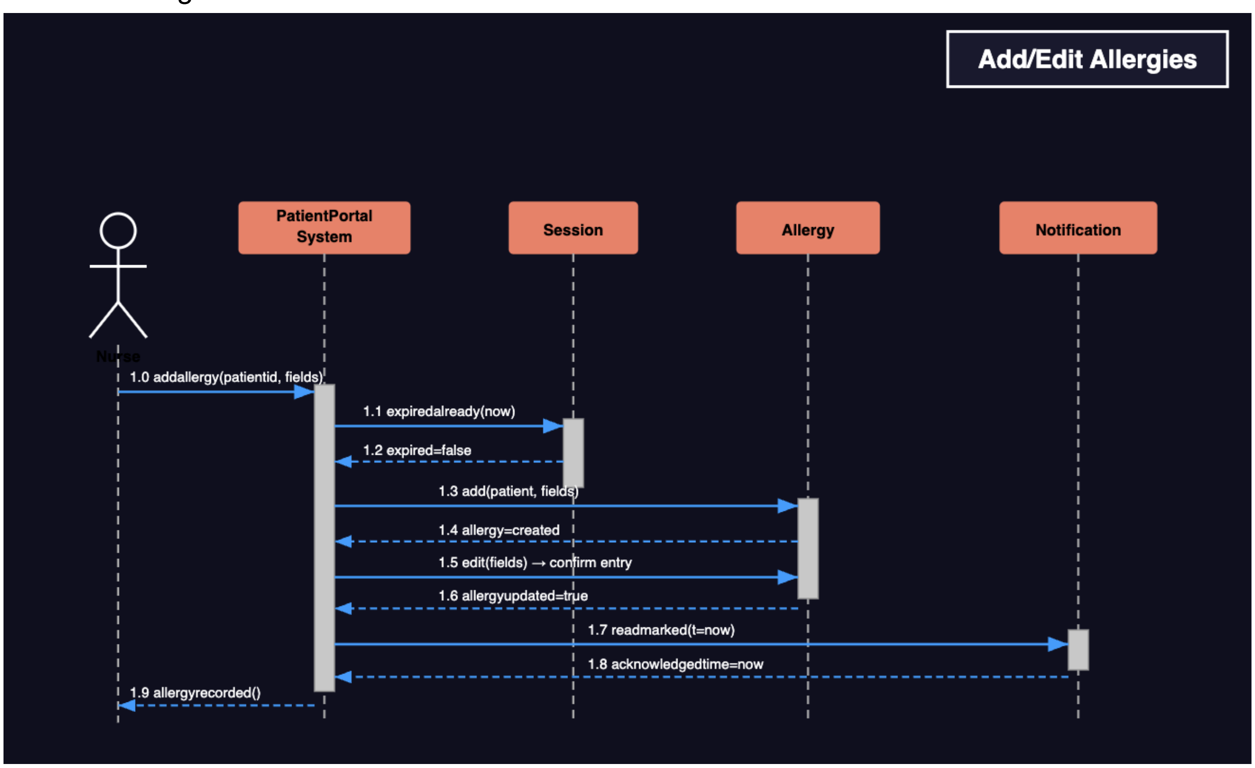
View Appointment Details:



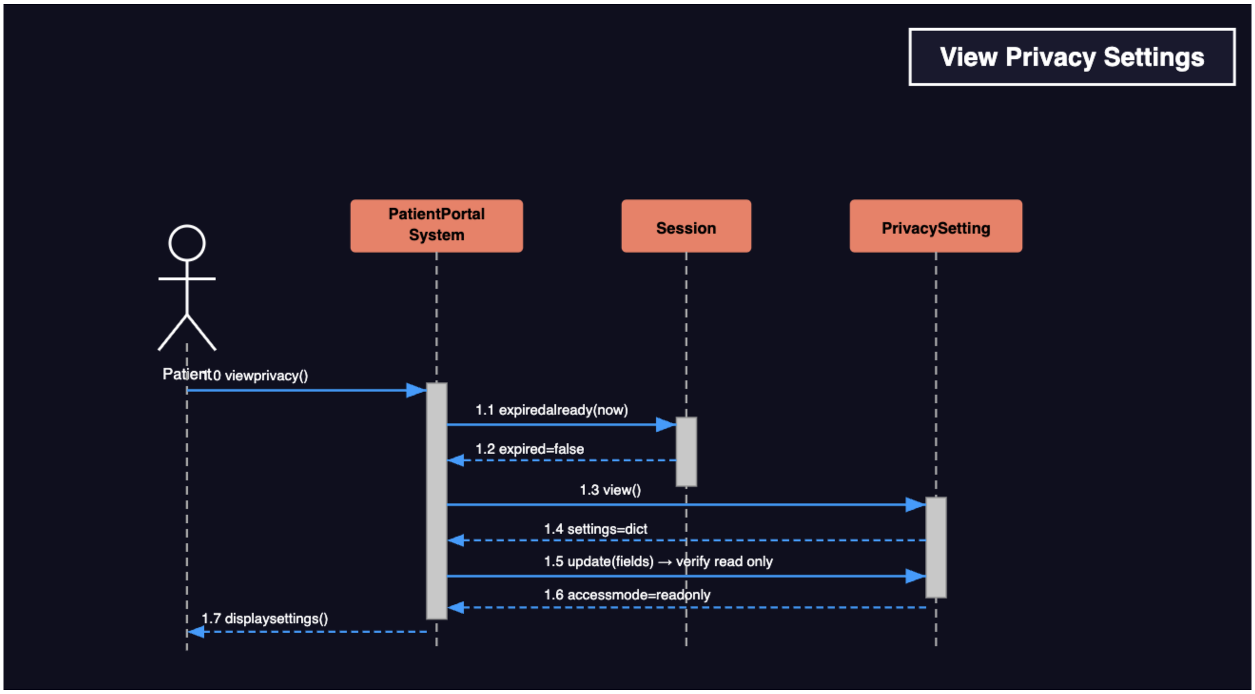
View Medical History:



Add/Edit Allergies:

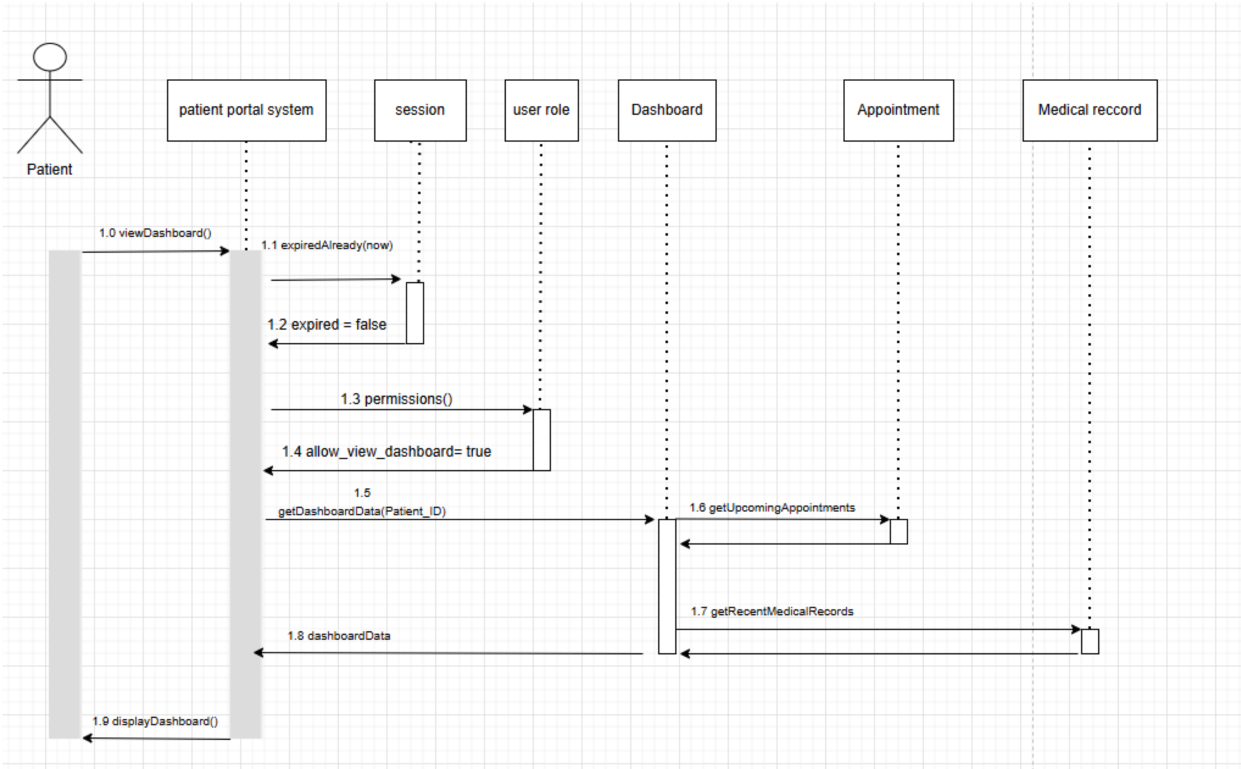


View Privacy Settings:

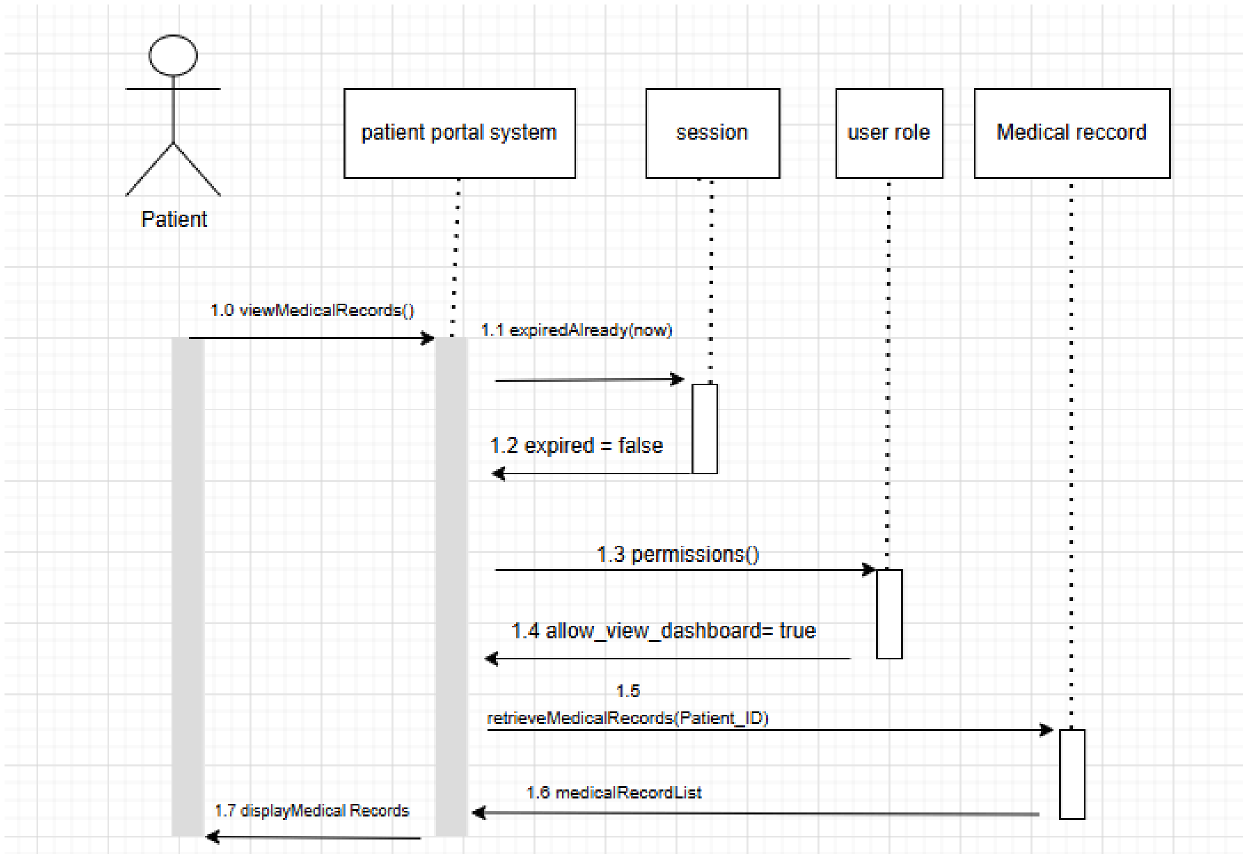


Keshon:

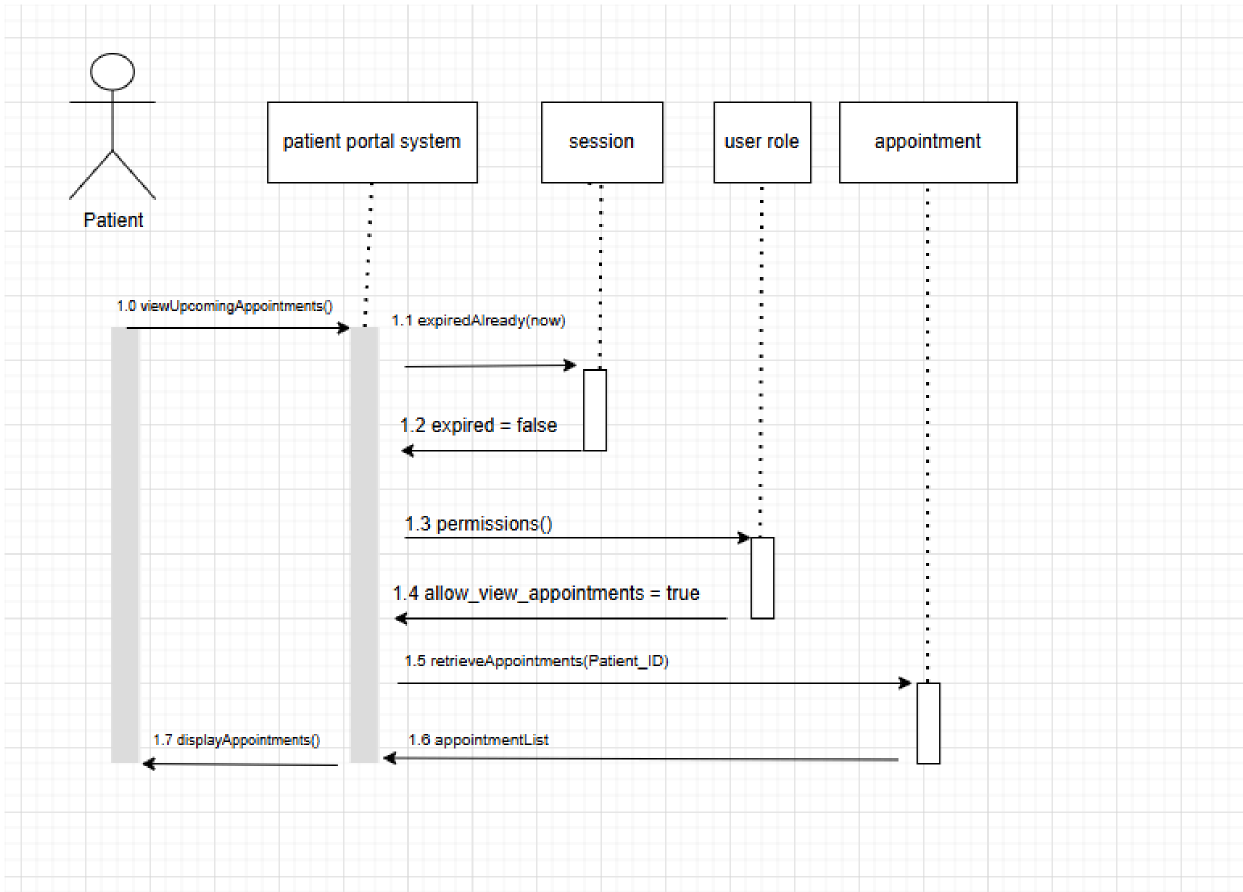
View Patient Dashboard



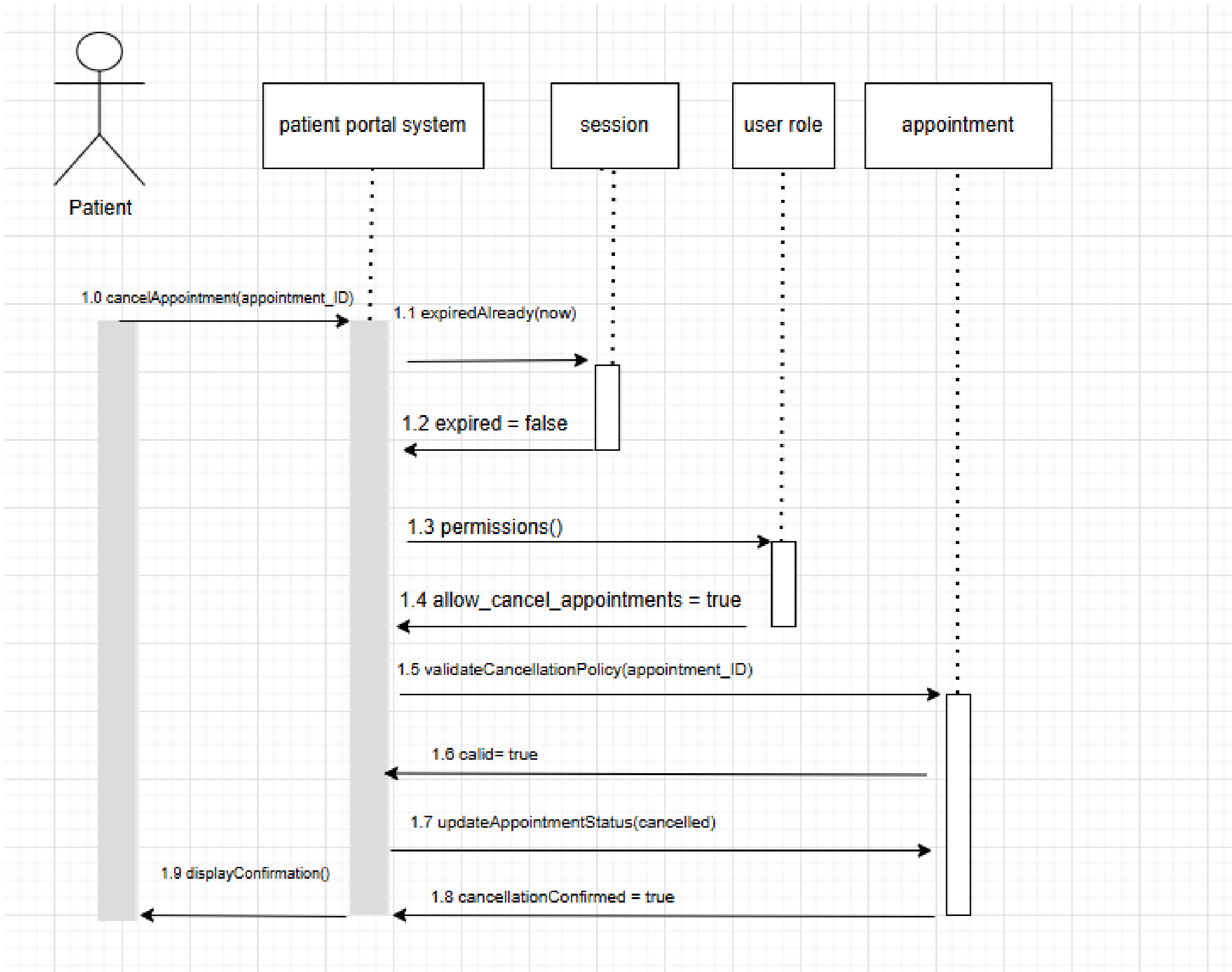
View Medical Records



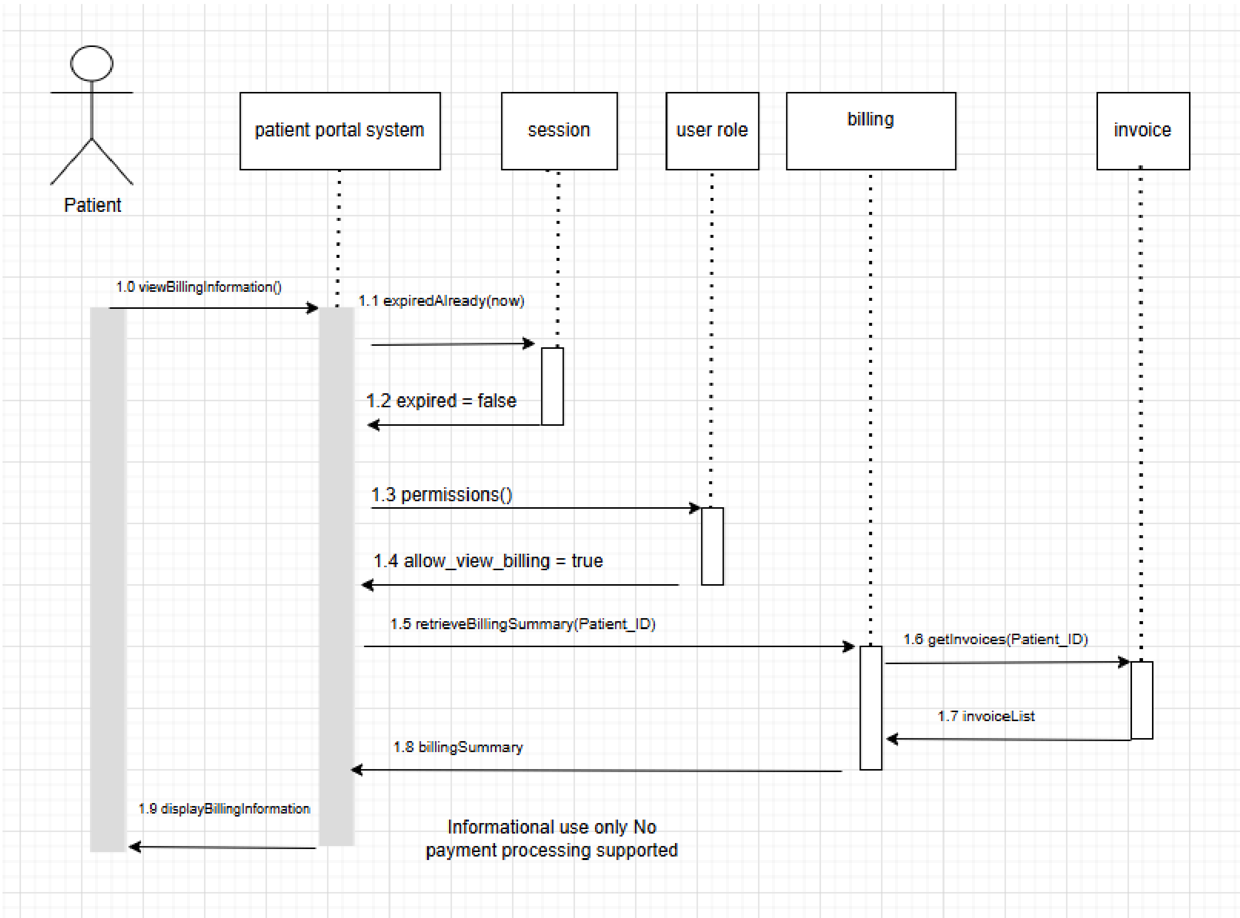
View Appointments



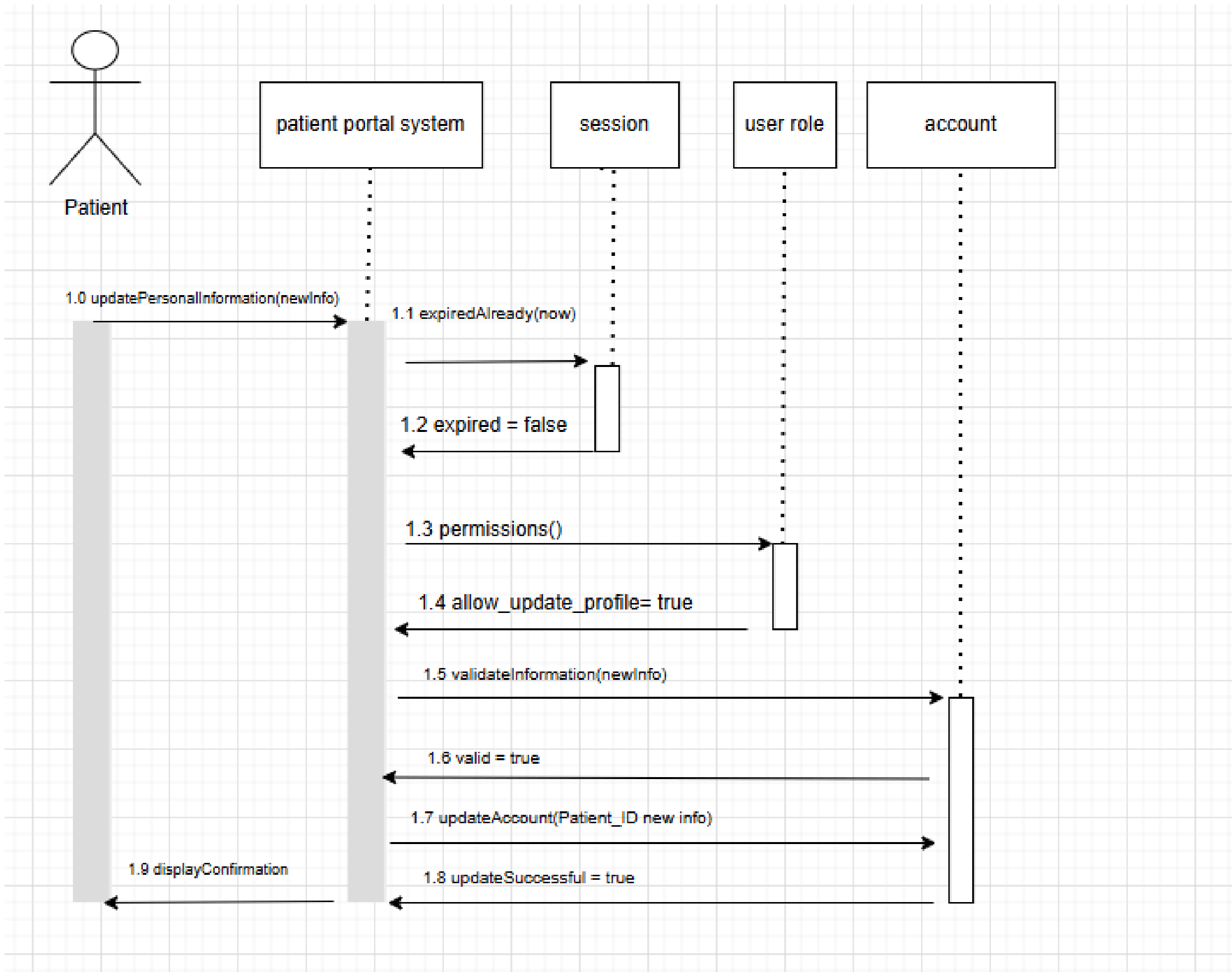
Cancel Appointment



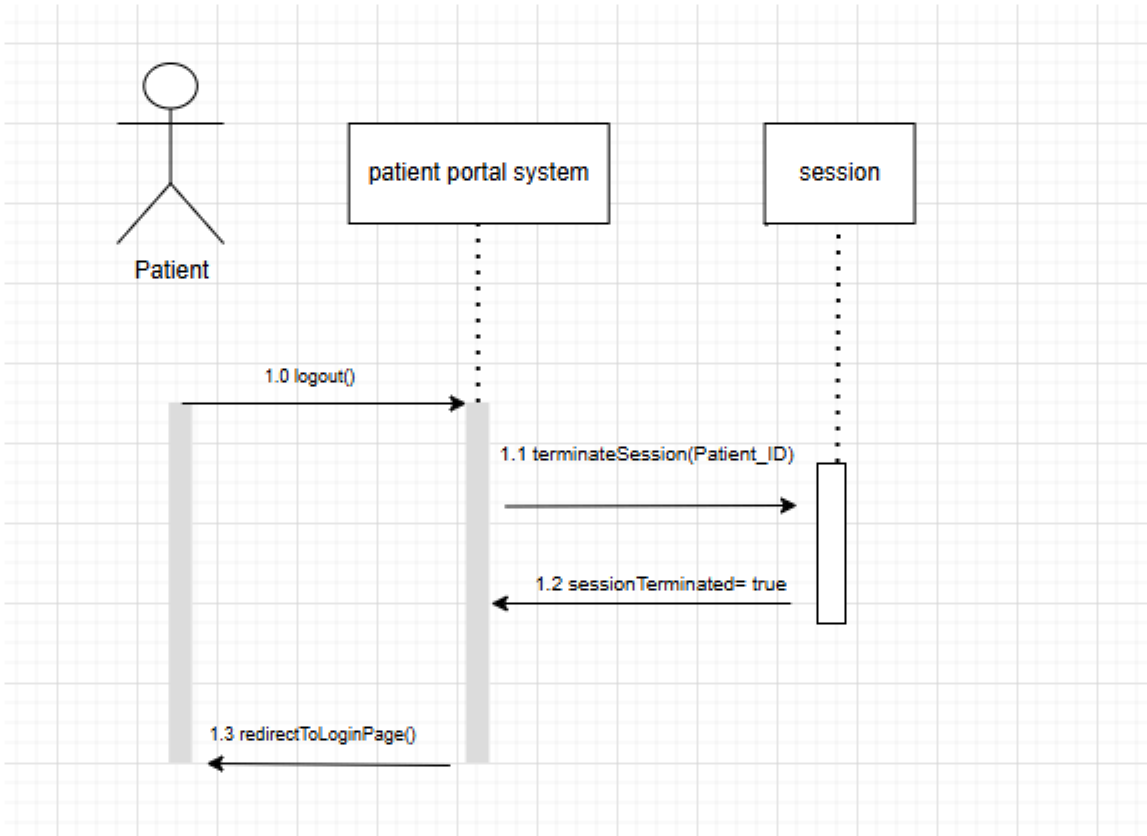
View Billing Information



Update Personal Information



Log out of Portal



3.2 External Interface Requirements

3.2.1 User Interfaces

The PPS will be using a web-based user interface that can be accessed through a standard web browser. The interface provides screens for user login, account registration, appointment scheduling and management, profile updates, and secure messaging. The layout is supposed to be clear and easy to navigate so that users can complete tasks with no confusion. Also, a clear message will be provided to the user for visual feedback on their actions, whether that is successful or when errors occur.

3.2.2 Hardware Interfaces

The PPS does not require any specialized hardware to work. Users will be able to access the system using desktop computers, or devices that have internet access. The system runs through a web browser, allowing patients and staff members to interact with the portal using common hardware.

3.2.3 Software Interfaces

The PPS will interact with common web browsers such as Google Chrome, Microsoft Edge, Safari, and Firefox to provide access to the system. The portal will also connect to the backend database and server software to store and recover patient information, appointment records, and user account details. There will be a secure communication process that will be used to make sure transmitted data stays protected.

3.3 Non-Functional Requirements of the Patient Portal

The Patient Portal System will provide secure authentication with bcrypt-hashed passwords while maintaining protected user sessions plus role-based access control enforcement so users can only access authorized features and data. The system should be responsive and very intuitive to navigate across desktop screen sizes with a consistent interface that supports accessibility and user feedback. The portal has to maintain an acceptable performance for routine actions such as page redirections while preserving data and MySQL database integrity. As well, the system has to be maintainable and modular, separate components for views, permissions, authentications, notifications, and the database so future updates and testing can be performed much easier.

4 - Other Non-functional Requirements

4.1 Performance Requirements

The Patient Portal System will provide response times deemed acceptable for common user interactions, such as login/logout, page redirections, loading data, and submitting information. With a small-scale deployment, most of the pages and form submissions should be completed within a few seconds (this assumes the setup is on a stable network connection and the database + Django are properly configured). Database queries should be efficient enough to support routine patient activity without any noticeable delay. The system should also handle sequential user requests without corrupting the session data.

4.2 Software Quality Attributes

The Patient Portal System should have important quality in its software attributes (usability, modularity, maintainability, reliability, and the buzzwords Chowdhury wants). Usability is supported through the consistent interface and page organization for common tasks. The maintainability is improved in the modular project organizations (such as the views.py and RBAC logic, or the HTML page templates). Reliability is required for the system to behave in a predictable manner, especially when a new user is logging in and there are data updates that need to be preserved with their integrity in the database.

5 – Project Backlog

This backlog tracks development, testing, and documentation for the Herd Resource Patient Portal using Agile sprints. By assigning priority levels, story points, and completion statuses to each task, the team can effectively monitor project momentum.
This is for the code, documentation of the program's version and what changed.

ID	Task Name	Sub-Task Name	Priority	Status	Story Points	Complete
	Sprint 1	-		FINISHED		YES
1	Developing Use Cases, Class Diagrams, Sequence Diagrams	-	HIGH	FINISHED	30	YES
1.1	-	Developing, Drawing, and Describing Use Case Diagrams	HIGH	FINISHED	10	YES
1.2	-	Developing the Project Backlog and Gantt Chart	HIGH	FINISHED	6	YES
1.3	-	Drawing and Describing Class Diagram	HIGH	FINISHED	6	YES
1.4	-	Developing and Drawing Sequence Case Diagrams	HIGH	FINISHED	8	YES
	Sprint 2	-		FINISHED		YES
2	Core Backend Development	-	HIGH	FINISHED	86	YES
2.1	-	Database Schema Design	HIGH	FINISHED	10	YES
2.2	-	User Registration Backend Logic	HIGH	FINISHED	9	YES
2.3	-	User Authentication and User Handling	HIGH	FINISHED	9	YES
2.4	-	Password Reset and Token Handling	HIGH	z	8	NO
2.5	-	Role-Based Access Control Logic	HIGH	FINISHED	12	YES
2.6	-	Appointment Management	HIGH	FINISHED	YES	NO

		Backend				
2.7	-	Profile Information API Endpoints	HIGH	NO PROGRESS	8	NO
2.8	-	Messaging System Backend	MEDIUM	FINISHED	6	YES
2.9	-	Notification Implementation	MEDIUM	FINISHED	6	YES
2.10	-	Session Management and Timeout	HIGH	FINISHED	6	YES
2.11	-	Data Validation and Error Handling Backend	MEDIUM	FINISHED	6	YES
	Sprint 3	-		FINISHED		YES
3	Frontend Development	-	HIGH	FINISHED	70	YES
3.1	-	Registration and Logout UI	HIGH	FINISHED	6	YES
3.2	-	Dashboard Layout	HIGH	FINISHED	8	YES
3.3	-	Profile View and Edit View	HIGH	FINISHED	6	YES
3.4	-	Appointment View UI	MEDIUM	FINISHED	5	YES
3.5	-	Messaging and Notification UI	MEDIUM	FINISHED	5	YES
3.6	-	Error Messages and Value Feedback	MEDIUM	FINISHED	4	YES
3.7	-	Define External Interface Requirements	HIGH	FINISHED	6	YES
3.8	-	Document User interface Screens and Navigation Flow	MEDIUM	FINISHED	4	YES
3.9	-	Specify Browser and Software interface Compatibility	MEDIUM	FINISHED	4	YES
3.10	-	Appointment Calendar View UI	MEDIUM	FINISHED	5	YES
3.11	-	Password Reset/Recovery UI	MEDIUM	ABANDONED	4	NO

3.12	-	Notification Display	MEDIUM	FINISHED	5	YES
	Sprint 4	-		IN PROGRESS		NO
4	Testing and Finalization	-	HIGH	IN PROGRESS	47	NO
4.1	-	Unit Testing (Authentication, Profile, Appointments)	HIGH	IN PROGRESS	8	NO
4.2	-	Integration Testing	HIGH	IN PROGRESS	8	NO
4.3	-	Bug Fixing and Requirement	MEDIUM	IN PROGRESS	6	NO
4.4	-	Final Documentation Updates	MEDIUM	IN PROGRESS	4	NO
4.5	-	Final Demo Preparation	MEDIUM	NO PROGRESS	4	NO
4.6	-	User Acceptance Testing	MEDIUM	IN PROGRESS	6	NO
4.7	-	Performance and Session Testing	MEDIUM	IN PROGRESS	5	NO
4.8	-	Security Testing and Access Control Testing	HIGH	IN PROGRESS	6	NO

Gantt Chart:

[Note: Blank Spaces in Assignees are “All”, For Some Reason Google Sheets Will blend “All” Into the background’s color (if you squint your eye and look real closely, you can see “All”)]

			Date	02/02/2026	02/09/2026	02/16/2026	02/23/2026	03/02/2026	03/09/2026	03/16/2026	03/23/2026	03/30/2026	04/06/2026	04/13/2026	04/20/2026
Sprint Phase	Task	Assignees	Duration	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12
Sprint 1		All	3 Weeks								SPRING BREAK				
	Developing, Drawing, and Describing Use Case Diagrams	All	2 Weeks								SPRING BREAK				
	Developing the Project Backlog and Gantt Chart	All	2 Weeks								SPRING BREAK				
	Drawing and Describing Class Diagram	Colton & Jeff	1.5 Weeks								SPRING BREAK				
	Developing and Drawing Sequence Case Diagrams	All	2 Weeks								SPRING BREAK				
Sprint 2		All	3 Weeks								SPRING BREAK				
	Database Schema Design	All	1 Week								SPRING BREAK				
	User Registration Backend Logic	Colton	1 Week								SPRING BREAK				
	User Authentication and User Handling	Michael, Kendra, Keshon	2 Weeks								SPRING BREAK				
	Password Reset and Token Handling	Colton	1 Week								SPRING BREAK				
	Role-Based Access Control Logic	Colton	1 Week								SPRING BREAK				
	Appointment Management Backend	Kiru	1 Week								SPRING BREAK				
	Profile Information API Endpoints	Jeff	2 Weeks								SPRING BREAK				
	Messaging System Backend	Kiru	1 Week								SPRING BREAK				
	Notification Implementation	Keshon	1 Week								SPRING BREAK				
	Session Management and Timeout	Michael	1.5 Weeks								SPRING BREAK				
	Data Validation and Error Handling Backend	Kendra	1.5 Weeks								SPRING BREAK				
Sprint 3		All	3 Weeks								SPRING BREAK				
	Registration UI	Jeff	1 Week								SPRING BREAK				
	Dashboard Layout	Jeff	1 Week								SPRING BREAK				
	Profile View and Edit View	Colton & Jeff	1 Week								SPRING BREAK				
	Appointment View UI	Michael	1.5 Weeks								SPRING BREAK				
	Messaging and Notification UI	Jeff	1 Week								SPRING BREAK				
	Error Messages and Value Feedback	Colton	1 Week								SPRING BREAK				
	Define External Interface Requirements [3.2]	Jeff	1 Week								SPRING BREAK				
	Specify Browser and Software Interface Compatibility [3.2.2 & 3.2.3]	Jeff	0.5 Week								SPRING BREAK				
	Document User Interface Screens and Navigation Flow [3.2.1]	Jeff	0.5 Week								SPRING BREAK				
	Appointment Calendar View UI	Kendra	1.5 Weeks								SPRING BREAK				
	Password Reset/Recovery UI	Keshon	1 Week								SPRING BREAK				
	Notification Display	Kiru	2 Weeks								SPRING BREAK				
Sprint 4		All	2 Weeks								SPRING BREAK				
	Unit Testing (Authentication, Profile, Appointments)	Kiru	1 Week								SPRING BREAK				
	Integration Testing	Jeff	1 Week								SPRING BREAK				
	Bug Fixing and Requirement	Michael	1 Week								SPRING BREAK				
	Final Documentation Updates	All	0.5 Week								SPRING BREAK				
	Final Demo Preparation	All	0.5 Week								SPRING BREAK				
	User Acceptance Testing	Kendra	0.5 Week								SPRING BREAK				
	Performance and Session Testing	Keshon	0.5 Week								SPRING BREAK				
	Security Testing and Access Control Testing	Colton	0.5 Week								SPRING BREAK				

Appendix – Group Log

Name	Date	Contributions
Colton Gebhard	1/17/2026	Created initial SRS document template and formatting layout. Authored Section 1.1 (Document Purpose), Section 1.2 (Product Scope), Section 1.3 (Document Overview and Intended Audience), Section 1.4 (Definitions and Acronyms), and Section 1.5 (Document Conventions). Drafted portions of Section 2.2 (Product Functionality), including User Registration and Account Creation and User Authentication. Contributed to Section 2.3 (User Characteristics), including Security Awareness, Accessing Requirements, Cultural Language, and Frequency of Usage. Drafted portions of Section 2.4 (Design and Implementation Constraints), including Visibility of Data, Access Restrictions, and Scope of Appointment Managing.
Jeffrey Jezewski	1/17/2026	Written 3 Use Cases (#1 - #3)
Colton Gebhard	1/18/2026	Drafted portions of Section 2.4 (Design and Implementation Constraints), including Authentication Framework Constraints, Database Constraints, Logging Limitation, Security Limitations, and Technology Constraints. Revised Section 2.3 (User Characteristics), including Technical Experience and Expected Behavior of Users. Updated Section 2.4 (Payment and Billing). Revised Section 2.5 (Assumptions and Dependencies) for clarity and consistency.
Colton Gebhard	1/19/2026	Drafted more descriptions in Section 2.2 (Product Functionality) for Password Management and Session Management. Authored Section 2.4 subsection detailing Deviations from the Final Project SRS baseline.
Michael Davis	1/20/2026	Written 2.2 (Product Functionality) [Viewing Personal Information, Updating Personal Information, Error Handling] Grammar fix for 2.2 (Product Functionality) [Session Management]
Kendra Adkins	1/20/2026	Helped work on section 2.2 [Viewing Personal

		Information], and started brainstorming on my Use Cases
Colton Gebhard	1/20/2026	Formatted Jeffrey's 3 Use Cases into a Table Grid Format Added Use Case Tables Retweaked Section 2.2 (Product Functionality) [Viewing Personal Information] Removed Herobrine
Colton Gebhard	1/21/2026	Retweaked Section 2.2 (Product Functionality) [Viewing Appointments and Schedules] Added 10 new use cases (only names)
Colton Gebhard	1/22/2026	Applied Headings, Added Table of Contents Filled In 6 use cases [#11 to #16]
Colton Gebhard	1/25/2026	Filled in the remainder Use Cases [#17 to #20] Created Section 5's Project Backlog Created Use Case Diagram (Draw IO), Class Diagram (Draw IO), and Sequence Diagram (Draw IO)
Kirubel Melaku Assefa	1/25/2026	Filled in Use Case #41, Use case #42, Use case #43, Use Case #44, Use Case #45, Use Case #46, Use Case #47, Use Case #48, Use Case #49, Use Case #50
Jeffrey Jezewski	1/25/2026	Started on Class Diagram
Michael Davis	1/25/2026	Created titles for Use Cases #31-40
Kendra Adkins	1/25/2026	Created titles for Use Cases #21-30, and started on my first 2 Use Case descriptions (not fully completed)
Kendra Adkins	1/26/2026	Worked on Use Cases #23-26
Kendra Adkins	1/28/2026	Worked on Use Cases #27-28
Kendra Adkins	1/29/2026	Worked on Use Cases #29-30
Keshon Bryant	1/30/2026	Worked on and published Use cases #51-60
Michael Davis	1/30/2026	Completed Use Cases #31-40
Colton Gebhard	1/31/2026	Tweaked/Grammar-Checked Use Cases #51-60 Finish use case diagrams portion (10 contributions) Started on Class Diagrams
Kendra Adkins	2/1/2026	Completely finished all 10 Use Case contributions and started on Use Case Diagram

Jeffrey Jezewski	2/1/2026	Completed class diagram portion in draw io
Colton Gebhard	2/1/2026	Extended Class Diagram in draw io Assisted & Contributed to Use Case Diagram
Kirubel Assefa	2/1/2026	Finished 10 Use Cases for Diagram
Colton Gebhard	2/3/2026	Majorly revamped the Use Case Diagram, added 2 sentences to the Use Case Diagram Description Merged Kiru's, Jeff's, and Kendra's use case diagrams into one
Michael Davis	2/3/2026	Updated Use Cases #31 & #33 from doctor's perspective
Keshon Bryant	2/3/2026	Updated Use Case Exceptions, Updated the Notes/issues, and added Includes to #55 and #54
Colton Gebhard	2/4/2026	Added additional definitions for UUID, BigINT, TinyINT, Signed INT Added +80% of the Class Diagram Added an additional 3 sentences to the use case diagram description
Keshon Bryant	2/4/2026	Fixed all of my use cases to make them align with the SRS document. Fixed my includes for each case and matched them with the correct actor.
Kendra Adkins	2/4/2026	Worked on changing my Use Case #29 to have the Nurse as the primary actor. Worked on changing use case #23 completely to "Enter Patient Vitals" to have another Nurse actor for variety.
Colton Gebhard	2/5/2026	Revamped Keshon's Use Case #51, #52, #57, #69 Merged Michael's Use Case Diagram into the group use case diagram Modified Includes and Excludes for #11, #12, #13, #15, #17, #19, #20 Use Cases
Kendra Adkins	2/5/2026	Got Use Case Diagram Description started with the first four sentences,
Jeffrey Jezewski	2/5/2026	I added my 4 sentences to the class diagram, also completed the first paragraph for Section 3.2.1
Colton Gebhard	2/6/2026	Modified Class Diagram, started on Sequence Diagrams
Kirubel Assefa	2/7/2026	Added to Section 5 (Project Backlog) by defining sprint goals, backlog items, priorities, and story point estimates.

Colton Gebhard	2/7/2026	Added & Finished Class Diagram Added 4 Sequence Diagrams - [Access Patient Files, Update Patient Files, Validate User, Session Timeout Handling]
Colton Gebhard	2/8/2026	Added 4 Sequence Diagrams - [View Message Details, Update Contact Information, Enter Personal Health Information, View Login History]
Jeffrey Jezewski	2/8/2026	Updated and cleaned the Use Case Diagram Completed Section 3.2.1 (User Interfaces) Completed Section 3.2.2 (Hardware Interfaces) Completed Section 3.2.3 (Software Interfaces) Added tasks to Sprint 3 (Frontend Development backlog)
Michael Davis	2/8/2026	Added 4 sentences to the Use Case Diagram Description
Kendra Adkins	2/8/2026	Added 5 sentences to the Use Case Diagram Description
Kendra Adkins	2/9/2026	Completed Section 5 - Project Backlog by adding 2.7-2.12, 3.10-3.12, and 4.6-4.8
Colton Gebhard	2/11/2026	Added 2 Sequence Diagrams - [Reset Password, Receive Appointment Reminders] Set-up Google Sheets Gantt Chart
Colton Gebhard	2/13/2026	Added to Gantt Chart (~1/10)
Colton Gebhard	2/14/2026	Added Class Diagram Description (18 sentences)
Colton Gebhard	2/15/2026	Added to Gantt Chart (~5/10) Modified Project Backlog (Removed a duplicate entry, split task 3.1 into two) Modified Class Diagram (Added Attributes) Removed Herobrine
Colton Gebhard	2/17/2026	Added Gantt Chart to SRS
Kendra Adkins	2/17/2026	Added 4 Sequence Diagrams
Kirubel Assefa	2/18/2026	Added 10 Sequence Diagrams
Kendra Adkins	2/18/2026	Added 6 Sequence Diagrams, Finished
Keshon Bryant	2/18/2026	Uploaded 10 Sequence Diagrams to the SRS document
Jeffrey Jezewski	2/18/2026	Finished Sequence Diagram

Michael Davis	2/22/2026	Added 10 Sequence Diagrams
Kendra Adkins	2/28/2026	Completed the Core Features, and Testing + Project Management slides on the PowerPoint
Colton Gebhard	3/28/2026	Added 3.3, 4.1, and 4.2 Modified 2.2, 2.4, and 2.5 to remove unimplemented features
Colton Gebhard	3/29/2026	Removed unimplemented use cases Modified 3.2.1, 3.2.2, and 3.2.3 to remove unimplemented features
Kendra Adkins	3/29/2026	Read through the use cases to remove unimplemented ones
Kendra Adkins	4/1/2026	Updated the README file with how to implement and run the program
Colton Gebhard	4/9/2026	Restructured Use Case and Sequence Diagrams Modified Use Case Diagram in deleting unused use cases Removed Herobrine

Group Meetings

Start Meeting Date/Time	End Meeting Time	Meeting Topics
01/25/2026 2:30pm	4:00pm	Starting on Use Cases with the group Discussion on Diagrams for Sections 3.1
01/31/2026 2:00pm	2:45pm	[Cancelled Meeting, Kiru and Jeff starting on use case diagram]
02/01/2026 4:30pm	6:00pm	Finishing up Use Cases Starting on Use Case Diagrams and Class Diagrams
02/08/2026 12:45pm	1:45pm	Finishing up Class Diagrams and description Starting on Sequence Diagrams. Working on Chapter 5 – Backlog
02/15/2026 1. 10:00am [Keshon, Colton] 2. 11:00am [Jeff,	10:30am 12:10pm	Finishing up Gantt Chart for Chapter 5 – Backlog Go Over Project Code Set Up Continue to work on Sequence Diagrams

Colton] 3. 5:30pm [Kiru, Michael, Kendra Colton]	7:00pm	
02/21/2026 1. 12:00pm [Jeffrey, Colton]	12:45pm	Starting on the Code Colton: MySQL [Database], Github Setup, Django Set-up, Python [.py] Jeffrey: Frontend [HTML files and Style Sheet]
02/22/2026 1. 12:00pm [Kendra, Michael, Colton]	12:55pm	Discuss with Michael about his Sequence Diagram Progress Starting on the Code Michael and Kendra: Python [.py] Started Michael and Kendra on coding redirecting pages for the prototype [login to dashboard, logout on any page to login, etc.]
02/28/2026 1. 12:00pm [Colton, Jeff, Kiru]	12:35pm	Work on prototype presentation slides Jeff: Get the Django Code Launched, Running, and Testing.
03/01/2026 1. 10:15am [Colton, Keshon] 2. 5:10pm [Colton, Kendra, Michael]	10:30am 5:35pm	Work on prototype presentation slides and go over them
03/08/2026 1. 11:00am [Colton, Kendra, Keshon (11:12am), Michael (11:32am)] 2. 6:35pm [Colton, Jeff, Kiru, Michael (6:50pm)]	11:45am 7:10pm	Group Presentation Review

03/28/2026 1. 11:30am	11:45am	[Canceled Meeting]
03/29/2026 1. 10:30am [Colton, Michael, Kendra]	11:15am	Going over the code Removing unimplemented use cases + Modifying SRS document Going over White-Box Testing [Template posted on Wednesday]
04/04/2026 1. 10:30am [Colton, Kiru, Michael]	11:00am	Starting on white box / unit testing Code development
04/05/2026 1. 10:25am [Colton, Jeff, Kendra (10:30)]	11:00am	Starting on white box / unit testing Code development